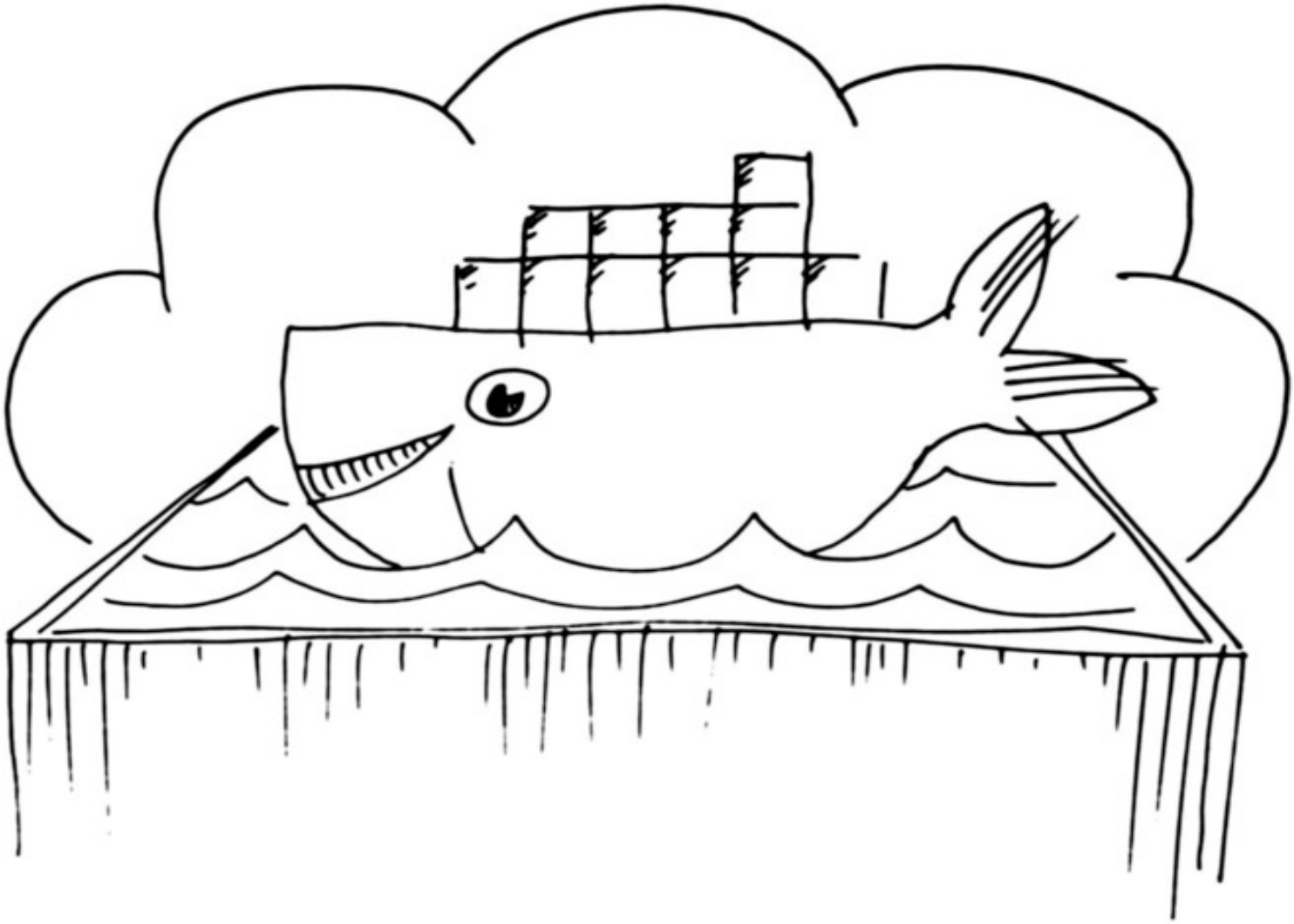
Валентин Юльевич Арьков

Виртуализация и контейнеризация

Учебное пособие



Валентин Юльевич Арьков

Виртуализация и контейнеризация

Учебное пособие

Шрифты предоставлены компанией «ПараТайп»

© Валентин Юльевич Арьков, 2022

Виртуальные машины и контейнеры — это современные технологии изоляции, которые решают проблемы совместимости, зависимости и развёртывания приложений. В учебном пособии рассматриваются примеры программных гипервизоров VirtualBox и Hyper-V, механизм песочницы Windows, а также Windows Subsystem for Linux и контейнеры Docker. Знакомство с целым спектром инструментов позволяет получить целостное представление об этом важном направлении информационных технологий.

12+

Создано в интеллектуальной издательской системе Ridero

ОГЛАВЛЕНИЕ

Виртуализация и контейнеризация
Предисловие
Введение
Традиционное выполнение программы
Изоляция приложений
- Динамические библиотеки
- Изоляция и администрирование
- Portable Applications

Виртуальные машины
- Виртуализация. Гипервизор — Hypervisor
- Виртуализация (Bare metal hypervisor)
- Гипервизор — Hypervisor
- Типы гипервизоров
- ЦОДы / Серверы / «Оркестрация»
- VirtualBox
- Hyper-V
- Windows + Hyper-V + Ubuntu
- Песочница Windows
- Общая папка / Mapped Folder
- Новый браузер в Песочнице
- Windows Subsystem for Linux (WSL)
- Microsoft Store — Ubuntu
- Microsoft Store — Windows Terminal
- Windows Terminal + Ubuntu
- Создание скрипта
- Ubuntu + Midnight Commander
- Программа Hello, World! на языке Си

- Microsoft Store — Linux
- Контейнеры
  - Контейнеризация
  - Контейнер для перевозки грузов
  - Установка Докера
  - Docker Desktop
  - Кто такой DOCKER?
  - Оркестрация
  - Микросервисы
  - Контейнеры Windows
  - Google и контейнеры
  - Материалы по контейнерам
  - Документация
  - Виртуальные лабораторные работы
  - Первое знакомство с контейнерами
  - Права доступа
  - Основные команды
  - Информация о системе (WSL + Ubuntu)
  - Первый контейнер
  - Запуск контейнера
  - Этапы запуска контейнера
  - Контейнеризация
  - Образ = файловая система + конфигурация
  - Docker Hub — хранилище контейнеров
  - Подписка
  - Docker Dashboard
  - Загрузка образа
  - Alpine Linux
  - Список образов
  - Запускаем контейнер
  - Повторный запуск контейнера
  - Диалог с контейнером
  - Подключение к контейнеру

Настройка образов
Запускаем Bash
GNU Bash
Обновление и установка
Управление пакетами программ
Установка программы
figlet
Запускаем figlet
Фиксируем изменения в образе
Переименование образа
Выполнение команды в контейнере
Создаём образ с помощью Dockerfile
Создаём образ
Запускаем контейнер
Что такое «слой» образа
Приглашение к вводу команд
Добавляем одну команду
Создаём вторую версию образа
Основные термины
Веб-приложение Getting Started
Ubuntu + Midnight Commander
Литература

ПРЕДИСЛОВИЕ

Всё, что мы обсуждаем в данном пособии, Вам предстоит сделать в рамках лабораторной работы. Вам нужно научиться это делать. Вы потренируетесь и должны быть готовы выполнить любое действие с пониманием. Работаем — желательно — дома, не спеша, разобравшись, натренировавшись.

Напомним, что в учебном плане предусмотрено много часов для самостоятельной работы студентов (СРС). В рамках СРС вам предлагается самостоятельно освоить материал лабораторных работ. Тогда на занятиях с преподавателем у вас будет время продемонстрировать свои познания и умения — каждому студенту индивидуально.

Операционная система — это инструмент, это вещь для работы, для использования. Это кнопки, которые каждый должен уметь нажимать. Вот это от вас требуется, и этим наш предмет отличается от всех остальных.

ВВЕДЕНИЕ

В данном пособии мы разберём тему, которая стала модной в последние лет пять. Всё начиналось с виртуальных машин. То, что мы имеем сегодня, называется словом «контейнеры». Это развитие идеи виртуальных машин.

Нас будут интересовать четыре технологии в области виртуализации и контейнеризации:

— Hyper-V;
— Sandbox;
— WSL
— Docker.

В качестве заданий мы предлагаем просмотреть некоторые статьи в интернете и выполнить простые упражнения на компьютере.

Статьи в Википедии часто доступны и на русском, и на английском языках. Но они не обязательно в точности соответствуют друг другу.

Если статья написана на английском, а читать такой текст тяжеловато, можно использовать машинный перевод. В нашем распоряжении как минимум два варианта.

Вариант первый. В интернете есть готовые бесплатные сервисы по переводу веб-страниц: Яндекс, Гугл и прочие.

А вот и первое **задание**. Просмотрите в Википедии статью под названием «Ложные друзья переводчика». Выясните, в чем заключается проблема перевода, если английское слово звучит очень похоже на русское.

Переключитесь на английский вариант этой статьи и просмотрите автоматический перевод сайта на сервисах Яндекс и Google:

Яндекс — Переводчик — Перевод сайтов — Введите адрес сайта

Google — Переводчик — Сайты — Сайт

Вариант второй. Некоторые браузеры уже интегрировали сервис автоматического перевода. Здесь достаточно попросить перевести страницу или же отказаться от перевода и просмотреть оригинал страницы. Яндекс Браузер и Google Chrome могут нам помочь именно таким образом.

В дальнейшем, когда мы предлагаем читателю «просмотреть» статью, требуется получить самое общее представление о выбранной теме. Нужно извлечь из длинного текста только основную идею. Технические подробности мы обсуждаем при выполнении практических заданий.

ТРАДИЦИОННОЕ ВЫПОЛНЕНИЕ ПРОГРАММЫ

Как происходит выполнение программ на компьютере?

Чаще всего, в обычной жизни рядовой пользователь запускает программу на своём персональном компьютере. Это может быть настольный (Desktop) компьютер, или ноутбук (Notebook), или какое-нибудь мобильное устройство (Mobile Device).

ПОЛЬЗОВАТЕЛЬ (user) — по определению — если понимать это название буквально — ПОЛЬЗУЕТСЯ компьютером, использует (use) его. Чтобы получить «пользу» для себя. То есть занимается своим делом, своей работой или своими развлечениями.

А для работы ему нужна ПРИКЛАДНАЯ программа — Application Software. Или короче — Application. То есть «приложение». Или совсем коротко — просто App. Опять же «приложение».

Но программу не удаётся просто так взять и запустить на «голом железе», на оборудовании компьютера — Hardware. Вначале нужно загрузить операционную систему (ОС) — Operating System (OS).

Так и получается схема из трёх уровней: Оборудование — Операционная система — Прикладная программа, см. рис.

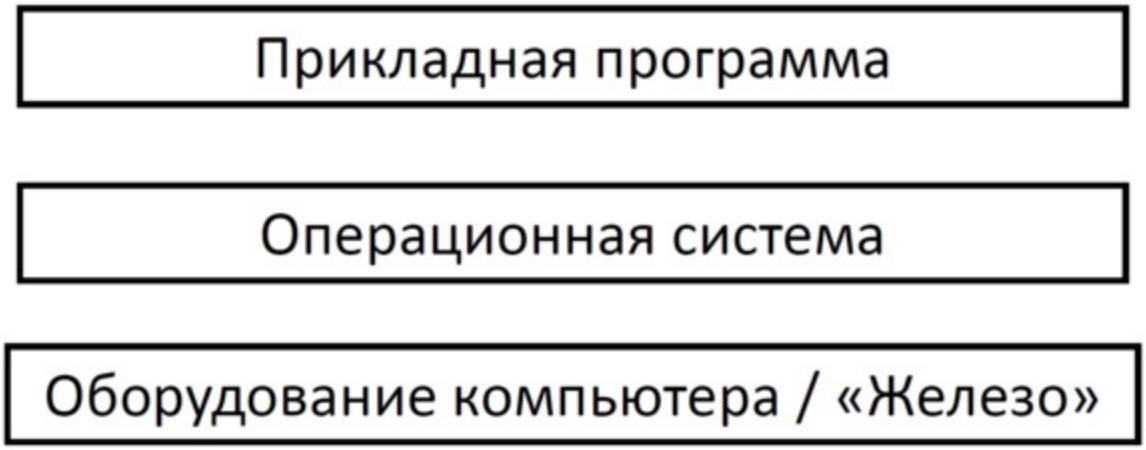


Рис. Традиционное выполнение программы

Итак, обычно программа выполняется с помощью операционной системы. Чтобы запустить прикладную программу, нужно включить компьютер и дождаться, пока загрузится операционная система.

Внизу у нас имеется то, что называется «железо» или hardware. Выше находится операционная система.

Но есть ситуации, когда операционная система входит в состав прикладной программы. Это может быть в простых встраиваемых системах. Что такое «встраиваемая»? Это означает, что процессор установлен внутри какого-то устройства. Это может быть микроволновка, холодильник или что-нибудь ещё со словами «умный» или «смарт». Если в этом устройстве хватает памяти, то запускается операционная система. Если это маленькое устройство, то там всё реализовано в одной программе — и функции операционной системы, и функции прикладной программы.

Итак, обычная схема «железо — ОС — приложение» хорошо работает для небольших задач.

ИЗОЛЯЦИЯ ПРИЛОЖЕНИЙ

Описанная вше «традиционная» схема хорошо работала и пока ещё работает, если у нас всего одна программа. Или всего несколько программ. И если эти программы не мешают друг другу.

Со временем выяснилось, что разработчики регулярно выпускают новые версии прикладных программ. При этом пользователи не просто привыкают к «своей» версии. Они нарабатывают множество файлов определённого формата. А потом в новой версии эти форматы немного меняются, потом ещё немного, потом меняются до неузнаваемости.

А на предприятии уже всё было отлажено, и всё работало. И любые изменения в работающей системе приводят к неприятностям. Поэтому, например, до сих на промышленных предприятиях есть системы управления производством на Windows XP (это реальный пример!). И на эту ОС даже не ставят обновления и исправления. Иначе конвейер или реактор просто перестанет работать.

Студентам интересно работать с «новой» версией, быть в курсе, быть в тренде. А для предприятия важно, чтобы производство шло без остановок и без перебоев.

Можно, конечно попытаться установить на один компьютер несколько версий одной программы. Чаще всего, это вообще невозможно — они будут «затирать» друг друга. Или, по крайней мере, одна версия будет мешать нормальной работе остальных.

Есть ещё и проблемы совместимости разных программ — ведь создают разные команды, разные фирмы-разработчики.

А ещё есть вопросы защиты и безопасности. Защита остальных приложений и самой операционной системы от запущенной

программы. Не говоря уже про защиту компьютера от вредоносного программного обеспечения (ПО).

Все эти проблемы приходится как-то решать. И общий подход здесь называется «Изоляция приложений». Одно приложение работает, но оно изолировано от всего остального мира. Это похоже на изоляцию пациента от всего остального мира, если есть подозрение на какую-нибудь опасную, заразную болезнь. Или же это как изоляция преступника от остального общества, чтобы он не мог наносить вред окружающим.

Задание. Просмотрите статью «Application Isolation»: https://devopsbootcamp.osuosl.org/application-isolation.html

Выясните, что такое изоляция и как это делается. При необходимости используйте машинный перевод веб-страницы.

Итак, на больших предприятиях появляются новые проблемы. Приходится держать несколько больших компьютеров для выполнения разных задач. Один сервер обслуживает электронную почту — это одна машина. Веб-сервер — это другая машина. Нужен какой-нибудь сервер для хранения файлов — это ещё одна машина. Для базы данных — ещё одна машина. И вот набирается несколько десятков отдельных компьютеров. Каждый для своей задачи.

Если нужно что-то обновить, возникает масса проблем. Чтобы увеличить оперативную память, нужно выключить машину, добавить память, включить, загрузить и так далее. В некоторых случаях вообще приходится полностью заменять «железо», если оно уже не справляется с новыми задачами и объёмами информации. Это традиционный подход.

Что ещё здесь интересного? Разные версии программ в рамках одного компьютера очень плохо ведут себя, если пытаться одновременно их держать на одной машине. Некоторые программы друг с другом плохо совместимы. Так появляется первый шаг: каким-то образом изолировать программы друг от друга. Эта проблема

встала, как только на одном компьютере установили больше, чем одну программу.

Естественно, есть некоторые программы, которые называют «вредоносными». Программы пишут с разными целями. Хорошо бы, чтобы они не мешали всем остальным, и уж тем более не мешали работе операционной системы.

ДИНАМИЧЕСКИЕ БИБЛИОТЕКИ

Программисты очень быстро обнаружили, что какие-то действия — и соответственно, какие-то части текста программы (его ещё называют словом «исходный код») используются много раз в разных программах. Одно и то же действие — одна и та же программа. Это вроде бы даже очевидно. Поэтому они почти сразу же придумали такое решение: то, что повторяется, вынести в отдельные «библиотеки». Это коллекция повторяющихся действий, часто используемых функций.

Примером таких библиотек является технология DLL. Это внешние файлы, из которых любая программа может вызвать нужную функцию. На первый взгляд, решение хорошее, удобное. Потом начали накапливаться неприятности — буквально как снежный ком.

Выясните, в чем состоит идея DLL, какие у этой технологии есть достоинства и недостатки, и что такое «зависимости».

Итак, существует такая технология как DLL — динамически подключаемая или динамически связываемая библиотека. Изначально технология была задумана, чтобы облегчить работу. Но потом оказалось, что все эти библиотеки складываются в одном большом каталоге, и там их несколько может набираться десятков тысяч. Они могут мешать друг другу. Всё это явление природы называется «DLL hell». Английское слово «hell» буквально означает «ад». Эта адская «свалка» файлов — огромная куча файлов, в которых разобраться очень сложно. К тому же они ещё и норовят

друг друга перезаписать при установке новой версии программы. Плюс к этому есть такая вещь как «dependency», то есть «зависимость»: для работы одной программы может потребоваться установка дополнительных библиотек, дополнительных файлов или вообще дополнительных программных продуктов. И это тоже разрастается как снежный ком. Эта ситуация называется «dependency hell» — тоже в рамках одного компьютера.

Задание. Просмотрите в Википедии следующие статьи:

— Динамически подключаемая библиотека
— Dynamic-link library
— DLL Hell
— Dependency hell

ИЗОЛЯЦИЯ И АДМИНИСТРИРОВАНИЕ

Существуют разные способы изоляции программ друг от друга. Один из таких способов называется «jail» — буквально переводится как «тюрьма». То есть программа буквально помещается в «тюремную камеру», откуда она не может вырваться, и не может никому помешать, и не может ни с кем общаться — буквально как преступник, заключённый в тюрьме.

Естественно, находятся любители взламывать системы. Эта технология называется «джейл брейк», то есть «побег из тюрьмы». Особенно это популярно для продукции компании Apple, потому что мобильное устройство Apple построено на основе операционной системы Unix. Здесь используют принципы «юникса», и в этом случае права пользователя очень сильно ограничены. И к тому же каждая прикладная программа (приложение, application) полностью отрезано от всех остальных. Для того, чтобы из одной программы передать данные в другую приходится использовать «обходные манёвры» — что-то типа экспорта и импорта.

Более того, мы фактически не можем обратиться к файловой системе iPhone напрямую. В лучшем случае, у нас есть фрагмент

файлового дерева, доступный для пользователя. Причём для каждой прикладной программы – свой кусочек дерева, и эти части файловой системы могут и не пересекаться. Большие любители программирования придумали, как взламывать эту защиту и получать права доступа администратора к продукции компании Apple. Технология взлома защиты называется «джейлбрейк». После взлома можно записывать всё, что угодно и куда угодно. Можно устанавливать любые программы. Естественно, это кончается установкой пиратских, взломанных программ из пиратского же магазина приложений. Теперь уже ни о какой безопасности речи не идет. Мы устанавливаем неизвестно что и неизвестно откуда, к тому же с полными правами доступа. Тем не менее, такая технология есть.

Здесь упоминается системный администратор. Он может называться «root», то есть «корень». Может называться «superuser» — «супер-юзер» — «супер-пользователь». Может называться «админ». В юниксе может встретиться su — сокращение от «супер-юзер».

Как мы уже говорили, изоляция приложений больше всего похожа на изоляцию заразного пациента или опасного преступника от общества. Поэтому и название программисты использовали готовое: «jail», то есть «тюрьма». Ситуация, когда программа может вырваться из этой «тюрьмы», тоже получила соответствующее готовое название. С общими подходами и общепринятыми названиями вам предстоит ознакомиться самостоятельно.

Задание. Просмотрите в Википедии следующие статьи:

— chroot
— FreeBSD jail
— Privilege_escalation#Jailbreaking
— iOS jailbreaking
— Rooting (Android)
— Superuser — root, admin
— su (Unix)

— sudo

В этих статьях встречаются разные названия для одного и того же человека — администратор, админ, суперпользователь, admin, superuser, roor. Это пользователь, имеющий полные права доступа к вычислительной системе. В некоторых упражнениях нам потребуются именно такие права.

PORTABLE APPLICATIONS

Один из популярных подходов к «изоляции» приложений — это так называемые «переносимые приложения» — Portable Applications. Здесь «переносимость» означает, что программы не нужно устанавливать, а достаточно просто скопировать файлы. Обычно это выглядит так: скачиваем в интернете архив и затем распаковываем его в любом каталоге. Запускаем программу из этого каталога — и всё работает!

С этим подходом к изоляции приложений мы постепенно знакомимся в рамках лабораторных работ. Это так называемые «переносимые» приложения или Portable Apps — Portable applications.

В простейшем, бытовом понимании «переносимость приложения» означает, что программу можно просто скопировать: взять одну папку, скопировать на другой диск, или на флешку, или на другой компьютер. Можно скачать архив и развернуть в любом каталоге — и оттуда начинать работать. Работа без установки — это главный фокус переносимых приложений — Portable Applications. Мы скачиваем, разворачиваем, запускаем. Если программа нам надоела, мы её стираем — и никаких следов в самой операционной системе. При этом все файлы лежат в одном каталоге, и они никак не пересекаются с файлами, которые лежат в другом каталоге. Нет общих папок между этими переносимыми приложениями. Так организована изоляция приложений — как разграничения на уровне файловой системы.

Переносимые приложения не участвуют в «свалке DLL», которая обычно находится в каталоге C:\Windows\System32. Здесь всё сложено в одной папочке и

На второй лабораторной работе мы используем Portable версию для файлового менеджера FAR Manager:

FAR Manager Portable:

https://www.farmanager.com/download.php



Рис. Варианты загрузки FAR Manager

Предлагаем самостоятельно посмотреть Portable версию для Libre Office:

LibreOffice Portable

https://www.libreoffice.org/download/portable-versions/

Есть целый сайт под названием Portable Apps. Здесь имеется целый комплект самых разных программ, см. рис.

https://portableapps.com/



Рис. Portable Apps

Главная хитрость в том, что не всегда это слово Portable появляется на сайтах. Например, если мы заходим на сайт FAR Manager, у нас есть несколько вариантов скачивания. Здесь есть «архив» и MSI. Слово «Архив» подразумевает, что всё упаковано в одном файле. Это и есть Portable приложение. Мы разворачиваем этот архив в отдельный каталог, откуда оно и запускается без установки. Если говорить про MSI, это Microsoft Installer, то есть установочная версия.

Итак, Portable — это одна из попыток разделить зоны влияния и изолировать программы друг от друга.

Задание. Просмотрите в Википедии следующие статьи:

— Переносимое приложение
— Portable application
— Software portability

ВИРТУАЛЬНЫЕ МАШИНЫ

ВИРТУАЛИЗАЦИЯ. ГИПЕРВИЗОР — HYPERVISOR

Следующим шагом было полностью изолировать целые операционные системы со всем комплектом программ. Это называется «виртуальные машины». С ними мы знакомимся в рамках 4, 5 и 6 лабораторных работ, когда запускаем Linux внутри операционной системы Windows. Если у кого-то есть Макинтош, здесь тоже можно в рамках виртуальной машины «покрутить» что-нибудь другое — Windows, или Linux, или Unix.

Здесь внизу имеется «железо». Дальше работает основная операционная система или Host OS. Английское слово «хост» означает «гостеприимный хозяин», то есть «тот, кто принимает гостей». Следующий вспомогательный слой называется «гипервизор» или «менеджер виртуальных машин». На наших лабораторных работах это программа VirtualBox. С помощью этого гипервизора мы запускаем другую операционную систему. Она будет называться гостевая ОС — Guest OS. Мы в отдельном окне получаем другую операционную систему. В неё мы можем установить всё, что хотим. Эта система может работать сама по себе, независимо от основной ОС. Можно развернуть несколько виртуальных машин на одном физическом «железе». Каждая виртуальная машина «живёт» сама по себе, и она не мешает никому. Так что в рамках виртуальной машины мы имеем гостевую операционную систему и любые прикладные программы — какие захотим установить.



Рис. Виртуальная машина

ВИРТУАЛИЗАЦИЯ (BARE METAL HYPERVISOR)

Есть и другой вариант виртуализации — когда у нас нет основной операционной системы. Есть только «голое железо». На него сразу ставится программа под названием hypervisor. По сути, это урезанная операционная система, которая нужна только для запуска виртуальных машин. Такие варианты тоже есть. Они так и называются: «Bare metal hypervisor» — «гипервизор, который работает на голом железе».

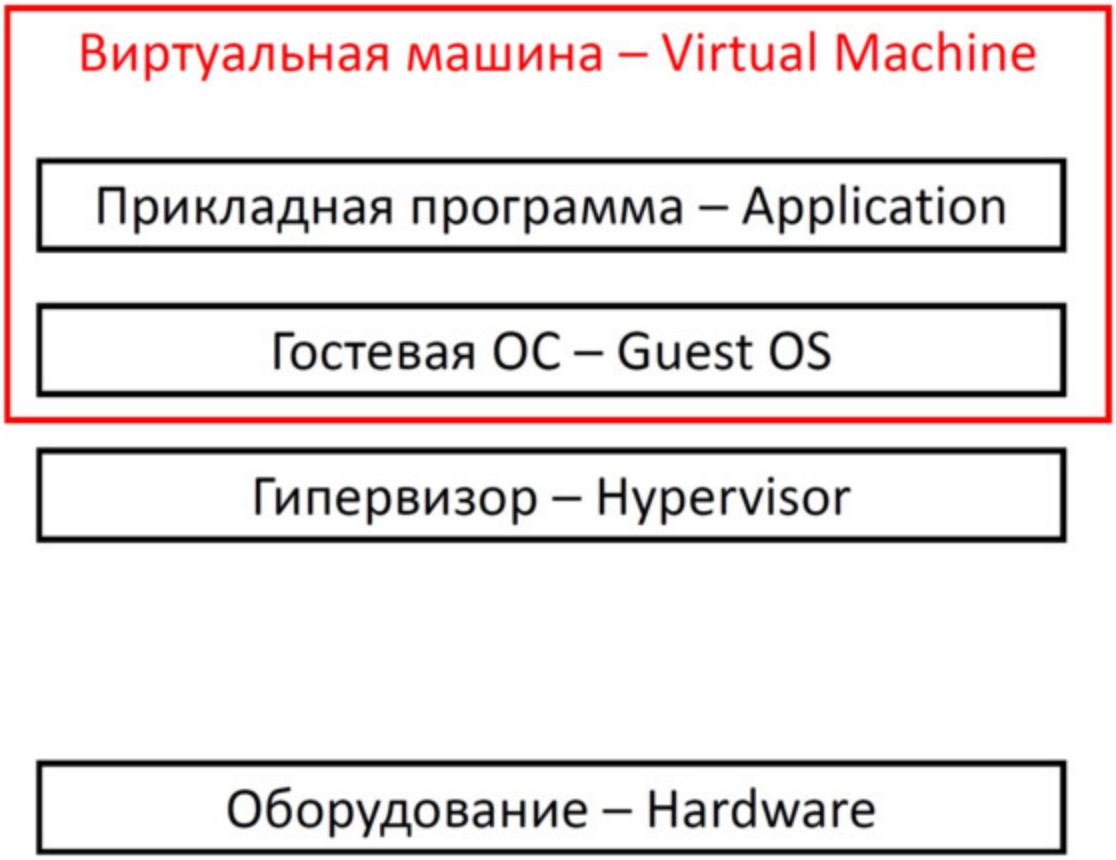


Рис. Bare metal hypervisor

ГИПЕРВИЗОР – HYPERVISOR

Итак, слово «гипервизор» означает «менеджер виртуальных машин» или «монитор виртуальных машин» – Virtual Machine Monitor (VMM). Это программа, которая позволяет запускать гостевые операционные системы. Пример гипервизора: Oracle Virtual Box, который мы используем на лабораторных работах. Это может быть встроенный Microsoft Hyper-V и бесплатный VMware Workstation Player. На компьютерах Макинтош это Parallels Desktop. Таких программ существуют десятки, а то и сотни

Задание. Просмотрите в Википедии следующие статьи:

– Virtual Box

— Hyper-V
— VMware Workstation Player
— Parallels Desktop для Mac

ТИПЫ ГИПЕРВИЗОРОВ

Отметим, что различают два типа гипервизоров. Тип первый — «голое железо», второй — это гипервизор поверх основной операционной системы. Эти технологии существуют и очень широко используются. Вам предлагается просмотреть дополнительные материалы, чтобы прочувствовать общую идею по поводу гипервизоров.

Надо сказать, что приложение Hyper-V уже входит в состав операционной системы Windows 10 Professional. Нужно только включить эту возможность, и можно сразу использовать. А ещё есть Hyper-V как отдельный продукт, он называется Hyper-V сервер. Это как раз гипервизор поверх «голого железа». Эта программа ставится на голое железо вместо основной операционной системы, а поверх неё разворачивается несколько виртуальных машин. Аналогично работает ESXi. Это гипервизор первого типа.

Задание. Просмотрите в Википедии следующие статьи:
Виртуализация
Virtualization
Гипервизор
Hypervisor
Type-1, native or bare-metal hypervisors
Type-2 or hosted hypervisors
Hyper-V
Hyper-V Server
VMware ESXi
Bare machine

ЦОДЫ / СЕРВЕРЫ / «ОРКЕСТРАЦИЯ»

Серьёзные технологии используются на крупных предприятиях, чтобы разместить сервисы в рамках одного очень большого компьютера. Такие большие компьютеры называются центры обработки данных (ЦОДы), или дата-центры. Эта установка может выглядеть как большой холодильник или может даже занимать отдельную комнату, а в каждой компьютерной стойке (шкафе) могут быть десятки материнских плат. Каждая такая плата будет называться словом «сервер» или «узел». Всё это вместе будет называться ЦОД или дата-центр.

В рамках одного большого ЦОДа разворачивают большое количество виртуальных машин, и они заменяют физические серверы. Такая технология называется «консолидация серверов». «Консолидация» означает, что отдельные серверы собрали в единое целое. Выясняется, что теперь можно на ходу, «на лету» менять количество ресурсов, которое выделяется разным серверам, разным виртуальным машинам — не выключая и не перезагружая большую машину. Если на почтовый сервер увеличилась нагрузка, можно сказать, что теперь он будет использовать 10,5 процессоров и у него будет 64 Гб оперативной памяти. Если нагрузка уменьшилась, можно сказать, что теперь у него будет 5,8 центральных процессоров и 16 Гб оперативной памяти.

Это всё происходит на ходу без выключения компьютера и называется «динамическая балансировка нагрузки» — перераспределение ресурсов в рамках одной большой машины. Каждая виртуальная машина получает столько ресурсов, сколько ей требуется для работы.

Предприятие может содержать свой собственный ЦОД либо арендовать у телекоммуникационной компании. Тогда даже не нужно тратиться на инженеров, там всё обслуживается — где-то там, в «облаке». Мы только получаем ресурсы и платим за такое количество ресурсов, которое требуется.

Слово «облако» сейчас становится всё более модным. «Облако» означает, что сервис находится «где-то там», на другой машине, «неизвестно где», а мы получаем доступ через интернет.

Кроме того, существует такая процедура как миграция виртуальных машин — опять же без выключения, без перезагрузки. Всё продолжает работать. Пользователь даже не заметит, что произошло. Фактически, виртуальная машина переехала с одной физической машины на другую.

Однако всё это все заканчивается одной большой проблемой: если у нас много виртуальных машин, то вручную этим заниматься становится тяжело. Когда несколько десятков или сотен виртуальных машин, то для управления таким «зоопарком» уже используют специальные программные средства. Эта вспомогательная технология называется новым словом — «оркестрация». Что такое оркестр? Это группа музыкантов, которые играют одно произведение. При этом каждый исполняет свою партию. Так вот, некоторое количество машин тоже будет называться «оркестром» — в больших кавычках. Управление таким «оркестром» и называется «оркестрация». Это уже из области системного администрирования.

Надо понимать, что для этого есть свои инструменты — для оркестрации. Например, компания VMware предлагает свои инструменты для оркестрации.

VMware Orchestrator

https://www.vmware.com/ru/products/vrealize-orchestrator.html

Microsoft Azure — облачный сервис майкрософта — там своя оркестрация. Предлагаем вам самостоятельно познакомиться с идеей оркестрации.

Azure VM Orchestration

https://youtu.be/CYCtKQhg4rU

VM orchestration with Azure

https://youtu.be/8Ug-GrOH3WU

Оркестрация виртуальных машин
https://youtu.be/5FytH4rKCnM

Задание. Просмотрите в Википедии следующие статьи:
— Hardware virtualization • server consolidation
— Load balancing (computing)
— Live migration
— Orchestration (computing

VIRTUALBOX

В рамках наших лабораторных работ мы знакомимся с одним примером гипервизора — VirtualBox. Первоначально он был разработан одной компанией, потом её скупила другая. На сегодняшний день это компания Oracle. То, что у нас есть на https://www.virtualbox.org/, можно просто скачать и установить — Download.

Здесь же можно почитать, на каких платформах эта система работает:

Runs on… hosts

Мы обнаружим список из нескольких, разных операционных систем. То есть в разных основных операционных системах можно установить VirtualBox и в нём уже разворачивать виртуальную машину.

Плюс к этому существует некоторое количество готовых виртуальных машин. Это просто один большой файл — несколько гигабайт. Мы его скачиваем, и там всё настроено, установлено. Вам нужно только запустить эту виртуальную машину. Это называется **Pre-built VirtualBox VMs**. Фактически, это предварительно установленные и настроенные виртуальные машины.

Более того, сам этот проект уже раскрыл свои исходные тексты VirtualBox Sources. Исходные коды — Source code. Все желающие

могут его посмотреть, использовать, модифицировать. Можно даже увидеть, где лежат эти «исходники» и на чём они написаны.

Другие компании этим воспользовались, и у нас есть продукт под названием Portable-VirtualBox: https://www.vbox.me/. Если вы не хотите устанавливать эту программу или если у нас нет прав администратора, то мы на компьютер заходим как пользователи. Теперь мы можем использовать переносимую версию Portable, которая работает без установки — в том числе, даже с флешки. В некоторых случаях это может быть полезно. После запуска переносимая версия ведёт себя абсолютно так же, как и установленная. Единственное, что она не требует прав администратора, и она не занимает никаких ресурсов на системном диске C:\ — там, где у нас находится основная операционная система.

Внешне всё выглядит очень просто. У нас есть основная операционная система — в данном случае Windows. В отдельном окошке у нас крутится что-нибудь другое, например, Linux.

Задание. Просмотрите в Википедии статью «VirtualBox».

Windows + Oracle VM VirtualBox + Ubuntu

На рисунке приводится пример запуска Linux. Этот вариант называется Ubuntu. У нас полноценный рабочий стол — все привычные окна, выход в Интернет — всё это работает.

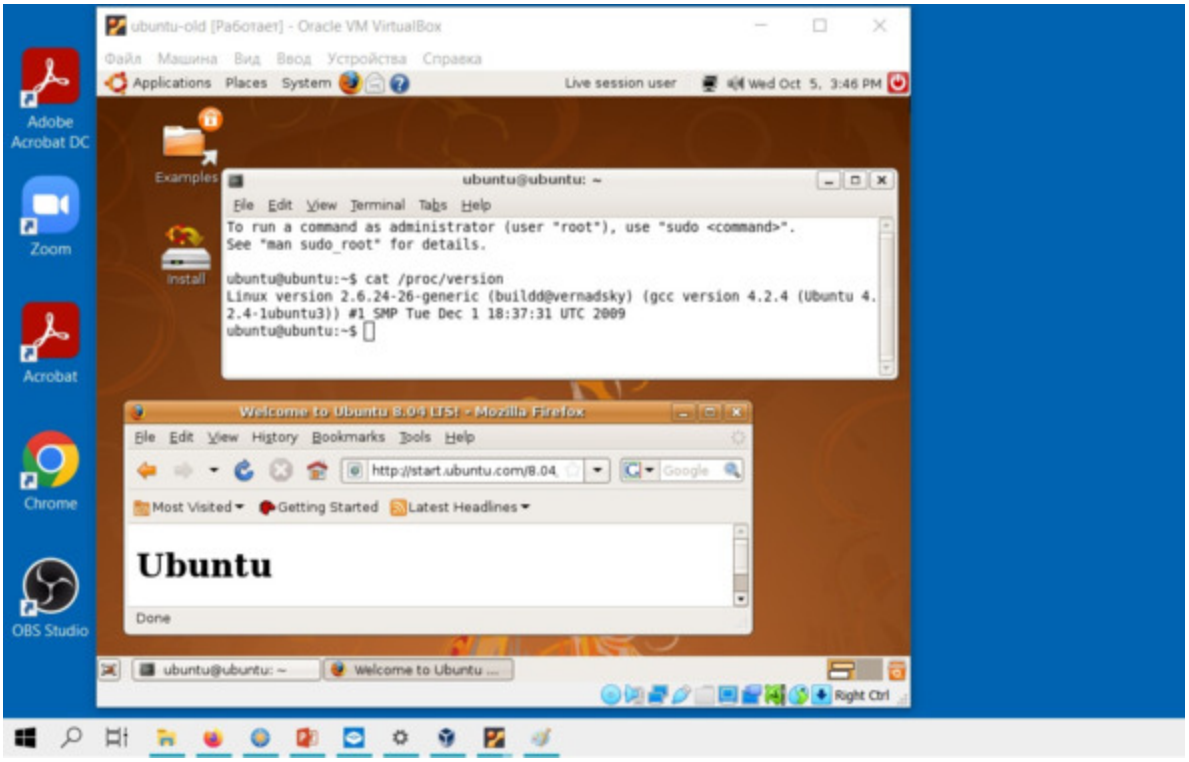


Рис. VirtualBox + Ubuntu

При желании можно виртуальную машину развернуть во весь экран. Тогда будет полная иллюзия, что мы работаем в Linux. Для наших учебных задач это возможность познакомиться с другой операционной системой. У нас уже не та ОС, которая установлена на машине, а какая-нибудь другая.

Лет 20 назад на нашей кафедре на каждый компьютер в дисплейном классе устанавливали по две операционных системы: Windows и Linux. При включении компьютера человек выбирал вариант загрузки: либо Windows, либо Linux. Далее система загружалась, и в неё можно было работать. Но оказалось, что это очень сложно в обслуживании. Представьте себе несколько десятков машин, и каждые полгода нужно было это всё переустанавливать. Виртуальная машина гораздо проще.

Итак, устанавливаем VirtualBox либо запускаем его переносимую версию. Скачиваем какой-нибудь образ Linux,

например, какой-нибудь вариант Ubuntu. Это файл с расширением *.ISO. Один-единственный файл, несколько гигабайт. Создаём виртуальную машину, делаем необходимые настройки и подключаем этот образ — как будто бы это компакт диск, который вставлен в дисковод. Далее запускаем гостувую ОС в «пробном» режиме: Try Ubuntu. Это режим работы без установки операционной системы. Начиная с четвертой лабораторной работы вы именно так и работаете. При желании можно даже использовать физический диск — компакт-диск или DVD — любой лазерный диск. Мы вставляем его в дисковод, а виртуальной машине говорим, что нужно загружаться с указанного диска. Да, мы можем даже такое производить: виртуальная машина грузится с физического диска или флешки.

Задание

— Установить VirtualBox
— Скачать образ Ubuntu *.iso
— Создать виртуальную машину
— Подключить образ *.iso
— Запустить Ubuntu в режиме Try Ubuntu

HYPER-V

Совершенно аналогично для пользователя выглядят и другие гипервизоры. Рассмотрим Hyper-V — продукт компании Microsoft. Само название вроде бы намекает на слово «гипервизор». С другой стороны, буква «V» может указывать на слово «Virtualization» — «виртуализация». «Гипер» + «виртуализация».

Чтобы эта функция заработала, придётся нужно нажать несколько кнопок и поставить галочку в настройках Windows — чтобы включить это вот функцию:

Параметры — Приложения — Программы и компоненты — Включение или отключение компонентов Windows

Здесь установка программ не потребуется, нужно именно активировать эту функцию в составе основной ОС. Это касается настольной версии. Если же у нас Windows Server, то нам потребуется включить соответствующую «роль» — Role. Каждый модуль серверной операционной системы Windows называется «роль» и нам нужно включить «роль».

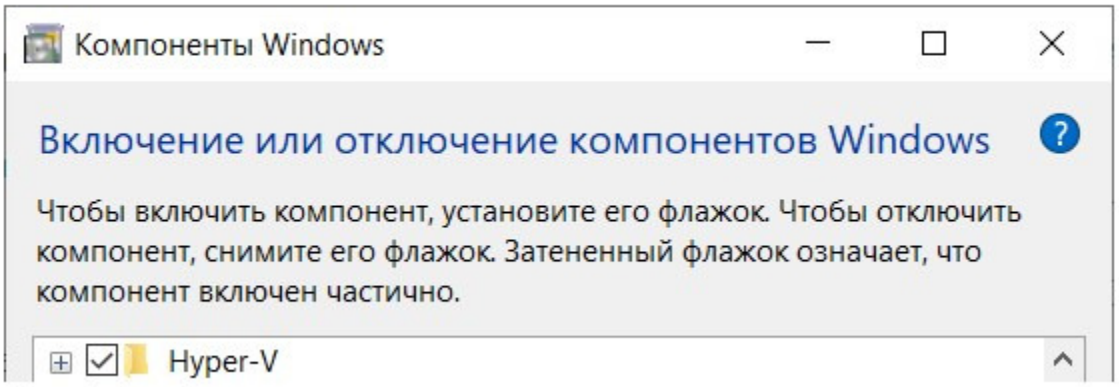


Рис. Включение Hyper-V

WINDOWS + HYPER-V + UBUNTU

После того, как мы эту функцию включили, повторяем те же самые действия: создаём виртуальную машину, выбираем параметры — сколько у нас будет места на жестком диске, сколько выделяем оперативной памяти. Затем подключаем образ и загружаемся с него. В результате в отдельном окошке мы получаем нашу гостевую операционную систему. В ней можно работать, и это никак не повлияет на основную ОС.

Кроме того, у нас есть возможность копировать, перебрасывать файлы из основной ОС в гостевую и обратно. Это дополнительная функция.

Вам тоже предлагается с этой технологией познакомиться. Используем тот же самый образ Ubuntu. Попробуйте подключить его в режиме виртуальной машины и загрузиться.

Единственное, что здесь иногда нужно знать сочетания клавиш. Если их не знаешь, то будет непонятно, как «вырваться» из этого гостевого окна. Мы щёлкаем мышкой внутри окна виртуальной машины, и теперь мышка перемещается только внутри окна. А чтобы выйти из него, нужна специальная комбинация клавиш. Так что желательно к этому подготовиться. Это может пригодиться.

Выход из окна ВМ:

[Ctrl + Alt + Left]

Выход из полноэкранного режима ВМ:

[Ctrl + Alt + Pause]

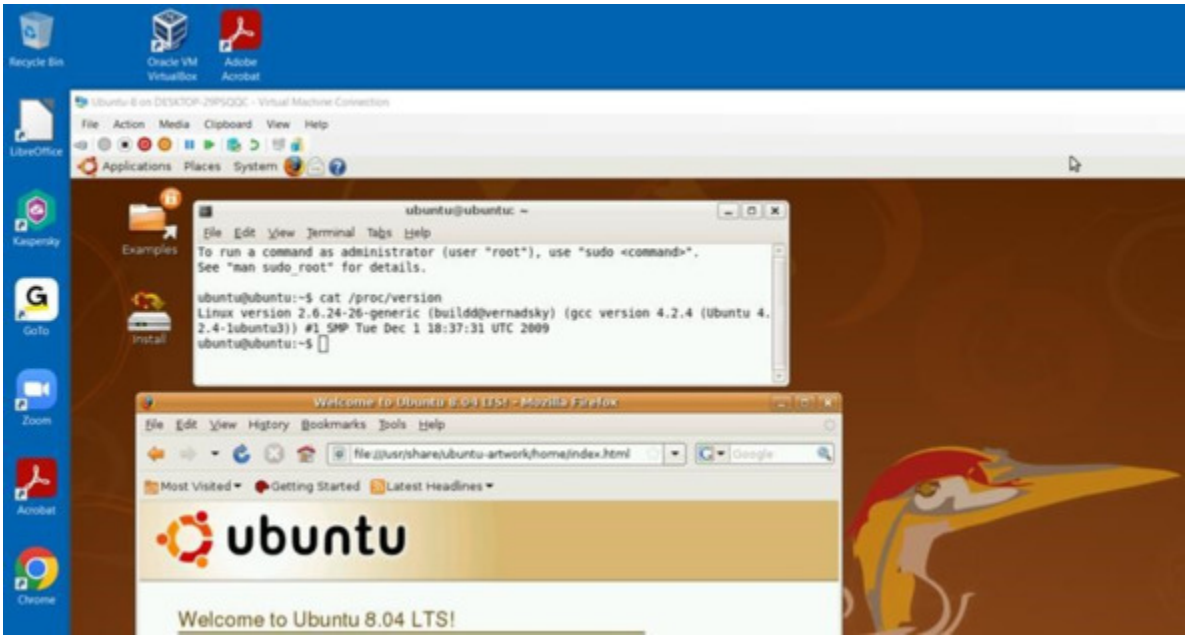


Рис. Hyper-V + Ubuntu

Задание

— Включить Hyper-V

— Создать виртуальную машину

— Подключить образ *.iso

— Запустить Ubuntu в режиме Try Ubuntu
— Использовать сочетания клавиш для выхода из окна

ПЕСОЧНИЦА WINDOWS

Развитие технологии виртуальных машин мы наблюдаем в форме Песочницы — SandBox. Это еще одна технология виртуализации. Вообще-то, «песочница» — это общее название для самых разных технологий и инструментов.

В буквальном понимании «песочница» — это огороженная, небольшая площадка, внутри которой сидят маленькие дети, играют с машинками, копают ямки в песочке. Их обычно наружу не выпускают, пусть дети в песочке играются.

Точно также существует «песочница» для разработки и внедрения информационных систем. «Песочница» — это изолированное окружение, либо изолированная машина, либо часть машины, внутри которой запускают очередную версию информационной системы. После обновлений её готовят к развёртыванию, проверяют — и это никак не влияет на работу предприятия. В песочнице поработали — убедились, что всё нормально. И после этого систему переносят на основную, работающую платформу.

Песочница Windows появилась в Windows 10, и в последующих версиях она тоже присутствует. Для использования этого инструмента нужно его включить. Не устанавливать, а именно включить: поставить галочку в разделе компоненты Windows — Песочница. После этого у нас в меню в меню пуск появляется возможность запускать Песочницу.

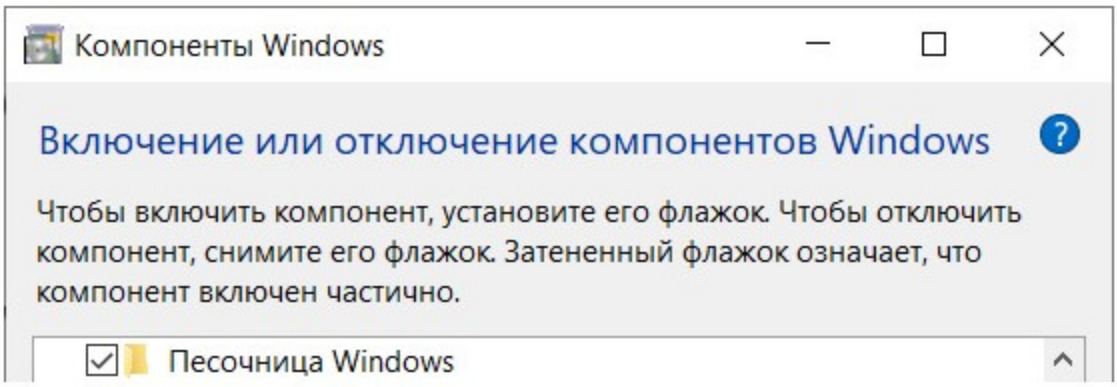


Рис. Включаем Песочницу

Задание. Просмотрите статью «Windows Sandbox» на сайте https://learn.microsoft.com

Задание. Просмотрите в Википедии следующие статьи:
Sandbox (computer security)
Sandbox (software development)
Википедия: Песочница

Задание. Изучите статью по адресу:
https://docs.microsoft.com/ru-ru/windows/security/threat-protection/windows-sandbox/windows-sandbox-overview

Мы запускаем Песочницу и получаем отдельное окно, внутри которого тоже выполняется ОС Windows. Но на этот раз это не просто гостевая операционная система. Фактически, это доступ к тому же самому ядру ОС, что и у основной операционной системы. Но есть отличие: в песочнице у нас «чистая» операционная система, в которой вообще ничего не установлено и не настроено. В основной операционной системе могут быть установлены прикладные программы, а в песочнице только «чистая», «голая» версия Windows.

Мы можем запустить Песочницу и поработать в ней. Но как только мы закрыли это окно, все наши данные стираются. Из этого окна у нас есть выход в интернет. В этом окне у нас все функции Windows. У нас полный доступ, у нас есть права администратора, и у нас есть все возможности делать всё, что угодно. Но эти изменения никак не повлияют на работу основной операционной системы. Это очень полезно для разных экспериментов, для опытов по установке программ — для временной установки. На полчаса установили программу, поработали и выключили — и никаких последствий для основной ОС и для компьютера.

Здесь можно запускать подозрительные программы и что-нибудь, связанное с вирусами. Всё это желательно делать в песочнице, чтобы не пострадал основной компьютер.

Вот простые задания: запустить песочницу, посмотреть свойства системы, менеджер устройств посмотреть, запустить программу и убедиться, что там ничего нет. И после этого мы можем через буфер обмена копировать текст или файлы между основной и гостевой операционными системами. У нас есть буфер обмена: **[Ctrl+C]**, **[Ctrl+V]**. Можем также копировать правой кнопкой мыши. Пока это единственный инструмент для обмена данными — по умолчанию это единственный канал. Всё остальное полностью изолировано.

Этот инструмент открывает возможности для экспериментов, для первого знакомства и для установки программ, которые не жалко будет тут же стереть.

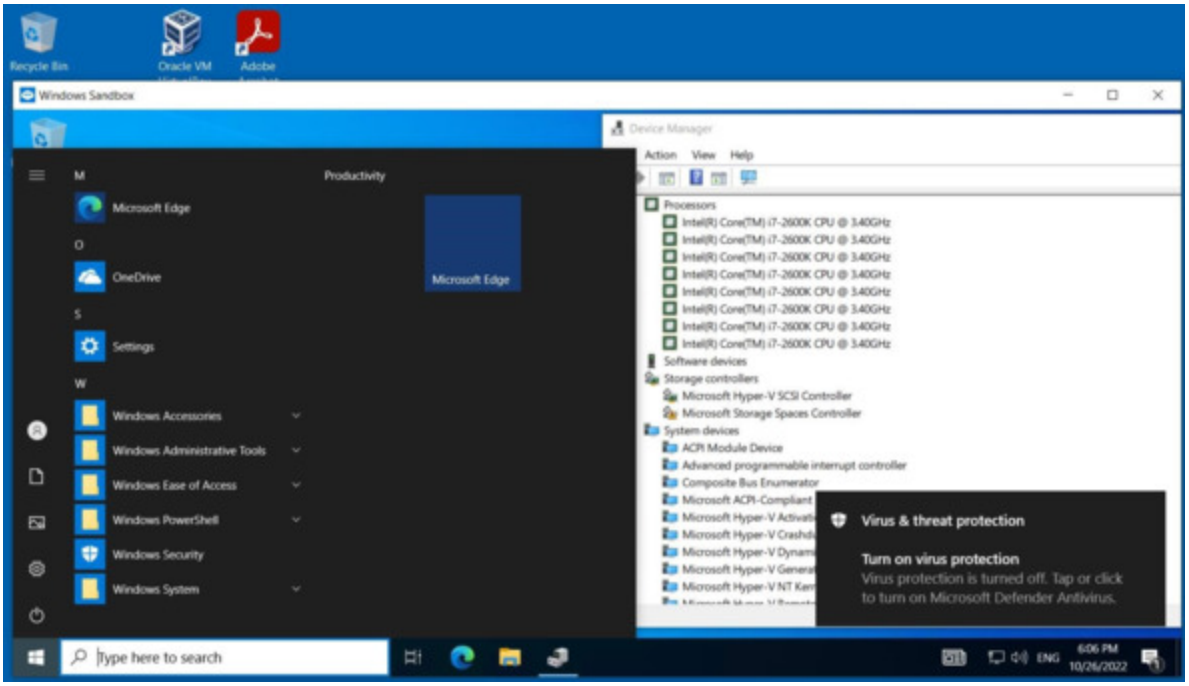


Рис. Песочница Windows

Задание

Включить Песочницу

Запустить Песочницу

Перейти в окно Песочницы

Запустить Диспетчер устройств / Device Manager

Нажать кнопку Пуск / Start

Изучить список установленных программ

Скопировать файлы между основной ОС и Песочницей через буфер обмена

ОБЩАЯ ПАПКА / MAPPED FOLDER

Следующая возможность — это общая папка для обмена файлами между основной операционной системой и песочницей. Делается это через файл с расширением *.WSB. Мы создаём этот файл в Блокноте.

sandbox-map-folder.wsb

```
<Configuration>
  <MappedFolders>
    <MappedFolder>
      <HostFolder>H:\WSB</HostFolder>
      <SandboxFolder>C:\WSB</SandboxFolder>
    </MappedFolder>
  </MappedFolders>
</Configuration>
```

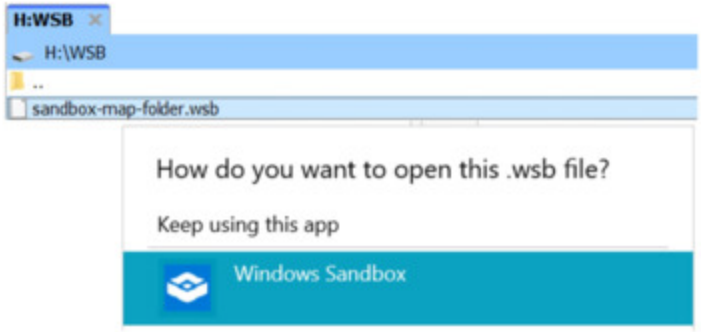


Рис. Конфигурация Песочницы

Тут есть раздел <Configuration> — «Конфигурация». Далее есть раздел <MappedFolders> — «Отображаемые папки» — во множественном числе. Внутри может быть несколько разделов <MappedFolder> — «Отображаемая папка» — в единственном числе. Английское слово «map» означает «отображение» или «соответствие». Здесь мы назначаем соответствие между папкой в основной операционной системе и папкой в песочнице. Так что из песочницы мы сможем обратиться к указанной папке на основном компьютере.

В нашем примере мы указали <HostFolder> — «папка в основной ОС». То есть это папка на «хосте». Мы можем указать здесь любой диск и любой каталог. Затем мы указываем каталог, который будет создан внутри песочницы <SandboxFolder> — «папка в песочнице». При включении песочницы этот каталог будет создан, но на самом

деле он будет работать как указатель на физически существующую папку в основной системе. Через эту папку мы и сможем обмениваться файлами. Мы можем в основной системе туда положить какой-нибудь файл. Затем при включении песочницы эти файлы будут там доступны внутри гостевой ОС.

Обратим внимание, что каждый раздел оформляется в стиле меток HTML: есть открывающие и закрывающие метки.

Мы запускаем нашу песочницу, но не через меню **Пуск**, а путём открытия файла конфигурации с расширением ***.WSB**. Для запуска два раза щелкаем по иконке этого файла. Нас спрашивают, открыть ли его с помощью Песочницы. Мы соглашаемся, и песочница использует эту конфигурацию. Теперь мы получаем доступ к общей папке, где уже что-то лежит. Таким образом, мы можем перебрасывать любое количество файлов туда и обратно через общую папку. С этой технологией вам предстоит познакомиться и потренироваться.

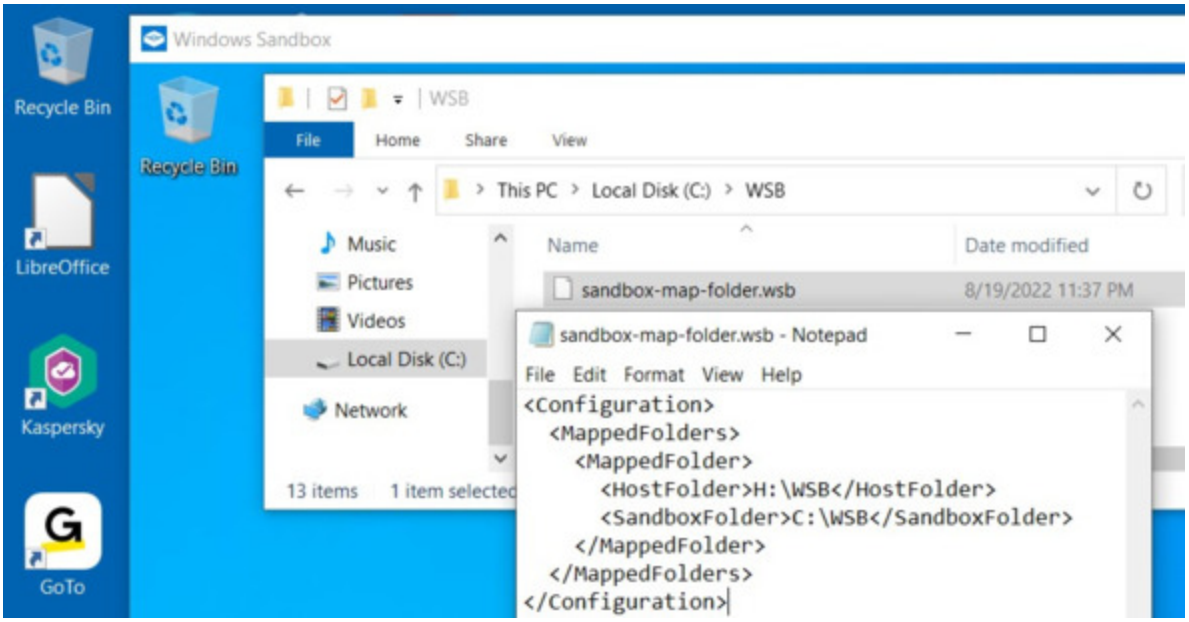


Рис. Общая папка

Задание

— Создать файл *.WSB для работы с общей папкой

— Открыть файл *.WSB с помощью Песочницы

— Скопировать файлы между основной ОС и Песочницей через общую папку

НОВЫЙ БРАУЗЕР В ПЕСОЧНИЦЕ

У нас возможность автоматически запускать какие-то команды или программы при включении песочницы. Это тоже прописывается в файле ***.WSB**, см. рис.

sandbox-opera.wsb

```
<Configuration>
  <MappedFolders>
    <MappedFolder>
      <HostFolder>H:\WSB</HostFolder>
      <SandboxFolder>C:\WSB</SandboxFolder>
    </MappedFolder>
  </MappedFolders>
  <LogonCommand>
    <Command>C:\WSB\OperaSetup.exe /silent
/launchbrowser=1</Command>
  </LogonCommand>
</Configuration>
```

Рис. Конфигурация Песочницы с автозапуском

В этом файле конфигурации появляется дополнительный раздел <LogonCommand> — «команда, выполняемая после регистрации пользователя в системе». Выражение «log on» означает регистрация

пользователя в системе или вход в систему. Проще говоря, при включении песочницы будет выполнена указанная команда. Мы можем назначить любую команду, и она будет выполнена. Мы можем положить какой-то «установочный» файл в общую папку. Тогда при включении песочницы его можно запускать автоматически.

В нашем примере это установка браузера Opera. Первоначально его в песочнице, естественно, нет — потому что Windows использует свой браузер Edge. Мы в качестве эксперимента складываем установочный файл в общую папочку и прописываем эту команду. Мы указываем, где лежит этот файл и указываем ключи. Ключ **/silent** — «молча; без сообщений» — означает, что установка происходит «молча». Другими словами, во время установки никаких сообщений, никаких окошек не появляется. Мы просто включили песочницу, и через несколько секунд браузер установлен автоматически. Следующий ключ **/launchbrowser=1** означает, что браузер запустится сразу после установки. Слово «launch» означает «запустить программу на выполнение». Мы включили песочницу, и через несколько секунд в ней уже открывается окошко браузера. Такой эксперимент вам тоже предстоит произвести. Мы скачиваем установочный файл, открываем песочницу через файл ***.wsb** и убеждаемся, что теперь у нас есть новый браузер и что он имеет выход в Интернет. По таким шагам можно пройтись и познакомиться с этой технологией.

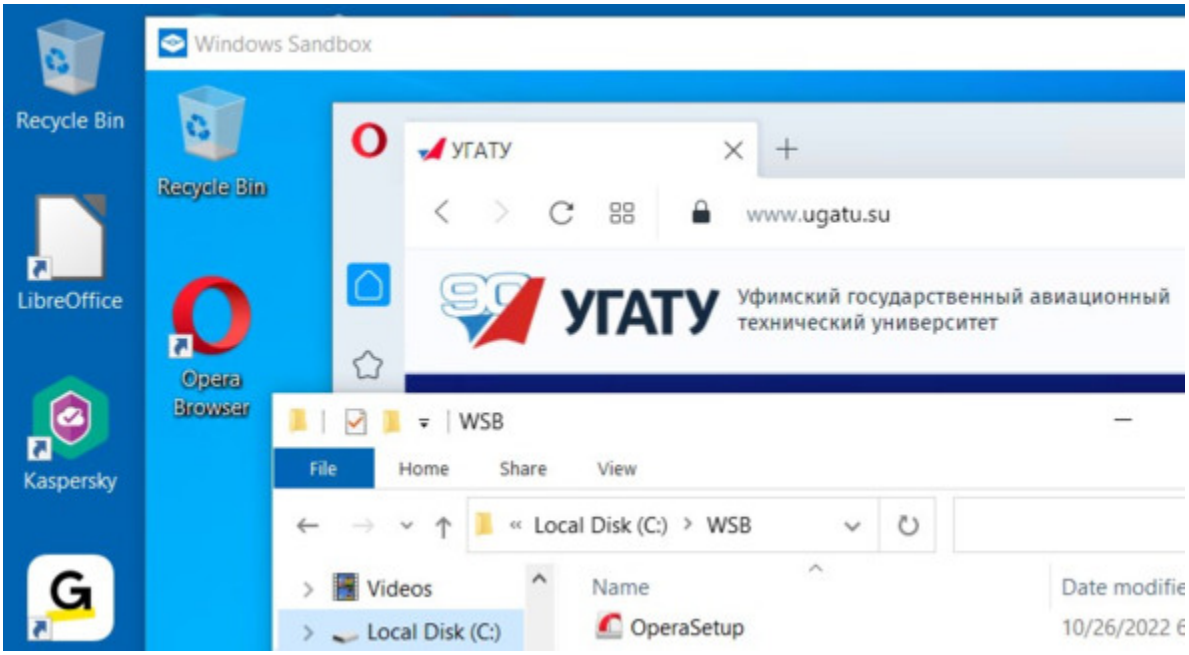


Рис. Браузер Opera в Песочнице

Задание

— Скачать файл OperaSetup. exe на сайте
— https://www.opera.com/
— Разместить файл в общей папке
— Создать файл *.WSB для установки программы
— Открыть файл *.WSB с помощью Песочницы
— Убедиться в работе установленного браузера
— Перейти на сайт вуза

WINDOWS SUBSYSTEM FOR LINUX (WSL)

Следующий этап виртуализации — это полная интеграция гостевой операционной системы. Полное название: подсистема Windows для Linux — Windows Subsystem for Linux (WSL). Этот компонент появился в Windows 10. Он точно также включается через управление компонентами Windows, см. рис.

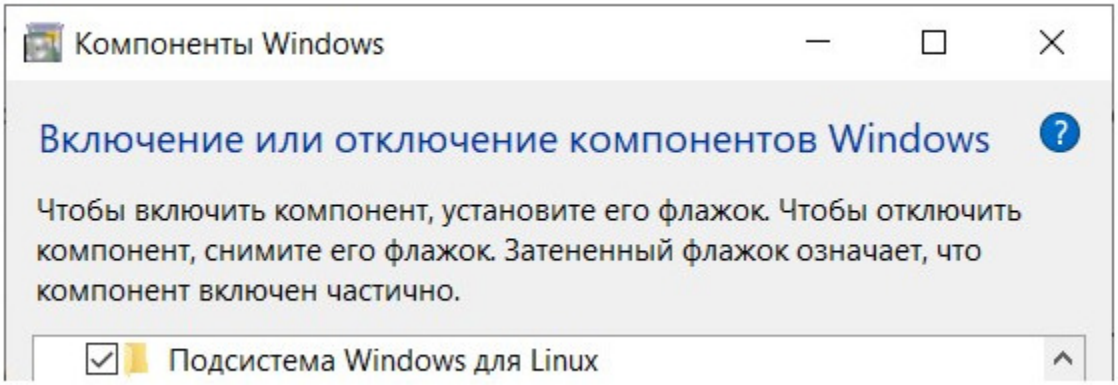


Рис. Включение WSL

Мы включаем этот компонент, и далее мы сможем параллельно работать в двух операционных системах.

MICROSOFT STORE — UBUNTU

В качестве эксперимента мы устанавливаем Ubuntu. Это одна из самых популярных версий Linux, особенно для облачных приложений. Эта ОС устанавливается теперь через магазин приложений Microsoft Store. Мы устанавливаем Ubuntu именно через магазин приложений. Там уже есть несколько вариантов Linux — можно задать поиск и пересчитать доступные варианты. Можно почитать описание Ubuntu. Здесь говорится, что эта версия прежде всего предназначена для работы в терминале, то есть в командной строке. Это позволяет разрабатывать кроссплатформенные приложения, то есть программы, которые будут работать на нескольких разных платформах. Это бесплатная версия, и она ставится поверх **WSL**.

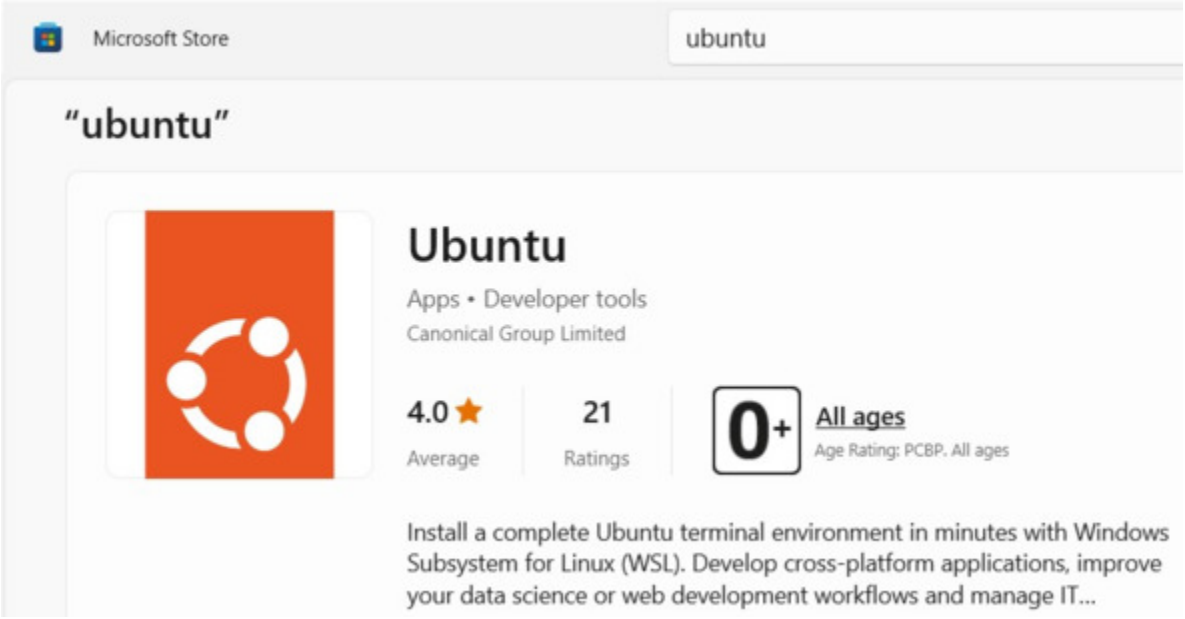


Рис. Установка Ubuntu

MICROSOFT STORE — WINDOWS TERMINAL

Дополнительно мы уставноим еще один бесплатный инструмент, который называется Windows Terminal. Слово «терминал» пришло из Unix и из Linux. Оно означает «командное окно». Сейчас оно проникло и в операционную систему Windows. Точно также через магазин приложений Microsoft Store мы устанавливаем Windows Terminal, см. рис. В описании программы мы видим символ приложения — иконку — приглашение к вводу командной строки.

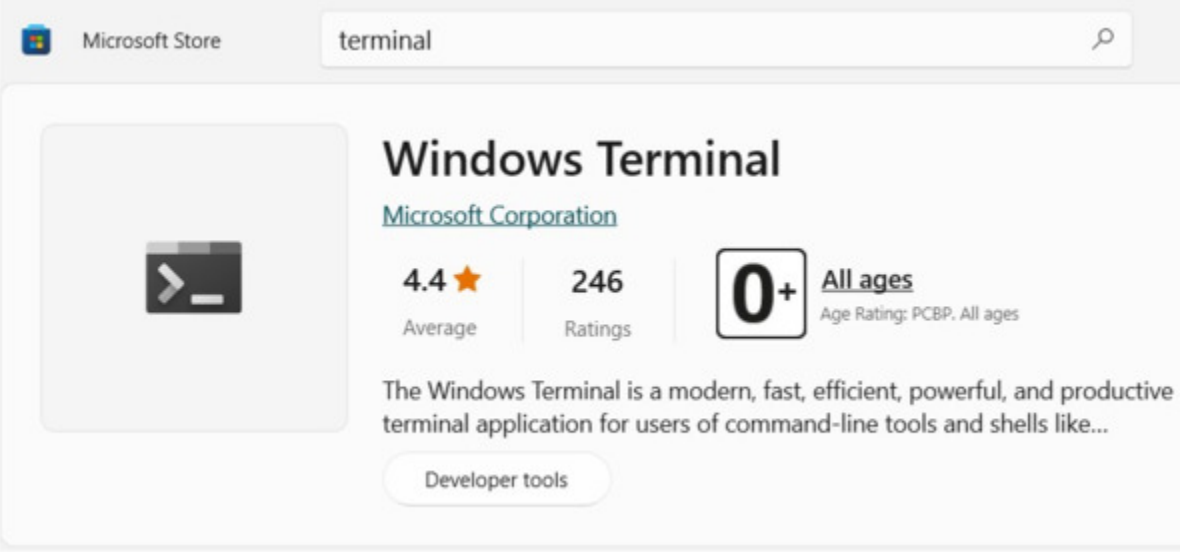


Рис. Установка Terminal

WINDOWS TERMINAL + UBUNTU

Как же работает Терминал Windows? По сути, это командное окно, в котором открывается несколько вкладок. В зависимости от того, что у нас уже установлено, мы можем выбрать нужный инструмент. Command Prompt — это обычная командная строка Windows. PowerShell — это командная строка с новыми возможностями, чаще всего для администратора. Теперь у нас есть возможность зайти в уже установленный вариант Ubuntu. Если установить систему управления версиями Git, то мы сможем выбрать эмулятор командной строки Git Bash.

То, что мы установим, то и появится в списке. В каждой вкладке будет свой вариант командного окна. Между ними можно будет переключаться.

На рис. показан пример. Мы зашли в одной из вкладок — в ту самую операционную систему Ubuntu. У нас появляется приглашение, оформленное уже в другом стиле. Это уже не Windows. Это Linux, и в этом окошечке мы сможем с ним с этим линуксом поработать.

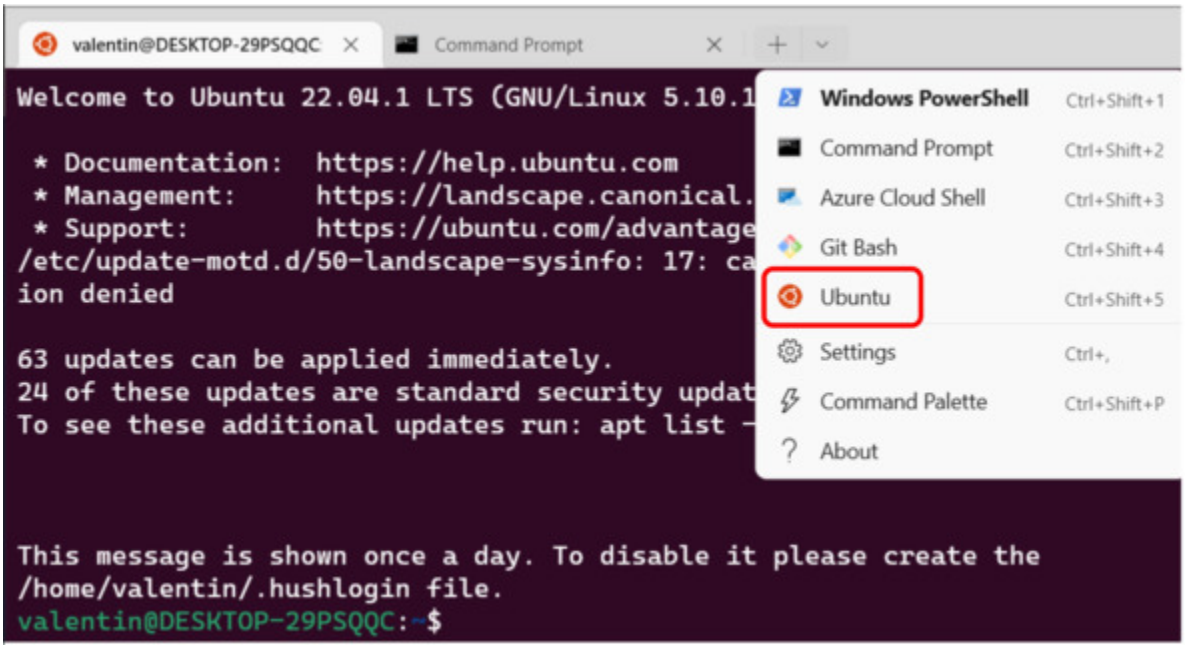


Рис. Выбор оболочки в терминале

СОЗДАНИЕ СКРИПТА

В качестве эксперимента мы проделаем примерно то же, что и на наших лабораторных работах. Мы создаём скрипт. Напомним, что такое скрипт. Беквально английское слово «script» означает «сценарий». В операционных системах это текстовый файл, в котором написаны команды ОС. В Линуксе это называется «скрипт», а в Windows — «пакетный файл». Итак, мы создаём скрипт и запускаем его на выполнение. Нам понадобится ввести несколько команд, и мы их сейчас поясним, см. рис.

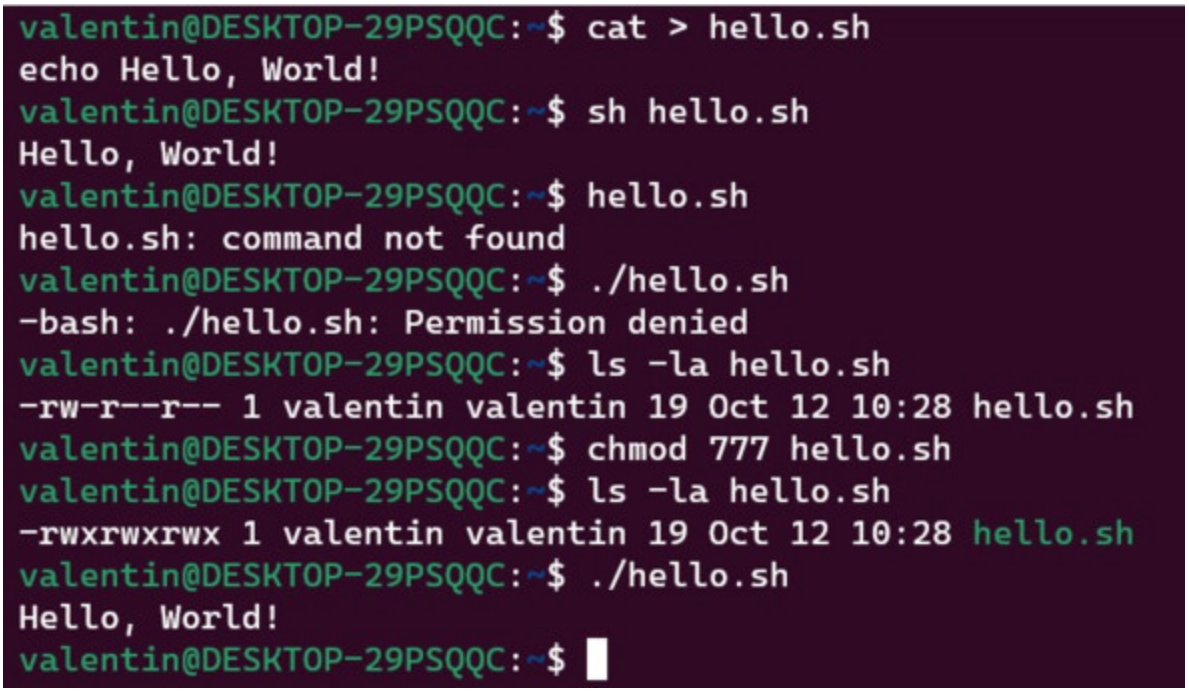


Рис. Создание и запуск скрипта

Первое, что мы делаем: мы вводим с клавиатуры текст и отправляем его в текстовый файл. В Линуксе это делается так. Вводим команду cat.

Это сокращение английского слова «concatenate» — «провести конкатенацию», то есть «объединить несколько элементов в одну цепочку, последовательность». Эта команда позволяет выводить содержимое файла на экран, а также объединить несколько файлов и потом уже вывести всё это на экран. Отсюда и такое название.

Обычно команда **cat** выводит текст из файла на экран. Но здесь мы используем «стрелочку вправо», или «правую угловую скобочку», или значок «больше». Так мы перенаправляем вывод вместо экрана в текстовый файл **hello.sh**. Расширение ***.sh** означает **Shell**. Вообще-то **Shell** — это «оболочка ОС», то есть командная строка. Здесь имеется в виду название «**shell file**», или «**Shell script**», то есть сценарий для выполнения в командной строке / оболочке ОС.

Задание. Просмотрите статью «**Shell script»** в Википедии.

Итак, мы вводим с клавиатуры команду **cat** и перенаправляем её вывод в файл. По умолчанию в качестве входной информации она берёт то, что мы вводим с клавиатуры. Мы сделаем в нашем скрипте всего одну команду: **echo** пробел и какое-то сообщение. Эта команда будет выводить наш «привет» на экран.

Обычно по окончании ввода мы нажимаем комбинацию клавиш **[Ctrl+Z]** — это когда мы работаем в виртуальной машине — чтобы закончить ввод. А здесь используется комбинация **[Ctrl+D]**. И на экране ничего не появляется, просто выводится приглашение к вводу новой команды. Если этого не сделать, будут неприятности.

Итак, мы ввели текст, и у нас появляется «файл-сценарий». Его можно запустить на выполнение с помощью той самой оболочки **Shell**. Мы пишем **sh**, пробел и имя файла скрипта. Он выполняется и выводит своё сообщение.

В принципе, мы могли бы запустить этот файл на выполнение попроще. Достаточно написать его название и нажать **Enter**. Мы пишем название файла, нажимаем **Enter**, и нам сообщают: файл не найден. Хотя мы можем убедиться, что он есть. Это особенность Linux. Для того, чтобы использовать файлы в текущем каталоге, нужно сказать, что они находятся именно в текущем каталоге. Для этого мы пишем **точка**, **наклонная черта**, **имя файла**. Одна точка означает «текущий каталог». Две точки означают «родительский каталог», то есть каталог «одним этажом выше по файловому дереву». Вот теперь мы действительно запускаем файл на выполнение.

Однако, нам говорят: «permission denied» — «отказано в доступе», или «доступ запрещен». У нас пока нет доступа на выполнение. У нас имеется файл, но мы не можем его запустить на выполнение. Сначала нужно установить разрешение. Для этого мы используем команду **chmod** — «**change mode»** — «изменить режим доступа». Три семерки означают полный доступ для любого

пользователя. Подробнее это разбирается у нас в разделе «Файловые системы». Итак: **change mode**, **три семерки** и **имя файла**.

Теперь мы можем запустить этот файл на выполнение, и он выполнится.

Мы можем посмотреть, какие атрибуты есть у этого файла. Здесь нас интересуют разрешения. На самом деле, есть разница между атрибутами и разрешениями, и в разных ОС это работает немного по-разному.

Мы вводим команду **ls** — то есть **LIST** — «список файлов». Дополнительные ключи **–la** говорят, что нужно вывести на экран все подробности — от слова **all**, то есть «всё». Символы **rwx** — это разрешения на Read-Write-eXecute, то есть на чтение, запись и выполнение файла. Это полный доступ к файлу для любого пользователя. Вот такой эксперимент вам нужно произвести в рамках Ubuntu который устанавливается поверх WSL.

Самое главное — не забываем: Shell Script — end of file: ^D.

UBUNTU + MIDNIGHT COMMANDER

Следующий эксперимент — это уже файловый менеджер под названием Midnight Commander. По умолчанию он не входит в состав Ubuntu, и его нужно дополнительно установить. Тогда он будет работать. Для этого нам нужно будет ввести три команды, см. рис.

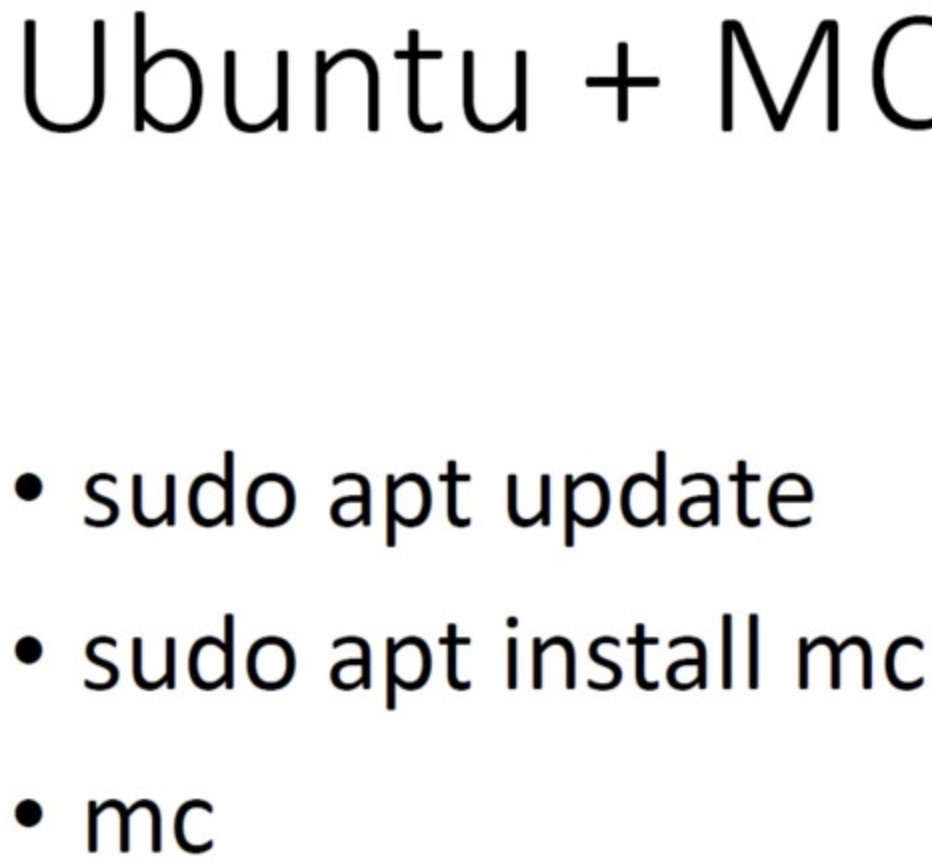


Рис. Установка MC

Первая команда **update** обновляет список ресурсов, откуда можно скачать установочную версию. **APT** — это приложение, которое занимается установкой программ. Причём программы в Линуксе называется словом «пакеты». В русском языке мы иногда тоже встречаем такое выражение: «пакет прикладных программ».

Вызываем пакетный менеджер **apt** или **apt-get**. Перед этим пишется **SUDO**. Это значит «запустить с правами администратора». А для этого у нас должны быть права администратора, у нас должен быть пароль администратора. Либо придется позвать лаборанта, чтобы он ввёл пароль. Итак, **update** обновляет базу данных адресов, откуда можно скачивать установщики.

Далее, во второй строчке мы устанавливаем программу — **install** и **название программы**.

После этого мы можем в командной строке написать название программы mc и нажать **Enter**. Теперь наша программа запустится, и мы получаем в рамках нашего терминала файловый менеджер **Midnight Commander** со всеми его возможностями, со всеми его функциями.

Перед нами файловая система Linux внутри операционной системы Windows. Это продолжение идеи виртуальных машин.

Здесь мы тоже можем получить доступ к жёстким дискам основной машины через каталог **mnt** — сокращение от слова MOUNT — «монтировать», то есть «подключить диск или каталог». Далее указываем букву диска и нужный каталог.

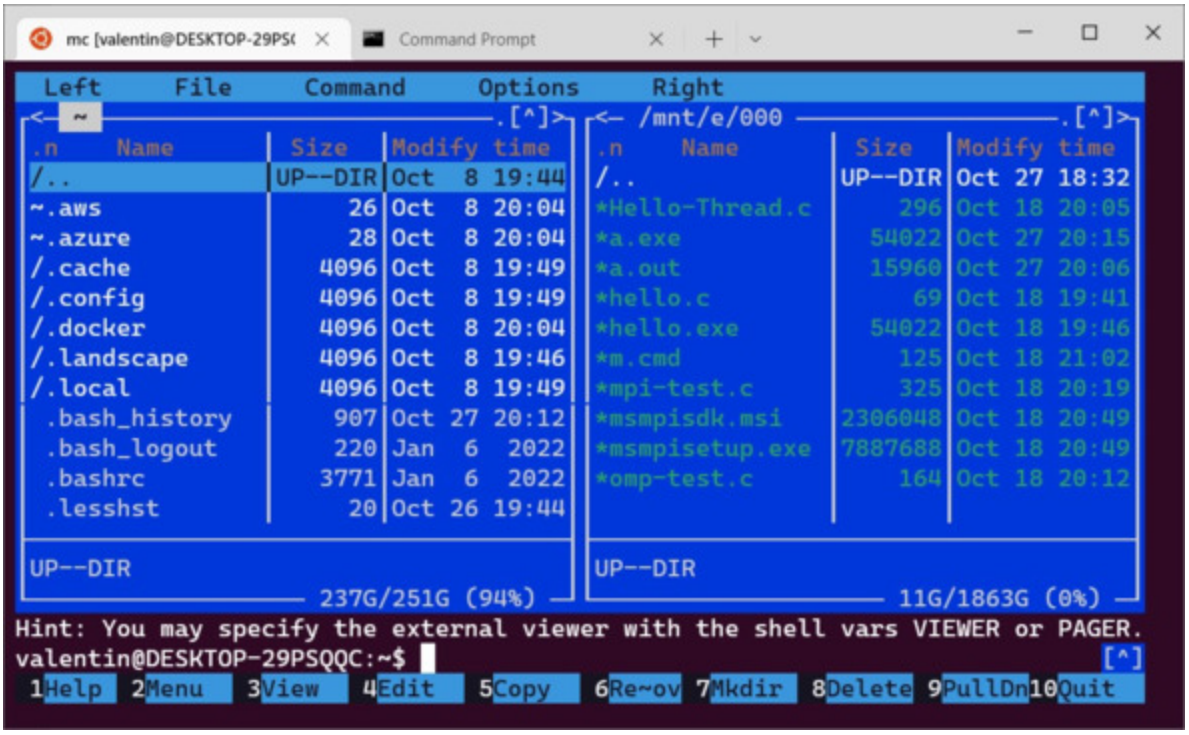


Рис. Файловый менеджер **MC** с доступом к диску **E:**

Нужно повторить все эти этапы, прочувствовать как это всё происходит: включаем WSL, устанавливаем Ubuntu и Терминал Windows, запускаем скрипт, запускаем Midnight Commander и убеждаемся, что всё работает.

Если мы запускаем программу из Midnight Commander, то она выводит сообщение и тут же это сообщение исчезает. Перед нами снова будут синие панели. Напомним: чтобы заглянуть за них, за эти «занавески», нужно открыть то самое «чёрное командное окно». «Открыть» по-английски будет «**open**». Поэтому используется комбинация клавиш **[Ctrl+O]**. Если всё это проделать, мы снова увидим чёрный экран, на котором будет тот самый «привет» от нашей программы-скрипта.

Задание

— Включить **WSL**
— Установить приложения из **Microsoft Store**:
— **Ubuntu** (пароль администратора)
— **Windows Terminal**
— Открыть **Terminal**
— Запустить окно **Ubuntu**
— Создать скрипт **Hello, World!**
— Установить разрешение на выполнение
— Запустить скрипт на выполнение
— Установить **Midnight Commander**
— Запустить **Midnight Commander**
— Запустить скрипт на выполнение
— **Ctrl + O**

ПРОГРАММА HELLO, WORLD! НА ЯЗЫКЕ СИ

Составим программу на стандартном языке Си. Самая простая программа, которая выводит приветствие «Hello, World!». Мы снова используем команду **cat**, вводим текст с клавиатуры и перенаправляем вывод в файл. Либо мы можем ввести текст

в Блокноте, сохранить его в файле и вывести на экран той же командой **cat**, см. рис.

```
valentin@DESKTOP-29PSQQC:/mnt/e/000$ cat hello.c
#include <stdio.h>
int main()
{
        printf("Hello, World!\n");
}
```

Рис. Программа Hello, World!

Далее скомпилируем нашу программу. Для этого понадобится бесплатный компилятор **GCC**. Как видим, он пока не установлен, но нам сразу дают подсказку, как его установить, см. рис.

Команда **APT** полностью организует весь процесс установки и — если нужно — устанавливает любые дополнительные библиотеки и пакеты программ — то, что называется **dependencies** — «зависимости». Мы с этими зависимостями ещё столкнёмся.

```
valentin@DESKTOP-29PSQQC:~$ gcc
Command 'gcc' not found, but can be installed with:
sudo apt install gcc
valentin@DESKTOP-29PSQQC:~$ sudo apt install gcc
[sudo] password for valentin:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  cpp cpp-11 fontconfig-config fonts-dejavu-core gcc-11
```

Рис. Устанавливаем компилятор GCC

Теперь переходим из домашнего каталога Линукса **/home/valentin** в каталог **/mnt** и находим здесь ссылки на все жесткие диски Windows. Переходим в каталог, где лежит наша программа, компилируем её и запускаем на выполнение исполняемый файл **a. out**, см. рис.



Рис. Компиляция программы в каталоге **E:\000**

Тот же самый файл мы сможем скомпилировать и запустить в среде Windows безо всяких доработок, см. рис. Используем бесплатный компилятор **g++**, компилируем программу и запускаем исполняемый файл **a. exe**.

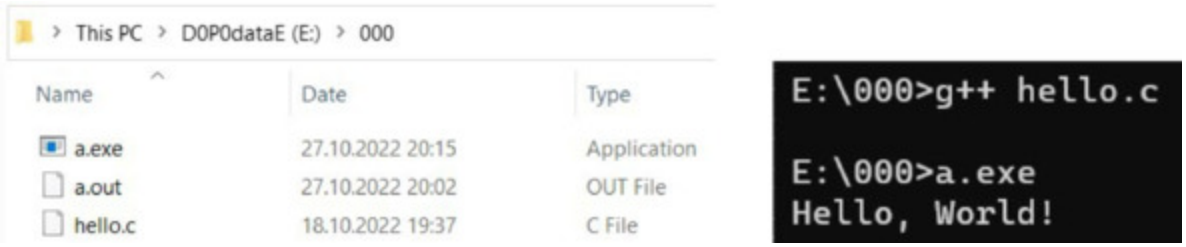


Рис. Компиляция в среде Windows

Интеграция Windows и Linux организована так, что мы сможем даже запускать оба вида программ, см. рис. Мы можем их вызывать даже в рамках одной строки. Здесь два амперсанда **&&** между

командами означают, что вторая команда выполняется только при условии, что первая команда прошла без ошибок.

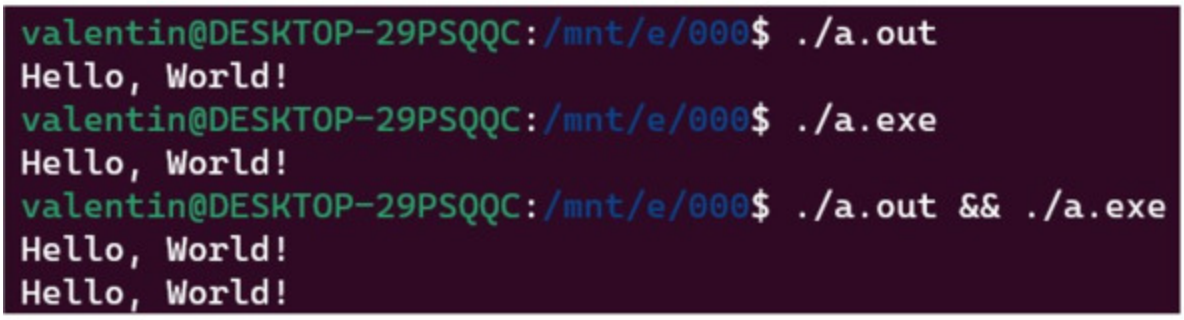


Рис. Смешанное выполнение программ

Точно так же мы можем вызвать команду Windows, находясь в командной строке Linux, см. рис. Здесь команда **ATTRIB** выводит список файлов и их атрибуты. Обратим внимание, что в Windows чаще говорят про «атрибуты» файла, а в Linux чаще упоминают «разрешения» — permissions.



Рис. Вызов команды Windows в среде Linux

Мы можем даже запускать оконные приложения Windows из командной строки Linux, см. рис. В данном примере мы вызываем Проводник — Explorer. После название команды мы указали точку.

Этот дополнительный параметр означает «текущая директория, каталог, папка». Причём речь здесь идёт о текущем каталоге Linux.

```
valentin@DESKTOP-29PSQQC:/mnt/e/000$ cd
valentin@DESKTOP-29PSQQC:~$ pwd
/home/valentin
valentin@DESKTOP-29PSQQC:~$ explorer.exe .
```

Рис. Запускаем **Проводник** Windows в текущем каталоге **WSL**

После запуска Проводника можно обнаружить, где находится ссылка на каталог **WSL**. Это спрятано где-то в сетевом окружении.

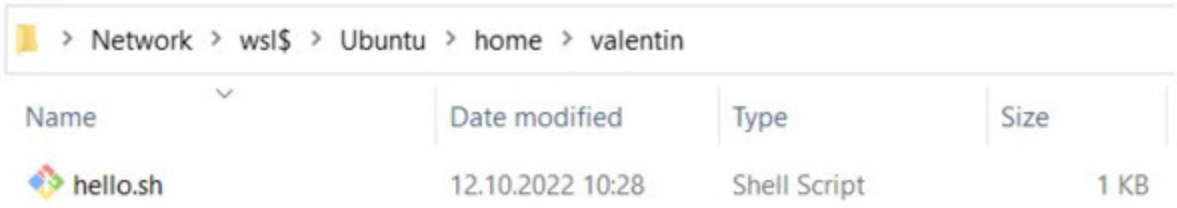


Рис. Проводник Windows в текущем каталоге Linux

Утилита **wsl** позволяет запускать команды Linux из командной строки Windows, см. рис.

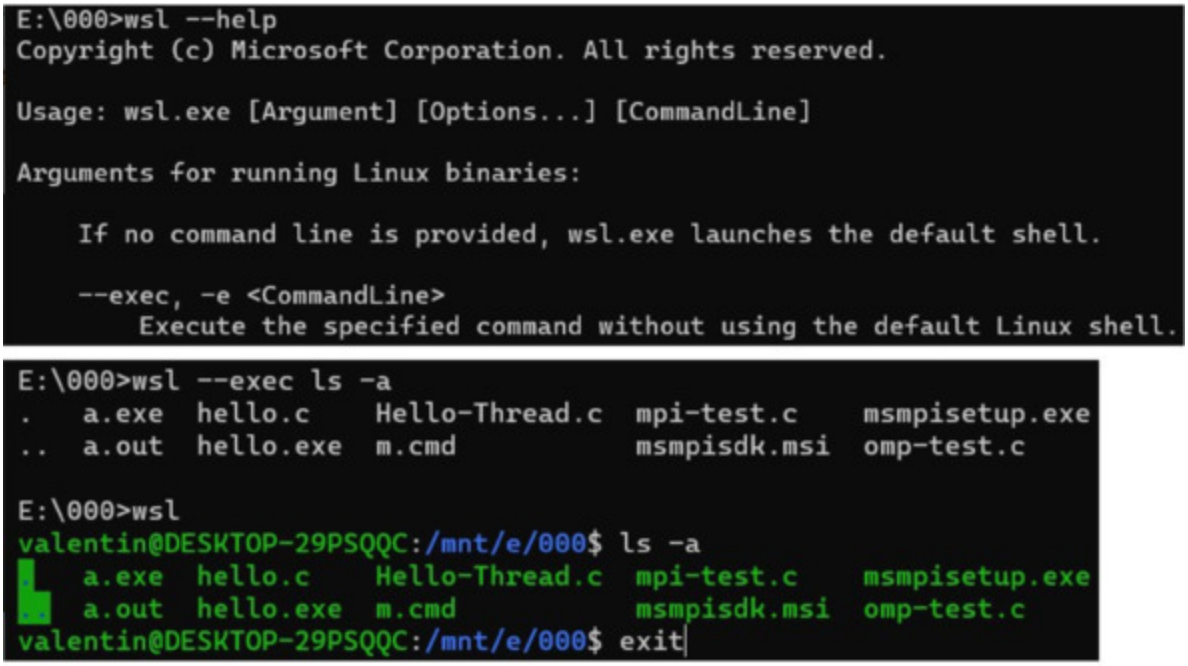


Рис. Запуск команд Linux через WSL

Далее мы можем организовать смешанный конвейер из команд Windows и Linux, см. рис. Попробуйте разобраться, что здесь на самом деле происходит, и откуда какая команда вызывается.

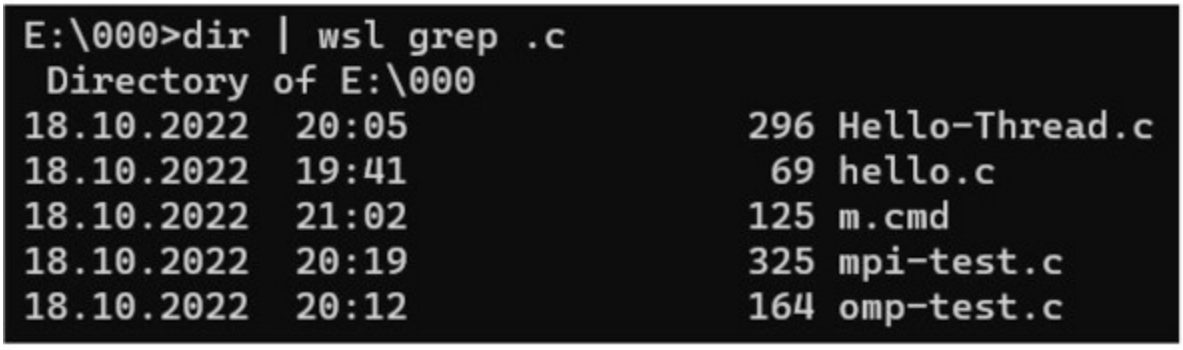


Рис. Конвейер команд Windows и Linux

Мы уже рассмотрели соответствие путей в Windows и **WSL**. Ещё одна любопытная утилита WSLPATH немного упрощает эту работу. Она занимается преобразованием путей, см. рис.

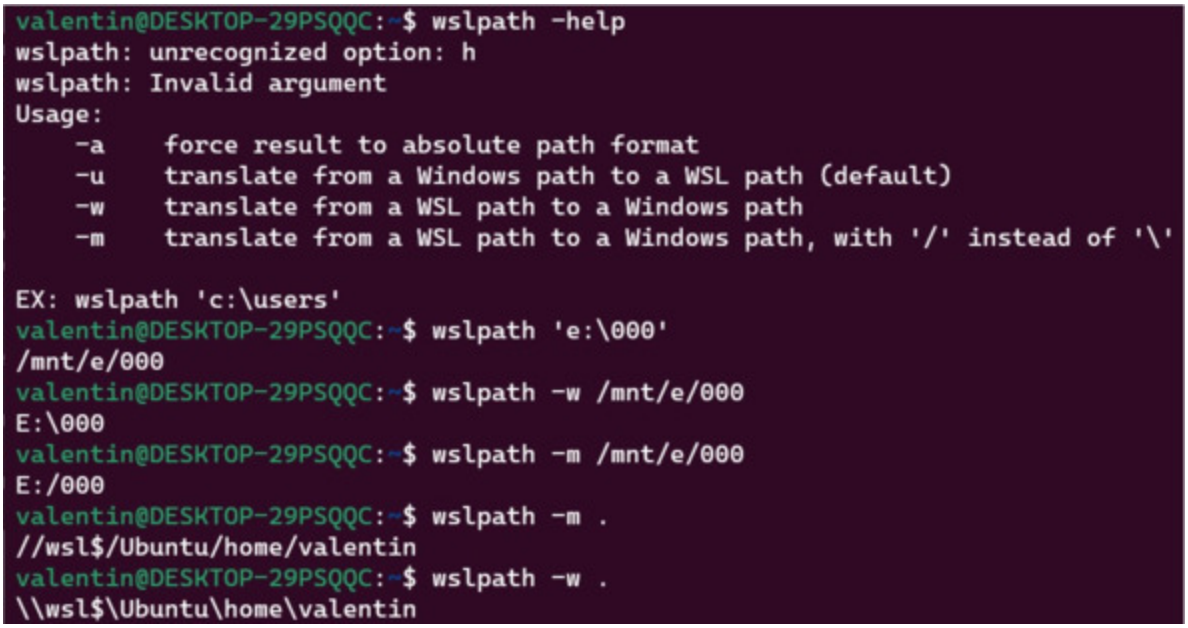


Рис. Утилита WSLPATH: преобразование путей

Следующий эксперимент — запуск программы **Блокнот** — **Notepad** в фоновом режиме. Здесь мы используем специальный символ — одиночный амперсанд **&**.

При запуске программы в фоновом режиме она продолжит выполняться в отдельном окне, а мы сможем продолжить работу в командной строке. Но если просто запустить Блокнот без этого значка амперсанд, то в командной строке не появится приглашения к вводу команд. Она будет ожидать завершения запущенной программы. Мы закрываем окно Блокнота и теперь сможем продолжить работу в командной строке.

.

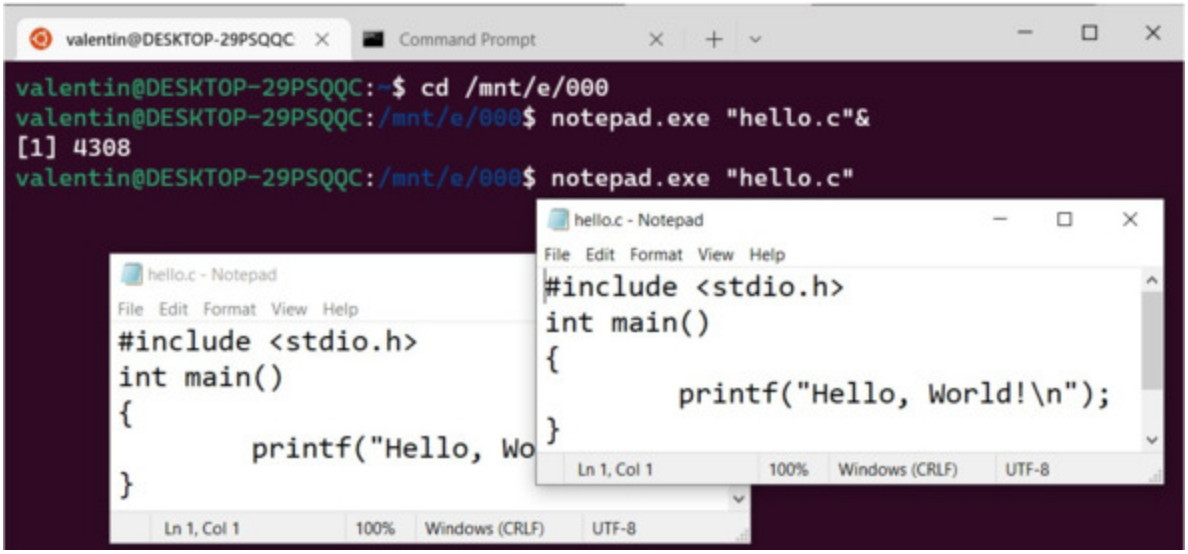


Рис. Запуск Блокнота

Наконец, заключительный опыт с **WSL** — утилита **FINDSTR**. Мы работаем в консоли Linux и вызываем команду **LS**. Далее мы пропускаем полученный список файлов через «фильтр» Windows и выбираем только строки с указанными символами.

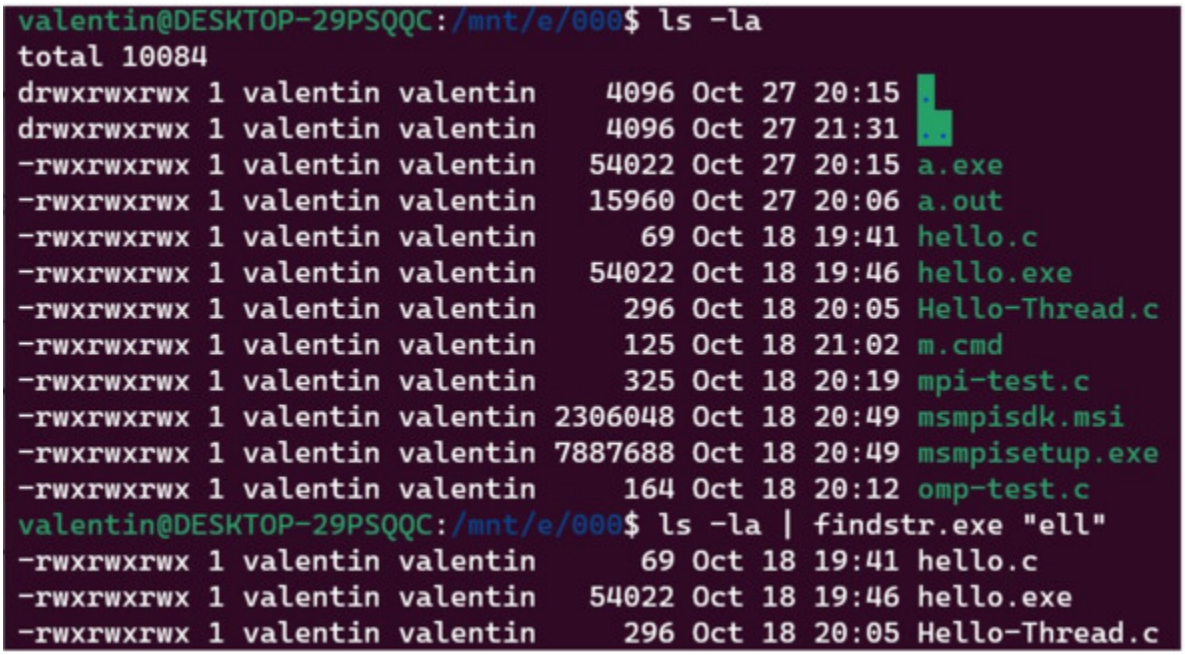


Рис. Утилита **FINDSTR**

Повторите все показанные эксперименты и убедитесь, насколько сильно проведена интеграция двух разных операционных систем.

MICROSOFT STORE — LINUX

Мы с вами посмотрели, как работает Ubuntu в среде WSL.

Если зайти в магазин приложений Microsoft Store и задать поиск по ключевому слову Linux, мы обнаружим и другие варианты Linux. Вызовите поиск и выясните, какие ещё варианты Линукса доступны для установки поверх WSL.

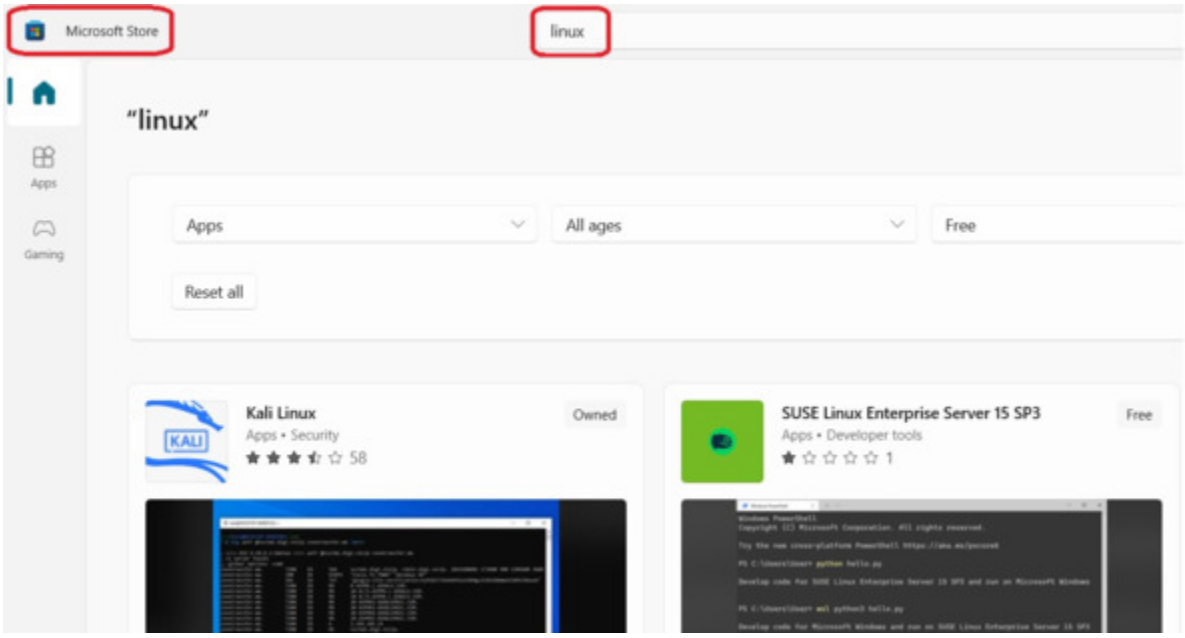


Рис. Поиск в магазине приложений

КОНТЕЙНЕРЫ

В первой части нашего пособия мы посмотрели, как работают виртуальные машины. Это было очень актуально до самого последнего времени. И это продолжает использоваться.

Однако, на сегодняшний день — в последние лет пять — к виртуализации добавились новые технологии, которые называются «контейнеры». Вот с этими контейнерами мы и будем дальше знакомиться.

КОНТЕЙНЕРИЗАЦИЯ

В чем же смысл контейнеров? Здесь внутри изолированного окружения запускается одна-единственная прикладная программа — Application.

Как это выглядит? Внизу, в основе у нас есть «железо» — Hardware. У нас есть основная операционная система. Поверх неё запускается некий сервис — Engine, который будет обслуживать наши контейнеры. Внутри контейнера будет крутиться прикладная программа, см. рис.

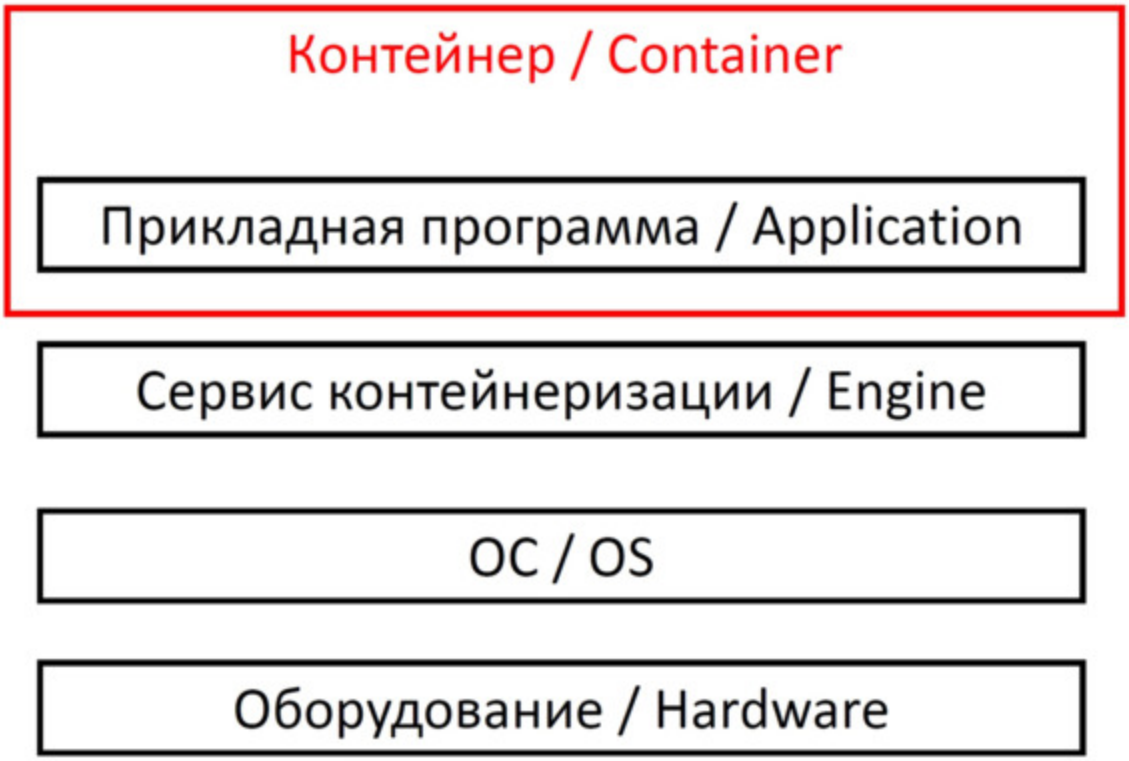


Рис. Контейнеризация приложений

В чём разница, в чём отличие контейнеров от виртуальных машин? Разница в размерах. Виртуальная машина — это несколько гигабайт, а контейнер — это несколько мегабайт. Разница в тысячу раз! Для многих задач, для многих простых программ это огромная экономия ресурсов. Итак, разница в сотни или в тысячи раз по размерам — это уже интересно.

Здесь у нас на схеме есть такое интересное слово: **Engine**. Чаще всего эти сервисы называются Engine, что переводится как «движок». По сути, это некая платформа, поверх которой запускается что-нибудь ещё. В самых разных ситуациях это слово — «движок» можно встретить. Многие компьютерные игрушки построены на каком-то «движке», то есть внутри у них может быть одна и та же платформа. Итак, «движок» — это некий сервис.

Что же такое «контейнер»? Это прикладная программа, которая полностью изолирована от всего остального. Но в рамках этого контейнера находится всё, что ей понадобится для работы. Это может быть часть операционной системы. Это могут быть дополнительные библиотеки. Это могут быть дополнительные программы, которые нужны для этой программы. Такие дополнительные программы называются «зависимости» — dependencies. Это могут быть какие-то файлы, какие-то настройки, какие-то параметры. И всё это упаковано в один особый файл. Его запускают, и оно работает. И самое главное, оно не оставляет после себя никаких следов в операционной системе. Это чем-то напоминает переносимые приложения Portable Applications. Но теперь это полностью изолированное приложение.

КОНТЕЙНЕР ДЛЯ ПЕРЕВОЗКИ ГРУЗОВ

Слово «контейнер» было позаимствовано из реальной жизни. При перевозке грузов используется такая технология, как контейнеры. Это большие ящики, куда загружается товар и всё, что нам нужно для его доставки — в целости и сохранности. Внутри такого ящика могут быть амортизаторы, мягкие прослойки — чтобы ничего не разбилось. Есть ситуации, когда внутри контейнера нужно поддерживать заданную температуру. Всё, что нужно для перевозки груза, находится внутри этого железного ящика.

Мало того, нынешние контейнеры — это вещь универсальная. Для транспортных задач строится универсальный контейнер — Shipping container, который просто перегружают с одного вида транспорта на другой:

— По морю на корабле (ship)
— По железной дороге (rail)
— На грузовом автомобиле (truck)
— По реке на барже (barge)
— По воздуху на самолете (air)

У нас есть баржи, которые тянут по речкам и по озерам. Баржа отличается от корабля, у неё плоское дно, её тащит «тягач». А контейнер тот же самый, универсальный.

Все эти металлические коробочки могут быть разного размера, но более-менее стандартные.

Вот эта идея и была позаимствована при построении программных контейнеров. В последнее время всё более модным становится слово «логистика», то есть «перевозка, доставка и хранение» любых грузов — в том числе, для интернет-магазинов. Даже появилась целая специальность — «Цифровая логистика».

Задание. Просмотрите в Википедии следующие статьи:

— Shipping container

— Intermodal freight transport

Итак, контейнеры. За этим термином скрывается целая история, десятки лет развития юникса и линукса. Это всё формировалось и развивалось постепенно, но в последние несколько лет мы наблюдаем взрывной рост популярности контейнеров. Потому что технология стала очень простой и очень доступной.

Мы с вами разбираем этот подход на примере технологий под названием **DOCKER**. Надо сказать, что Докер — это на самом деле только пример, это один из инструментов для работы с контейнерами. Но для нас эта платформа бесплатная, и к тому же имеется огромная библиотека готовых приложений в контейнерах. Когда вы это освоите, то легко сможете освоить и любую другую платформу для контейнеров.

Задание. Просмотрите в Википедии следующие статьи:

— Контейнеризация

— OS-level virtualizations

— Пользовательское пространство

— Адресное пространство (информатика)

— namespace

— Пространство имён

— Контрольная группа (Linux)

— cgroups
— LXC

УСТАНОВКА ДОКЕРА

Чтобы установить Docker, мы заходим на сайт:
https://www.docker.com/

Находим кнопочку Download и скачиваем вариант для нашей платформы. Здесь можно увидеть, что есть варианты для Apple, для Linux, для Intel, для Windows. На чём хотите, на том и можно эту платформу развернуть. Мы скачали Docker Desktop Installer. Запускаем и устанавливаем программу, см. рис.

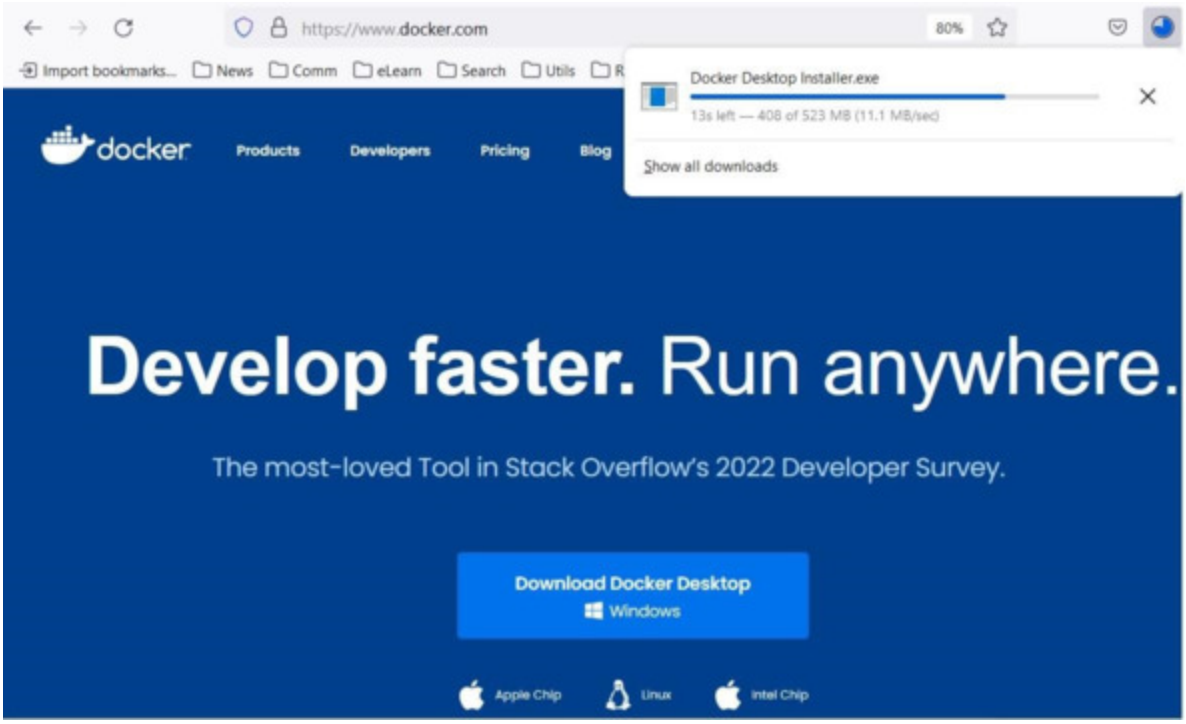


Рис. Установка Докера

DOCKER DESKTOP

Мы установили Desktop, то есть это настольная приложение — для настольного компьютера, работающее в отдельном окне рабочего стола. Мы запускаем эту программу и видим новое окошечко. Здесь будет несколько слов, с которыми нам предстоит разобраться:

— Containers — Контейнеры
— Images — Образы
— Volumes — Тома файловой системы

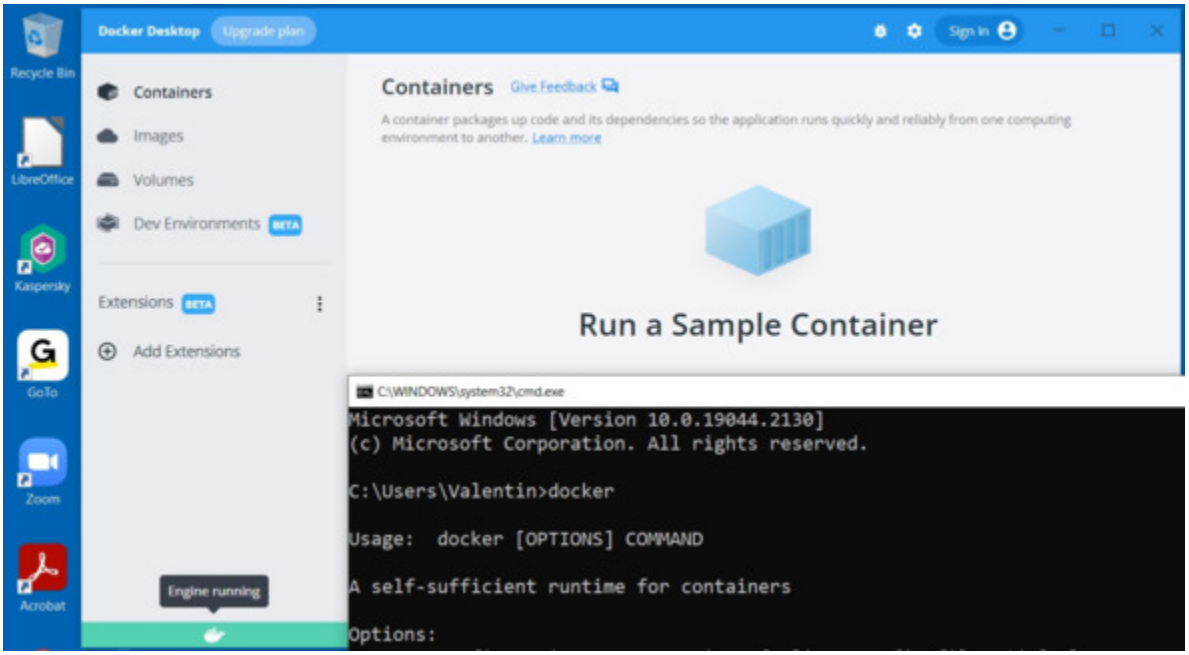


Рис. Настольная версия Docker

После установки и запуска платформы мы можем написать в командном окне слово docker. Мы получим первое сообщение, где говорится, что докер — это инструментарий для запуска контейнеров. Буквально **runtime** означает «во время выполнения», «в момент работы программы». Итак, перед нами библиотека, инструментарий, который позволяет запускать контейнеры.

Здесь еще говорится self-sufficient, то есть это «полный набор, для которого больше ничего не нужно», «самодостаточный

инструмент» для запуска контейнеров.

КТО ТАКОЙ DOCKER?

Пришло время разобраться, кто такой «докер». Слово это иностранные, и оно имеет свое значение — совсем даже не очевидное. Это очередная «ловушка», «ложный друг переводчика».

Английское слово **docker** означает «грузчик», то есть «тот, кто перемещает грузы с места на место» — в том числе контейнеры. Ещё это называется «портовый рабочий». И смысл в том, что слово **docker** происходит от слова **dock**. Русское слово «док» означает «место, где корабль встаёт на ремонт». Английское слово **dock** означает «любой порт, куда причалил корабль» — в том числе, для погрузки и для разгрузки. Так что DOCK и ДОК — это разные слова, которые звучат одинаково.

Английское слово **dock** означает «порт». Соответственно, **docker** — «портовый рабочий». Он занимается погрузкой-разгрузкой контейнеров. И это сокращение. Полное слово звучит как **dockworker**, то есть работник порта. А сокращенно получилось **docker**.

У нас описание этой программы

https://docs.docker.com/

В качестве упражнения нужно будет установить Desktop-приложение. Дальше нужно открыть командное окно Command Prompt. Кстати, в рамках первой лабораторной работы вы знакомитесь с тем, что можно десятью разными способами запустить командное окошко Windows.

Итак, в командном окне мы пишем **docker** без параметров и смотрим сколько у него возможностей. Получаем список на несколько страниц из тех команд и настроек, которые нам доступны. Вам нужно будет просто обратить внимание на те возможности, которые тут есть.

Задание. Просмотрите в Википедии следующие статьи:
— Dock
— Dockworker
— Docker_ (software)
— Docker logo

Задание
— Установите Docker Desktop
— Запустите Docker Desktop
— Откройте командное окно / Command Prompt: Win + R, CMD
— Запустите docker без параметров: docker
— Обратите внимание на количество команд и параметров

ОРКЕСТРАЦИЯ

Мы можем запустить один-два контейнера вручную, а что дальше? Контейнеров может быть не один и не два, их могут быть сотни и тысячи. Здесь возникает проблема управления большим количеством контейнеров. И это называется «оркестрация» — ещё более модная технология, чем докер — под названием CUBERNETES.

Если вы «погуглите» это слово, то обнаружите, что в последние годы наблюдается взрывной рост интереса к этому инструменту. Предлагается множество курсов по «куберу», требуется масса специалистов. Эта тема такая популярная потому, что растет интерес к контейнерам и к управлению ими.

Конечно, слово «оркестрация» больше намекает на управление оркестром и на разделение музыки на отдельные партии для разных инструментов. Но проблему оркестрации обычно объясняют, сравнивая множество программных контейнеров с домашними животными. Представьте себе, что у вас дома есть домашние животные — всего одно или два. Кошка, собачка, или ящерица. У него есть кличка — Пушок или Шарик. Конечно об этом одном-

единственном любимом животном хозяин будет заботиться, как о члене семьи.

А теперь представьте, что у вас не одна, а целая тысяча кошек, или собак, или баранов, или коров. Самое большее, у них может быть порядковый номер (написан на боку). А может даже и номера не быть. Как управляться с таким стадом? Приходится по-другому работать и обращаться с ними уже не так ласково и нежно. Если какая-то корова или овечка заболела, то скорее всего, с ней будут поступать не так, как с любимым домашним питомцем. Очень легко будет заменить на другое животное.

Точно так же поступают и с контейнерами. Если их много и с кем-нибудь из них что-то произошло, его физически устраняют и заменяют на другой. Программисты так и говорят: «убить» — «kill».

Задание. Просмотрите в Википедии следующие статьи:

— Orchestration (computing)

— Оркестровка (ИТ) — Оркестрация

— Kubernetes

— Docker (software) — Docker Swarm

— OpenShift

— Хабр

— Оркестрация данных

— Kubernetes и другие оркестраторы

Сайты разработчиков

— https://kubernetes.io/ru/docs/concepts/overview/what-is-kubernetes/

— https://www.docker.com/resources/what-container/

— https://www.hashicorp.com/products/nomad

— https://www.redhat.com/en/technologies/cloud-computing/openshift

МИКРОСЕРВИСЫ

Ещё один интересный термин, относящийся к этой системе, это так называемые «сервисы». По сути, контейнеры — это инструмент разработки сложных программных продуктов. Большую программу делят на мелкие кусочки, и каждый элемент работает независимо, отдельно от остальных. Эти части только общаются друг с другом, и каждая часть может быть реализована в виде контейнера. Такой подход называется «микросервисная архитектура».

Микросервисы — это очередной этап в развитии программных систем. Первоначально простые программы представляли собой нечто единое, целое. Называлось это «монолитная архитектура», «монолитная система». Дальше постепенно начали делить большую систему на отдельные модули. Даже в рамках операционных систем есть ядро операционной системы (kernel) и есть множество библиотек, есть оболочки. Следующим шагом — в том числе и в операционных системах — стало так называемое микро-ядро (microkernel) и несколько колец защиты. То же самое мы наблюдаем и в рамках прикладных программ, информационных систем. Это называется «сервисно-ориентированная архитектура». На сегодняшний день много говорят про «микросервисы» и так называемые «бессерверные приложения».

Если первоначально большую программу выкладывали на сервер, и она там работала. Сейчас решили работать вообще без сервера. Вам предстоит узнать самостоятельно, что такое бессерверные приложения / вычисления — тоже очень модное название — serverless computing.

Почему при разработке программных продуктов всё чаще используются контейнеры? Потому что разработка программы делается на одном компьютере, а тестирование на другом. Но нужно обеспечить одинаковые условия и для разработчика, и для тестировщика, и при развертывании приложений для конечного пользователя. И в песочнице перед развертыванием тоже нужно обеспечить точно такие же условия. А это значит установить множество программ, массу библиотек, сделать какие-то настройки

в операционной системе. И ещё не факт, что это будет полностью идентично исходному компьютеру.

Так вот, в рамках контейнера всё это находится внутри одного файла, и перекинуть контейнер с одной машины на другую — это скопировать парочку мегабайт или десяток мегабайт. Это вообще не проблема. И это очень сильно облегчает работу разработчика. Это позволяет решить все проблемы с совместимостью, с версиями и прочими вещами. В том числе, с установкой. Развёртывание приложений занимает считанные секунды. А если полноценную установку делать традиционным способом, это занимает несколько дней — полная установка, настройка, отладка и прочее. Есть разница?

Задание. Просмотрите в Википедии следующие статьи:
— Микросервисная архитектура
— Microservices
— Сервис-ориентированная архитектурas
— Service-oriented architecture

Задание. Просмотрите статьи по микросервисам:
— https://martinfowler.com/articles/microservices.html
— https://blogs.sap.com/2018/01/05/multi-target-applications-microservicesand-apis/

КОНТЕЙНЕРЫ WINDOWS

Корпорация Microsoft тоже существенно продвинулась в этом направлении. В рамках Windows у них есть свои собственные контейнеры. И про это тоже можно почитать. Но далее мы, в основном, сосредотачиваемся на докере.

Задание
— Windows and containers
— Windows и контейнеры

— https://learn.microsoft.com/en-us/virtualization/windowscontainers/about/

Напомним, что страницы сайта Microsoft имеются на десятках разных языков. Переключение языка — кнопка в левом нижнем углу веб-страницы, см. рис.



Рис. Переключение языка веб-страницы

GOOGLE И КОНТЕЙНЕРЫ

Компания Google — крупнейшие IT-гигант — утверждает, что у них все сервисы работают в контейнерах. Вам предстоит просмотреть пару статей на эту тему, чтобы уловить общую идею, общее направление развития информационных технологий.

Задание

— EVERYTHING at Google runs in a container

— https://www.theregister.com/2014/05/23/google_containerization_two_billion/

— Open Container Initiative

— https://opencontainers.org/

— Википедия: Lmctfy

МАТЕРИАЛЫ ПО КОНТЕЙНЕРАМ

Мы возвращаемся к докеру.

На сайте docker.com на главной страничке внизу мы находим ссылочку: хотите что-нибудь узнать про контейнеры? Это что-то

новое для вас? Для вас есть множество материалов, которые можно изучить, см. рис.



Рис. Ссылка на материалы по контейнерам

Если проблемы с английским — включаем автоматический переводчик странички. Автоматический перевод есть для Google Chrome и для Яндекс Браузера. Есть много других сервисов, если потребуется.

Итак что здесь можно почитать? В этом разделе есть несколько полезных сведений, есть весёлые картиночки, которые всё это объясняют. Но изучение займет минут 10—15. На самом деле нужно понять самые ключевые моменты:

Каким образом упаковывают программы в эти стандартизированные файлы?

Где это применяется?

Как это всё согласуется с виртуальными машинами — можно ли их сравнивать?

Можно ли комбинировать виртуальные машины и контейнеры?

Каким образом доводится эта технология до промышленных стандартов?

Вот это вам предстоит ПРОСМОТРЕТЬ. Материал очень просто даётся. Всё это ради одного — чтобы человек быстренько приступил к работе, причём с некоторым пониманием.

Задание

— Docker.com — Learn more

— https://www.docker.com/resources/what-container/

— Package Software into Standardized Units for Development, Shipment and Deployment

— Docker Containers Are Everywhere: Linux, Windows, Data Center, Cloud, Serverless, etc.

— Comparing Containers and Virtual Machines

— Containers and Virtual Machines Together

— Container Standards and Industry Leadership

ДОКУМЕНТАЦИЯ

В разделе «Документация» мы находим «Начало работы» — Get Started. Это «Быстрый Старт», «быстрое начало». Есть и более подробные разделы, описания, инструкции и прочее, см. рис.

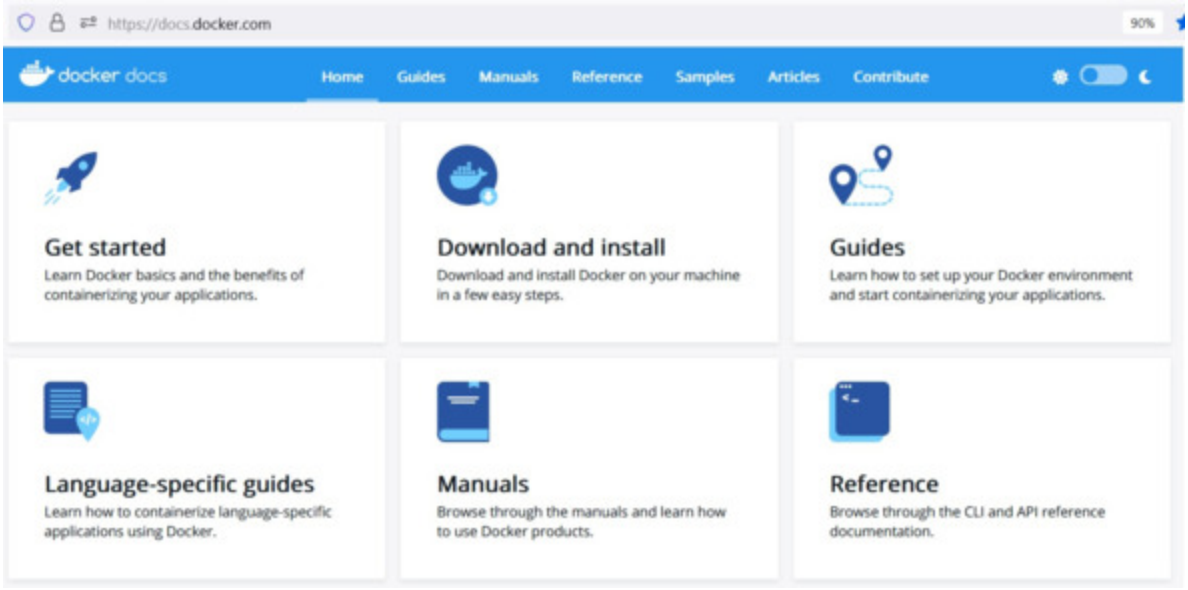


Рис. Документация по докеру

ВИРТУАЛЬНЫЕ ЛАБОРАТОРНЫЕ РАБОТЫ

Далее мы знакомимся с ещё одним любопытным инструментом. Это виртуальные лабораторные работы. Это значит, что мы получаем доступ через браузер. Мы можем поиграться с этим инструментом. Если всё сработает — хорошо. Если не сработает, то мы смотрим описание.

Потом мы используем те же команды в командной строке на своём компьютере. После того, как мы установили докер, мы можем повторять те же действия в своей командной строчке.

Для этого есть ресурс, который называется «Поиграем с Докером» — **Play with Docker**. И этот проект — как указано на первой же страничке — спонсирован самой корпорацией Докер — Sponsored by Docker. То есть это данные из источника, это данные от разработчиков. И это не пересказ, а самые-самые ключевые вещи, которые нам хотят сообщить. Это то, что фактически жизнь заставила их рассказывать каждый раз — каждому новому клиенту объясняют одно и то же.

— Interactive playground to learn Docker

— Sponsored by Docker Inc.

— https://labs.play-with-docker.com/

И вот из этого постепенно как раз и сложились виртуальные лабораторные работы. Еще их словом TUTORIALS.

— Beginning tutorials (self-paced learning exercise)

Tutorial — это английское слово, похожее на слово тьютор — «учитель, наставник». Слово tutorial означает «работа для самостоятельного ознакомления с каким-то программным продуктом». Это не просто демонстрация. Это действия — шаг за шагом — которые вам показывают и рассказывают. Вы их тут же делаете, и вы смотрите, как это всё происходит.

— Hands-on Learning (involving active participation)

При этом нам через браузер дается облачный доступ к Linux, где установлено всё, что нужно. Нам буквально требуется нажать кнопочку, где копируется команда из описания работы в командное окно Linux. Мы сразу же видим, как это всё там происходит. Весь этот подход у них называется **Hands-on Learning** или **Experiential learning** — «обучение на собственном опыте». Hands-on буквально означает «приложить к чему-то руки», а переводится это выражение как «активное участие». То есть мы не сидим и слушаем. Не листаем странички, а нажимаем кнопочки и участвуем в самом процессе. Мы обычно называем эту часть обучения «лабораторными работами».

Иногда мы можем встретить выражение «не бойтесь запачкать руки».

То есть мы полностью погружаемся в эту среду и в эти инструменты.

Вот какие слова здесь ещё могут встретиться:

Hands-on Learning (involving active participation)

•Beginning tutorials (self-paced learning exercise)

•Virtual Labs

•Free Linux Virtual Machine in browser

•Build and run Docker containers

Итак, виртуальные лабораторные работы — Virtual Labs, см. рис.

Рис. Темы виртуальных лабораторных работ

Здесь для самого первого знакомства предлагается три темы:

Как запускать контейнеры

Как создавать свои собственные контейнеры

Как управлять большим количеством контейнеров

Мы проходим регистрацию на сайте и начинаем с первой работы.

ПЕРВОЕ ЗНАКОМСТВО С КОНТЕЙНЕРАМИ

Посмотрим, как это всё выглядит.

Мы заходим на этот сайт, переходим по ссылочке:

— Virtual Lab 1. Your First Linux Containers

— https://training.play-with-docker.com/ops-s1-hello/

И мы получаем окошко, в котором две части, см. рис.

В левой части будет текст — описание команды. В правой части мы имеем командное окно Linux, в котором мы сами можем вводить любые команды и смотреть, как что работает.

Кроме того, в левой части окна есть возможность щелкнуть по команде, и она сразу скопируется в окно терминала.

- First Linux Containers
  - pulling images
  - running a container
  - container isolation
- Требуется регистрация на docker.com

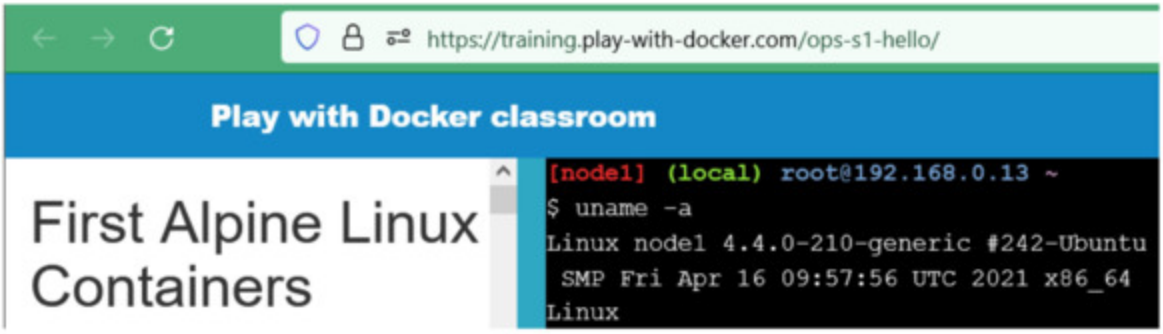


Рис. Первая виртуальная работа

Для выполнения этой работы нам потребуется зарегистрироваться на сайте docker.com. Это бесплатная регистрация. Если всё сработает, можно будет пользоваться этим облачным сервисом. Если же возникнут трудности, то можно смотреть на левую часть окна и читать описание. А потом команды выполнять в своем собственном командном окне. Особой разницы мы на самом деле не очень и заметим.

Итак, в рамках первого виртуального занятия у нас будут рассматриваться три основные технологии:

— pulling images — Как загрузить из интернета образ

— running a container — Как запустить контейнер на выполнение

— container isolation — Как выглядит изоляция контейнеризированных приложений

Все эти слова и понятия должны у нас постепенно сформироваться и отложиться в памяти.

В рамках первой виртуальной лабораторной работы используются несколько основных команд, и мы их будем с вами

постепенно разбирать.

ПРАВА ДОСТУПА

Для некоторых операций могут потребоваться права доступа администратора.

Задание

Запустить командное окно «под админом»:

Start — Command Prompt — More — Run as administrator

Перейти в рабочий каталог:

cd C:\Users\Valentin

Создать подкаталог для опытов:

mkdir docker

Перейти в подкаталог:

cd docker

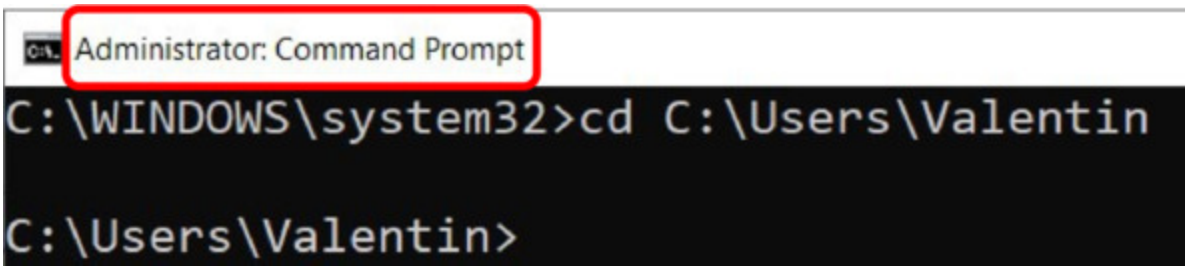


Рис. Командное окно с правами администратора

ОСНОВНЫЕ КОМАНДЫ

На рис. приведены основные команды докера, которые нам понадобятся при выполнении первой виртуальной лабораторной работы. К концу этого этапа вы должны быть в состоянии объяснить смысл этих команд и продемонстрировать их работу практически — с некоторым пониманием.

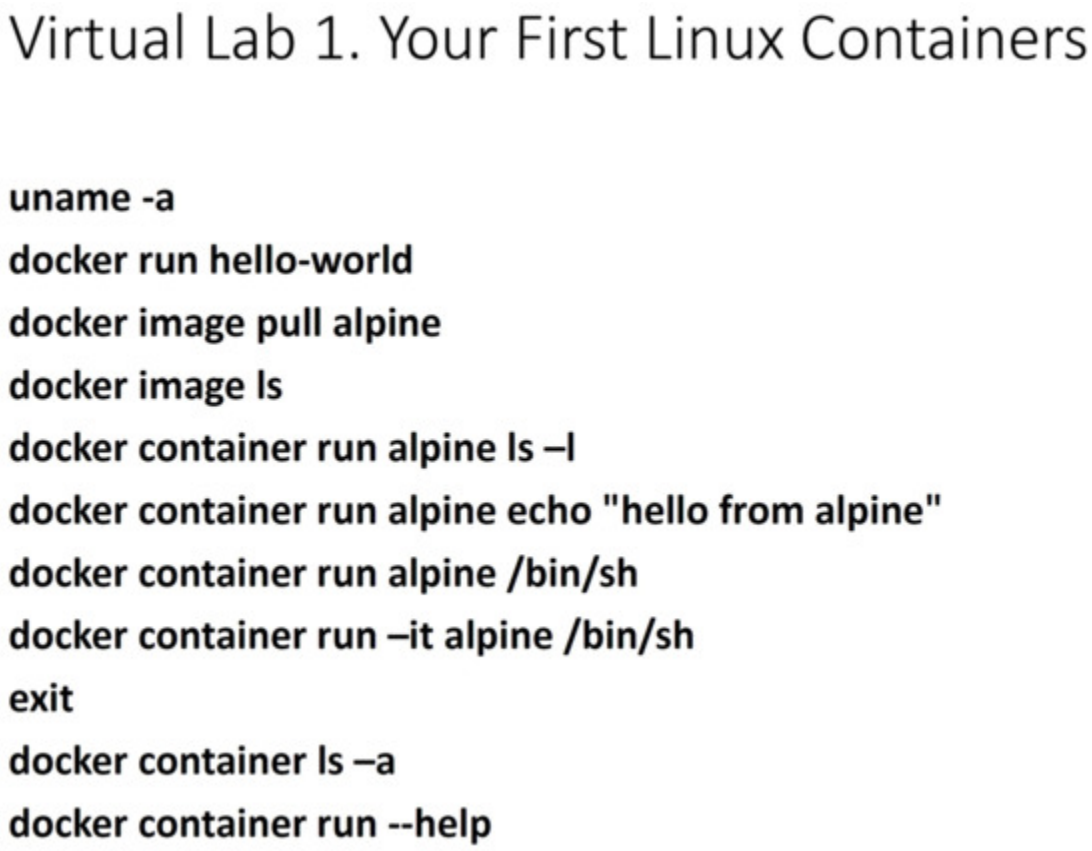


Рис. Основные команды первой виртуальной работы

ИНФОРМАЦИЯ О СИСТЕМЕ (WSL + UBUNTU)

Первая команда:

uname -a

Это команда Unix/Linux. Она сработает только в Линуксе.

Мы можем посмотреть описание команды:

uname — help

Здесь нам сообщают сведения о вычислительной системе, в том числе и об операционной системе, и о самом компьютере. Разные ключи позволяют узнать разные сведения. Ключ **–a** от слова **all** — «всё». Здесь будет выводиться на экран практически вся информация, см. рис.

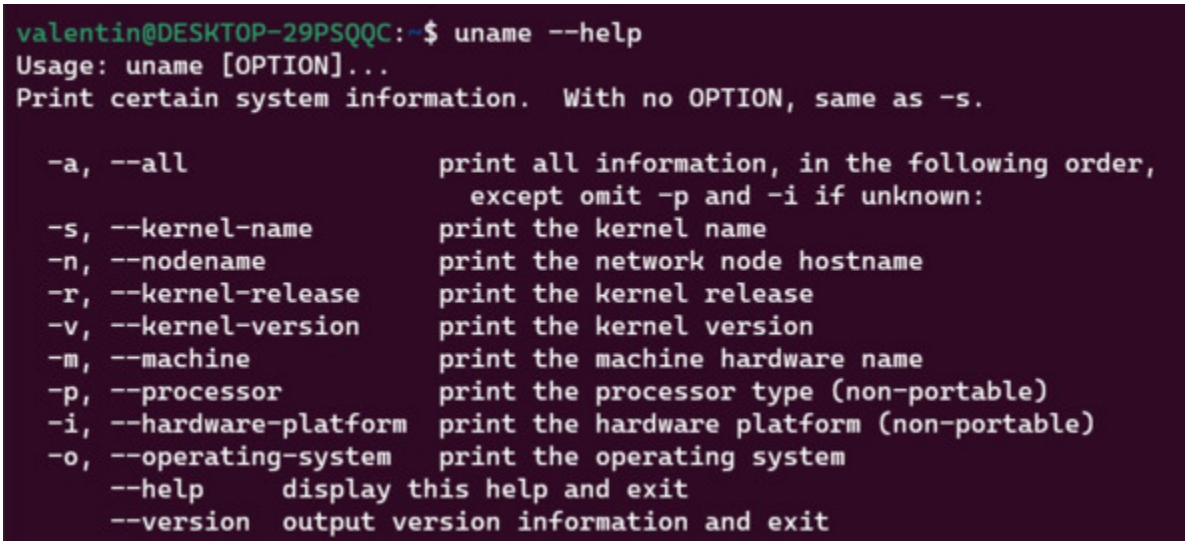


Рис. Описание команды **uname**

Здесь у нас пример того, как Ubuntu поверх WSL отвечает на наши запросы.

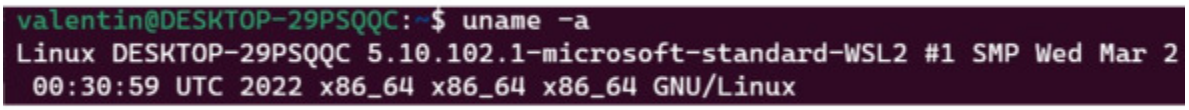


Рис. Описание системы WSL

Ключевые моменты этого сообщения:

— Microsoft WSL2

— Linux + WSL

— 64-разрядный процессор

Когда мы выполнили эту команду на сайте — в облаке, в чёрном окошечке — нам сообщили, что перед нами 64-разрядная версия Linux Ubuntu. Так мы можем узнать, какой нам предоставили компьютер — в том числе, и в облаке.

Кстати, **uname** означает **unix name**, то есть «название версии юникса». Эта команда перекочевала из юникса в линукс. Вот такая история вычислительной техники — в кратком изложении.

ПЕРВЫЙ КОНТЕЙНЕР

Наша следующая команда:

docker run hello-world

Напомним, что «Hello, world!» — это традиционное название для самой первой программы на любом инструменте. Неважно, будет ли это Си, Паскаль, Питон, база данных или что угодно ещё.

Для контейнеров тоже есть свой **Hello-world**. Это самый первый контейнер, который можно запустить. Он готовый. Всё, что от нас требуется, это только ввести команду **run** — «запустить, выполнить», см. рис.

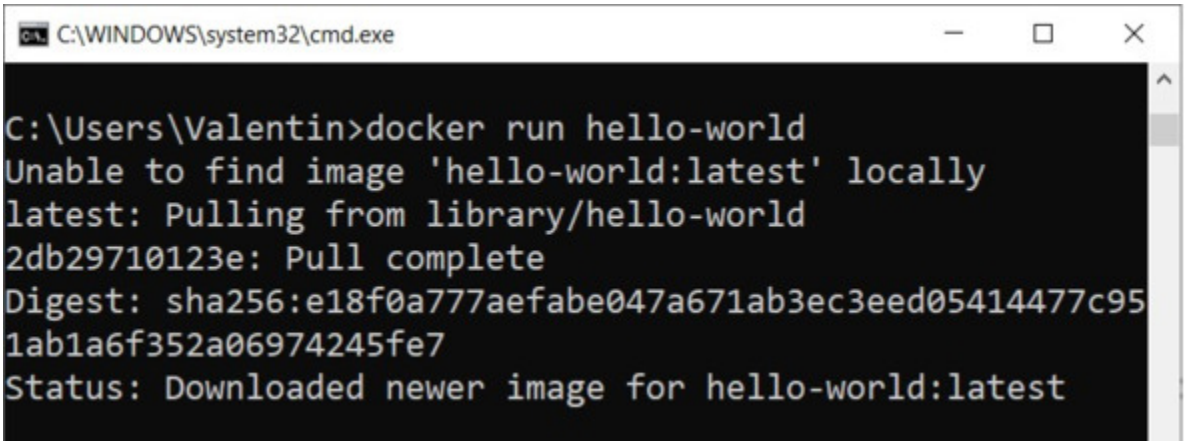


Рис. Запускаем контейнер

Итак, мы его запускаем и наблюдаем на экране много-много сообщений. Во-первых, нам говорят: **Unable to find image** — «не могу найти образ», причём **locally** — «на локальном компьютере». То есть при запуске контейнера докер вначале проверяет наличие образа на локальной машине.

Если образ не удалось найти, тогда этот файл скачивается автоматически откуда-то из интернета. Здесь говорится **Pulling** — «производится вытягивание, вытаскивание образа». Английское слово **pull** — «вытянуть, вытащить, скачать, загрузить из хранилища».

Дальше идут разнообразные цифры, которые мы разберём чуть позже.

Самое главное для нас сейчас — здесь говорится: **Pull complete** — «загрузка образа завершена успешно».

Следующее сообщение. **Status: Downloaded newer image** — «Состояние нашего задания: загружен новый образ».

После названия заказанного образа, через двоеточие говорится: **latest** — «последняя, самая свежая версия». Мы можем сами указывать версию контейнеров, если очень захочется. Но если версию не указываем, то всегда берётся последняя.

ЗАПУСК КОНТЕЙНЕРА

Что делает команда **run**? Она запускает контейнер на выполнение.

Мы можем узнать подробности, указав ключ help с двумя чёрточками (минусами):

docker run — —help

```
C:\Users\Valentin>docker run --help

Usage:  docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Run a command in a new container
```

Рис. Справка для команды **run**

Две чёрточки и слово **help** — это традиционный способ вызова справки в Линуксе. Помогает во многих ситуациях.

Почему здесь две чёрточки, а не одна?

Очень часто одна чёрточка в параметрах команды означает, что ключ состоит из одной буквы. Если после одиночной чёрточки идут несколько букв подряд, их нужно рассматривать как отдельные буквы, то есть как ключи из одной буквы. Обычно это первая буква английского слова.

Две чёрточки в параметрах говорят компьютеру, что ключ — это слово из нескольких букв, и что его нужно воспринимать целиком, как один ключ.

Итак, здесь говорится не просто «запустить контейнер». Здесь говорится: **Run a command in a new container** — «запустить на выполнение конкретную команду, причём сделать это в новом контейнере».

Все эти красивые названия и обозначения мы с вами будем постепенно разбирать.

Теперь ещё раз внимательно посмотрим на формат команды. Здесь ряд параметров указан в квадратных скобках — это необязательные параметры, без них можно обойтись. К примеру команда — **[COMMAND]** — в квадратных скобках. Мы не указали никаких команд, но всё сработало.

А вот образ — **IMAGE** — без скобок. Значит, это обязательный параметр, без него никак нельзя. Мы обязаны указать, какой образ использовать при запуске. И мы его указали: **hello-world**.

Так что минимальный формат для запуска выглядит так:

docker run IMAGE.

Теперь таким же способом вызовем справку для всего докера:

docker — help

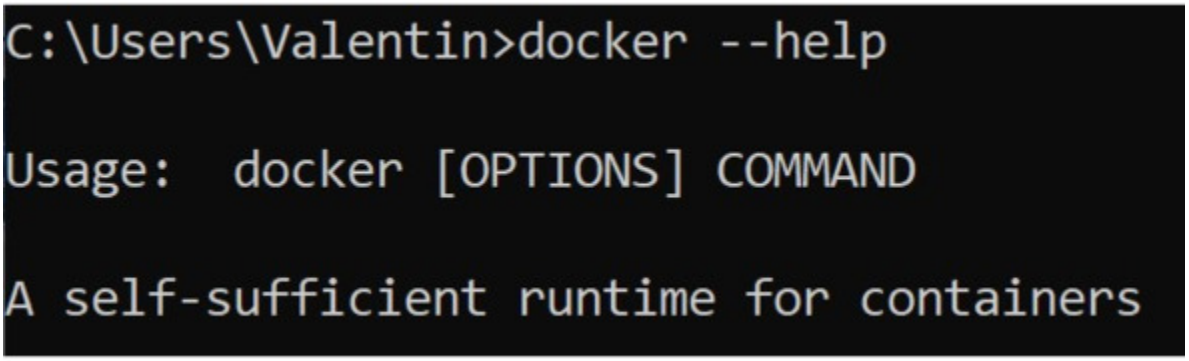


Рис. Справка по докеру

Теперь разберёмся с командой запуска. Что будет, если не указывать слово **container**?

Сравним два текста справки, см. рис.

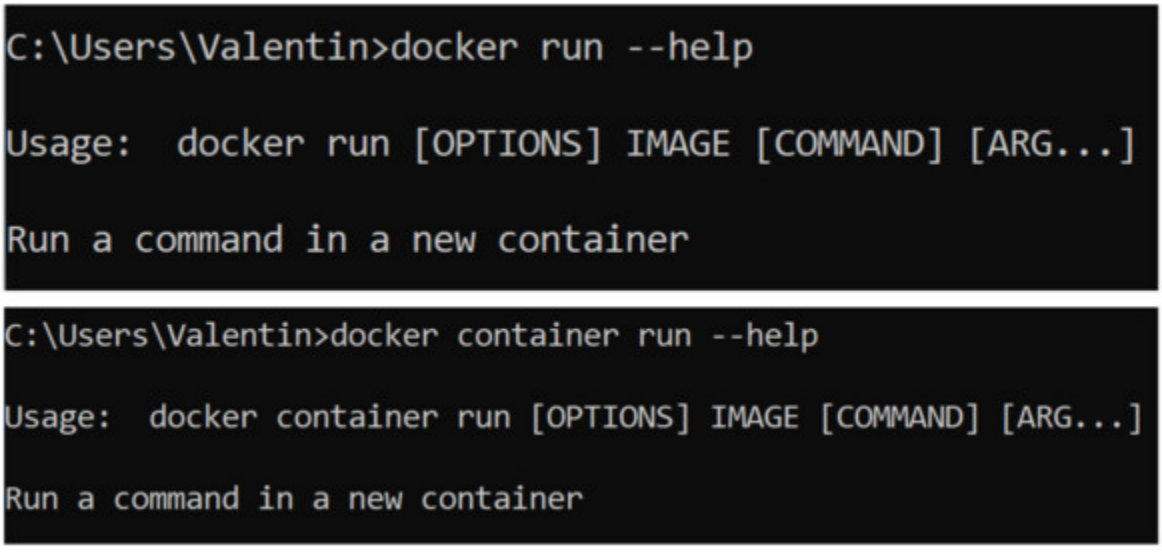


Рис. Аналогичные команды запуска контейнера

Объяснения вроде бы одинаковые. И для первого знакомства с технологией будем действовать как можно проще — будем использовать самые короткие команды. Потом, со временем, мы будем добавлять деталей, подробностей и сложностей.

ЭТАПЫ ЗАПУСКА КОНТЕЙНЕРА

Что же происходит при выполнении этой команды, при запуске контейнера? Сначала будут появляться служебные сообщения, а потом **Hello-world** выведет несколько строчек текста. Эти строчки просто объясняют, что происходит при запуске контейнера, см. рис.

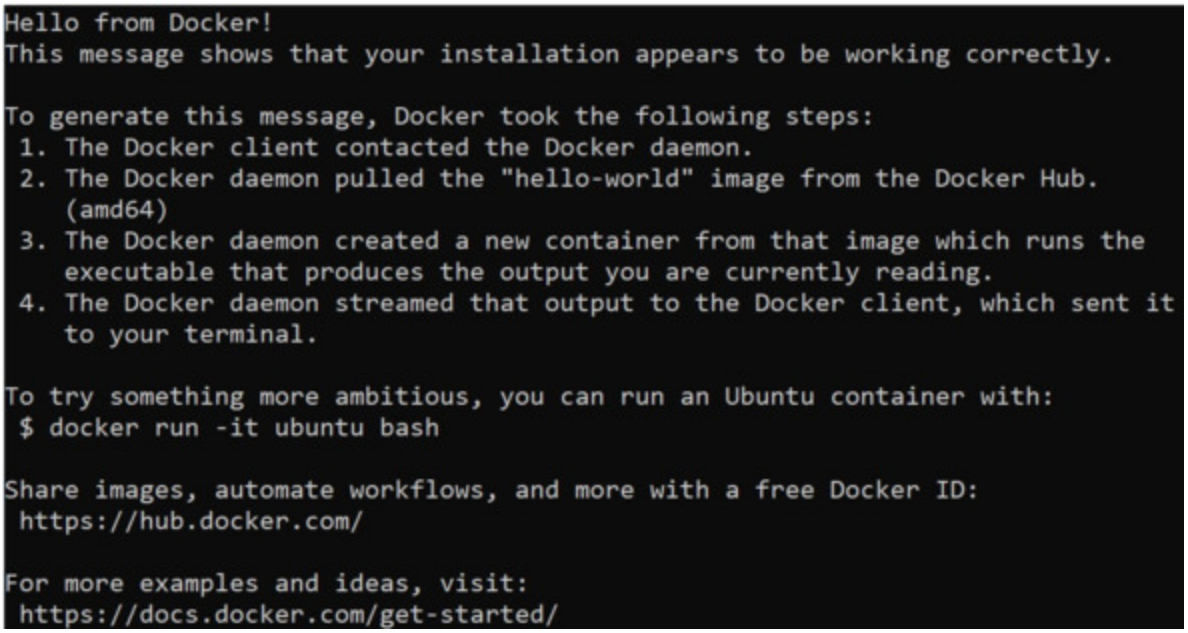


Рис. Запуск **hello-world**

Здесь говорится: чтобы на экране появилось сообщение, нам пришлось выполнить следующие действия.

Шаг первый. Так называемый «клиент» установил связь с так называемым «демоном». В Юниксе и Линуксе демоном называют системную службу, или «резидентную программу», или «фоновый процесс», или «программу-сервер», или «движок». Это программа, которая выполняется где-то на заднем плане, незаметно для пользователя, и обслуживает запросы от других программ. Итак, у нас есть «клиент», и у нас есть «демон» — принимаем к сведению.

Шаг второй. Демон докера вытягивает, вытаскивает, загружает образ из сервиса под названием докер-хаб. Здесь у нас участвуют

«образ» и «хаб» — тоже запоминаем.

Шаг третий. Демон докера создает новый контейнер из этого образа. Стало быть, мы скачиваем образ, затем из него создаем контейнер, и там внутри выполняется программа, которая пытается вывести текст. Здесь упоминается слово «контейнер» — тоже себе отметим.

Шаг четвёртый. Демон направляет выходные сообщения от программы (стандартный поток вывода) из контейнера клиенту, а тот уже отправляет эти сообщения на наш текущий экран — в терминал.

Затем нам предлагают повеселиться и ввести более сложную команду. Этим мы тоже займёмся.

КОНТЕЙНЕРИЗАЦИЯ

Теперь попробуем представить описанные выше шаги в виде схемы, см. рис.

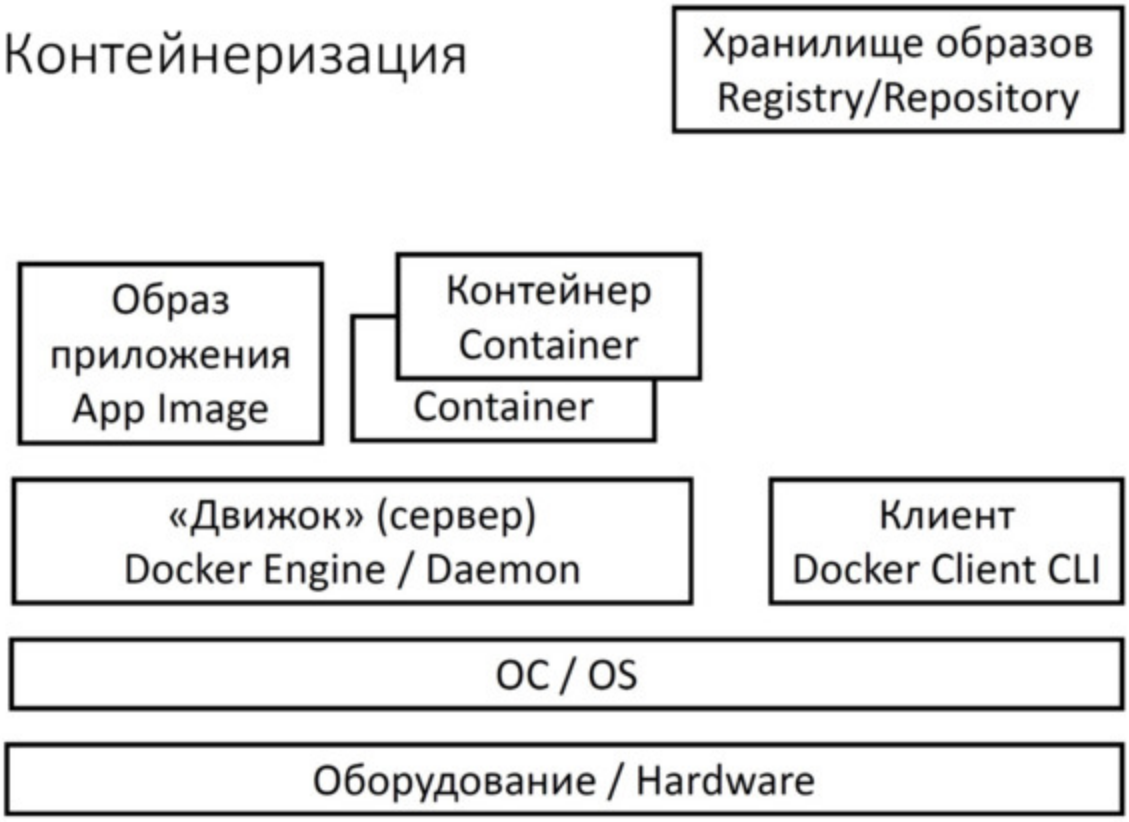


Рис. Организация запуска контейнеров

У нас есть «железо». На нём запущена основная операционная система. В ней работает демон, он же engine, он же «движок». Напомним, что это вспомогательный сервис, системная служба, которая обслуживает контейнеры.

Программисты позаимствовали слово «демон» из религии. В священных книгах упоминают демонов, которые живут у человека в голове и заставляют его делать не самые приятные вещи. Есть даже специалисты по изгнанию демонов. И ещё есть фильмы ужасов про такие ритуалы. А вот у программистов демон — это программа, которая «засела в голове» у компьютера. Только изгонять её не нужно, потому что она делает полезную работу. Вот такое у разработчиков чувство юмора.

Мы общаемся с этим движком-демоном через клиента в командной строке. CLI — Command Line Interface — «интерфейс командной строки», то есть командное окно, или терминал. Клиент — это дополнительная программа для общения с демоном. Пользователь работает напрямую только с клиентской программой, клиентским приложением.

Через клиента мы говорим демону: прошу запустить **hello-world**. Демон связывается с хранилищем образов. Образ — это файл, в котором есть всё, что нужно для работы конкретного приложения, прикладной программы.

Демон скачивает образ из облачного хранилища на локальный компьютер.

Затем он запускает этот образ на выполнение, но в момент выполнения этот объект уже называется словом «контейнер».

Из одного образа можно запустить несколько разных контейнеров. Точно так же можно взять исполняемый файл с решением *.EXE и запустить его несколько раз. Теперь у нас будет несколько разных окон, которые будут называться процессами. Один файл-программа и несколько процессов. Напомним, что процесс — это программа в момент выполнения.

Здесь видна аналогия: образ похож на исполняемый файл, а контейнер похож на процесс. Из одного файла можно запустить несколько процессов. Из одного образа можно запустить несколько контейнеров.

Мы так подробно разбираем картинки с одной целью — чтобы достичь понимания. И это для тех, кто хочет учиться. Когда пишем какую-то команду, у нас в голове должна появляться картина того, что происходит. Недостаточно просто прочитать текст. Это не поможет. А вот когда человек может мысленно представить себе картинку, то и слова можно любые подобрать, и объяснения выдать.

ОБРАЗ = ФАЙЛОВАЯ СИСТЕМА + КОНФИГУРАЦИЯ

Мы ещё раз пройдёмся по этой картинке. У нас тут есть имеется одна единственная операционная система. В отличие от виртуализации, здесь одна операционная система. Поверх неё ставится дополнительная программа, которая называется сервер, или движок, или даже демон. К этой программе мы обращаемся с помощью отдельного приложения, которое называется клиент. Чаще всего работаем с клиентом из командной строки, хотя есть и графический, оконный клиент. Это бывает удобно.

Так вот, клиент просит демона запустить определенную программу. Если эта вещь лежит где-то далеко, то оттуда её надо будет скачать. В интернете расположено отдельное хранилище, из него скачивается этот файл под названием образ. Потом его можно запустить, причём в нескольких экземплярах. Когда этот образ запущен на выполнение, он называется словом контейнер.

У нас имеется хранилище образов — основная, официальная платформа под названием Docker Hub.

Образ — image — это не просто файл. Это что-то вроде упакованного архива, в котором находится и сама прикладная программа, и всё, что ей нужно для выполнения. Это может быть небольшая файловая система — целое дерево с файлами и каталогами. Плюс внутри образа лежат все дополнительные файлы и настройки, которые нужны для работы.

Вот с этими словами мы дальше и будем работать, см. рис.

Основные термины

- Приложение – прикладная программа
  - Application
- Образ – файловая система + конфигурация
  - Image: Application (file system + configuration)
- Контейнер – Экземпляр образа для запуска приложения
  - Container: Running instance of Docker image
- Сервер, служба, демон, «движок»
  - Docker daemon/engine: background service
- Клиент для общения с сервером
  - Docker client: CLI tool
- Хранилище образов
  - Docker Hub: registry/directory of Docker images

Рис. Терминология контейнеров

Далее на рисунке мы ещё раз показали аналогии:
– образ и контейнер
– файл и процесс
– класс и объект.



Рис. Аналогии для контейнеров

Так что отношения образа и контейнера можно сравнить с отношением класса и экземпляра класса. Если вы знакомы с идеями программирования — у нас программирование бывает структурное и объектно-ориентированное. Здесь используется идея из области объектно-ориентированного программирования.

Образ — это что-то вроде класса объектов, а контейнер — что-то вроде экземпляра объекта.

Можно сравнить их также с файлом и процессом. Файл может присутствовать на диске компьютера в одном экземпляре, а запустить его на выполнение мы можем несколько раз. И тогда у нас одновременно будут работать несколько окошек, а разным окошкам будут соответствовать разные процессы. Аналогия: один файл и несколько процессов.

Конечно же, это очень упрощенная и очень отдаленная аналогия. Но мы хотя бы используем что-то более или менее знакомое.

Внутри образа у нас имеется программа. А кроме самой программы, внутри образа у нас упакованы файловая система и дополнительные объекты. Это могут быть библиотеки программ. Причём для каждого приложения нужен свой набор библиотек. Это могут быть какие-то настройки… Всё, что угодно — что требуется для работы программы.

За счёт этого механизма контейнеризации достигается полная изоляция приложений друг от друга.

Docker Hub — Hello world!

Переходим к хранилищу образов.

Заходим на сайт:

https://hub.docker.com/

Для скачивания образа мы можем использовать этот сервис без регистрации. Здесь ввести название образа в окне поиска. Так мы сможем найти ту самую программу, тот самый образ, который нас волнует.

Например, в строке поиска мы пишем **hello-world** и получаем большой список. В верхней части списка будет наиболее часто скачиваемый образ, см. рис.

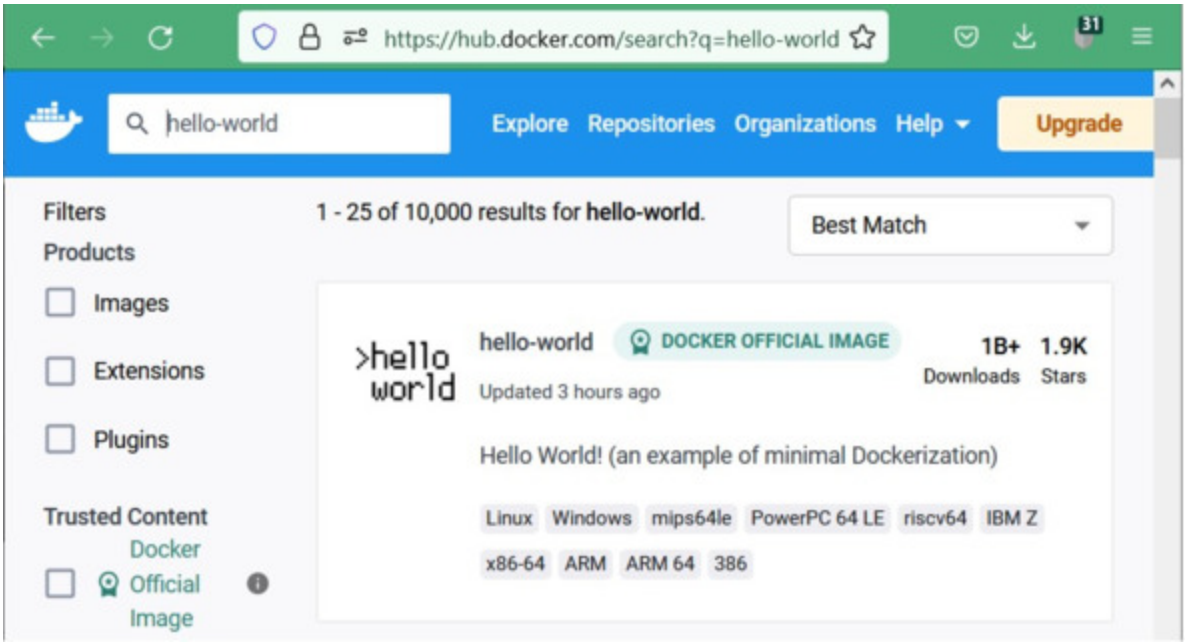


Рис. Контейнер **hello-world** в хранилище **Docker Hub**

Здесь мы видим количество скачиваний: **1B+ Downloads**. Буква **B** — от английского слова Billion. По-русски это миллиард. Плюсик означает «больше, чем». Стало быть, **1B+** означает больше миллиарда скачиваний для этого образа с этого ресурса.

Дальше указано **1.9k Stars**. Буква k — от слова «кило-», то есть 1000. Другими словами, 1900 раз люди поставили свою звёздочку, выразили свою положительное оценку.

Hello Word — это пример очень маленького приложения, которое запускается в рамках контейнера.

Чуть ниже мы видим список платформ, на которых этот образ можно запустить. И на самом деле это список операционных систем. Мы с вами в основном знакомимся с Windows и Linux. Но если присмотреться, в этом списке здесь есть разные аппаратные платформы, например, Power PC. Здесь также есть мейнфрейм IBM Z. Это серия больших машин, которые размером больше, чем хороший холодильник. Есть в этом списке и мобильные платформы.

Дальше здесь говорится x86—64, что означает 32-разрядные и 64-разрядные системы.

Когда мы задаём поиск hello-world, то видим 10 тысяч таких образов, самых разных. Но из них мы выбираем образ с пометкой: DOCKER OFFICIAL IMAGE. Это официальный образ, одобренный компанией Docker. К нему, соответственно, будет немножко больше доверия.

Когда мы что-то скачиваем из интернета, это всегда риск. Если эта программа прошла некоторые проверки, если есть положительные отзывы от пользователей, то будет немножко больше доверия. Но в любом случае, отвечать за результаты (и расплачиваться за неприятности) будет компания, которая использует этот продукт. Так что наше дело — предупредить.

DOCKER HUB — ХРАНИЛИЩЕ КОНТЕЙНЕРОВ

Итак, Docker Hub — «Докер-Хаб». Его называют по-разному. На разговорном уровне можно сказать, что это место, где хранятся образы, то есть хранилище образов. Могут использоваться и другие слова. Warehouse — тоже означает «хранилище». Store — тоже означает хранилище, а также одновременно магазин. Registry — реестр. Иногда говорят «база данных». Иногда говорят «Центральный репозиторий». Все эти слова означают примерно одно и то же, см. рис.

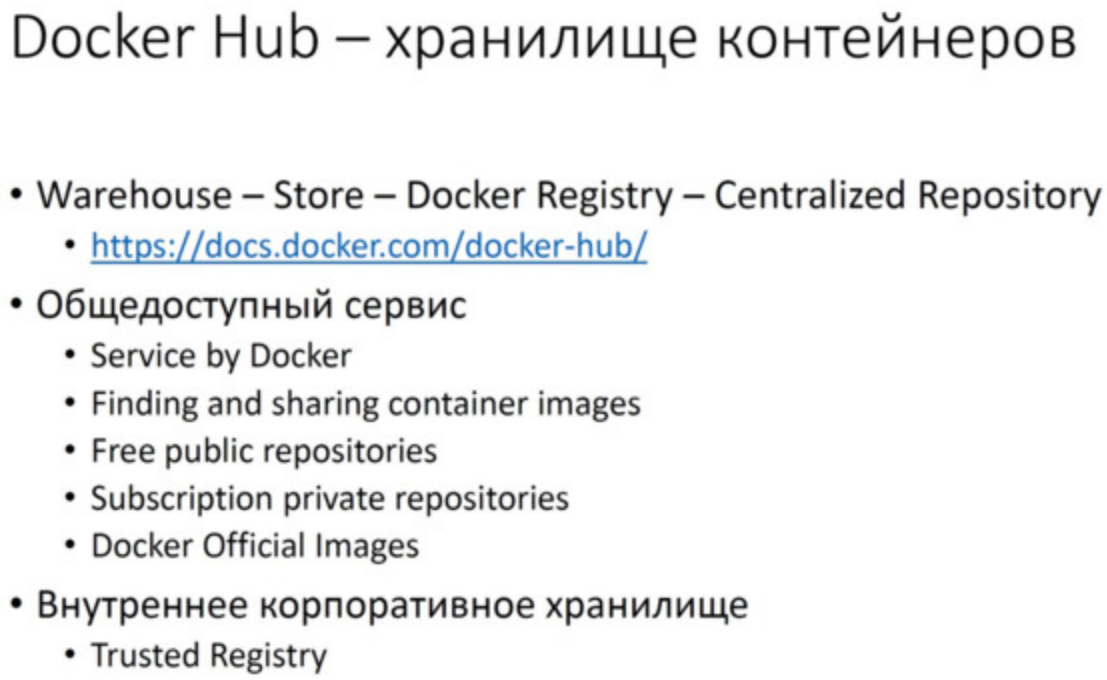


Рис. Возможности хранилища образов

Здесь у нас, естественно, нет слова «контейнер». На этом сервисе хранятся образы. А контейнер появится, когда мы скачаем и запустим образ.

Если почитать описание, то выясняется, что этот сервис поддерживается самой компанией Docker, которая и разрабатывает эту платформу контейнеризации.

Здесь можно задавать поиск и скачивать образы.

Можно также поделиться своими образами. Для этого придётся зарегистрироваться. Регистрация бесплатная для большинства сервисов. Мы можем собрать свой образ и выложить его для общего пользования. Это будет называться Free Public Repository. Public — значит «в открытом доступе, для общего пользования».

Также существуют платные подписки — subscription. И можно организовать Private Repository — частный репозиторий. Это значит, что человек оплачивает этот сервис, а затем использует его для

ограниченного количества людей. Раздаёт им ссылку или пароль. И это уже для корпоративного применения.

Ну и самое главное, что здесь можно найти официально одобренные образы.

Здесь также можно найти и совершенно «левые», непроверенные, разработки. В том числе и зловредные. И то и другое здесь присутствует.

На этой же технологии можно построить внутреннее корпоративное хранилище образов, которое вообще разворачивается на сервере самого предприятия. По той же самой технологии «докер-хаб».

Итак, Docker Hub — это место, откуда скачивают образы. Может также называться «хранилище». Если посмотреть в толковых словарях английского языка слово **ё**, то окажется что это «хранилище», то есть «место, где хранится большое количество товаров», см. рис.

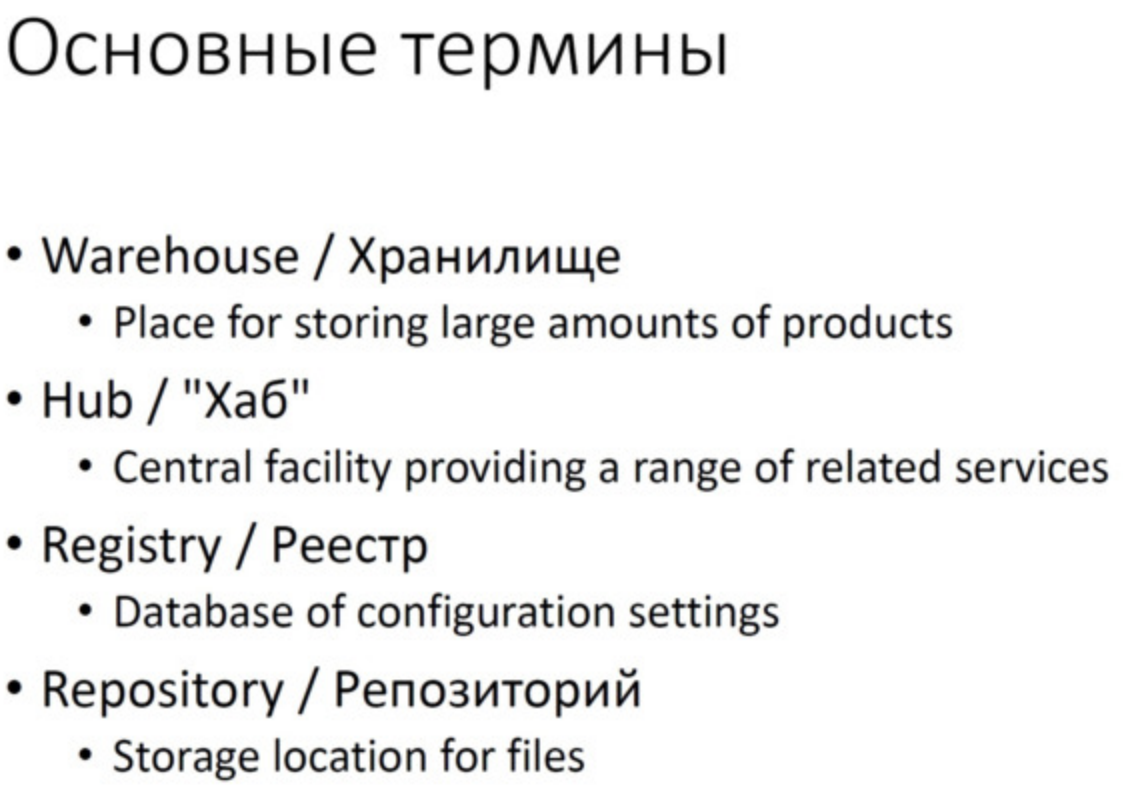


Рис. Основные названия хранилища

Само название hub — «хаб» — это особое словечко. У него много самых разных значений. И это не всегда переводится одним русским словом. В нашем случае, хаб можно перевести как «устройство, установка, комплекс, который предоставляет целый набор разнообразных сервисов, набор услуг». И мы получаем эти услуги на этом сайте.

Далее, реестр — это, по сути, база данных.

Репозиторий — это место, где что-то хранится.

Все эти разные названия относятся к одному и тому же сервису **Docker Hub**. Это хранилище образов, это база данных и это сервисы по скачиванию, по размещению, по обслуживанию этих самых образов для контейнеров.

ПОДПИСКА

Посмотрим стоимость подписки на сайте:
https://hub.docker.com/

Стоимость услуг разная. Здесь всё зависит от категории пользователя и от его запросов и потребностей, см. рис.

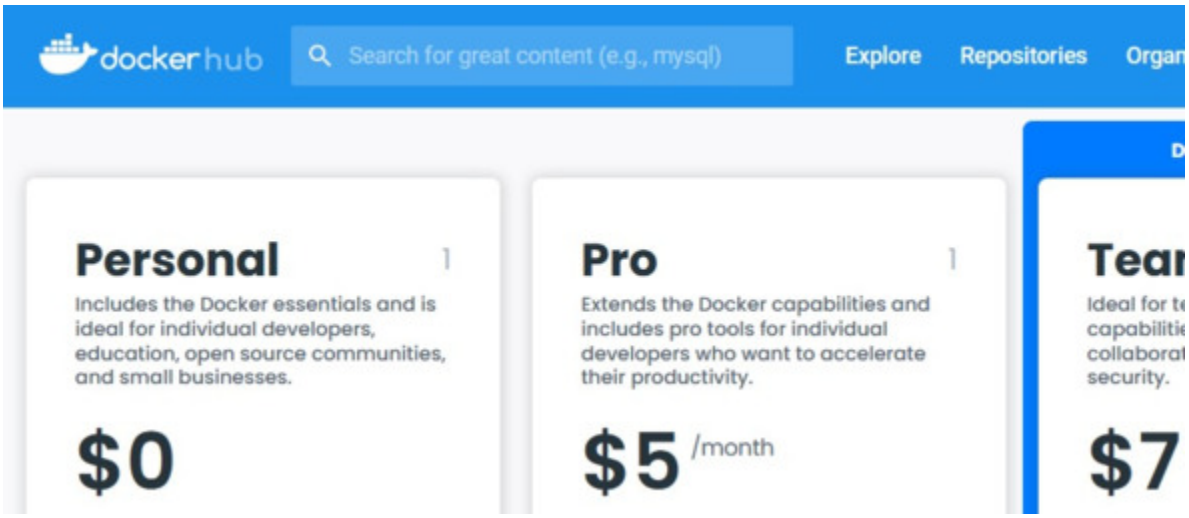


Рис. Виды подписки

Для личного пользования — Personal — основные сервисы Докера предоставляются бесплатно. В этом же разделе мы видим: Individual Developers — отдельные разработчики программ. Здесь же указано: Education — для образовательных целей, для образовательных учреждений. А также: Open Source Communities — сообщества разработчиков программного обеспечения с открытым исходным кодом — Open Source. Даже есть Small Businesses — для малых предприятий. Всем этим категориям базовый набор услуг предоставляется бесплатно.

Продвинутые сервисы — для профессиональных разработчиков — платно.

DOCKER DASHBOARD

Теперь познакомимся с оконным интерфейсом нашей платформы контейнеризации — **Docker Desktop** –«настольное» приложение.

Это так называемый «дэшборд», или «дашборд» — кому как нравится. В любом случае, это английское слово **Dashboard**, написанное русскими буквами.

Фактически, перед нами одновременно информационная панель и инструменты для управления работой, см. рис.

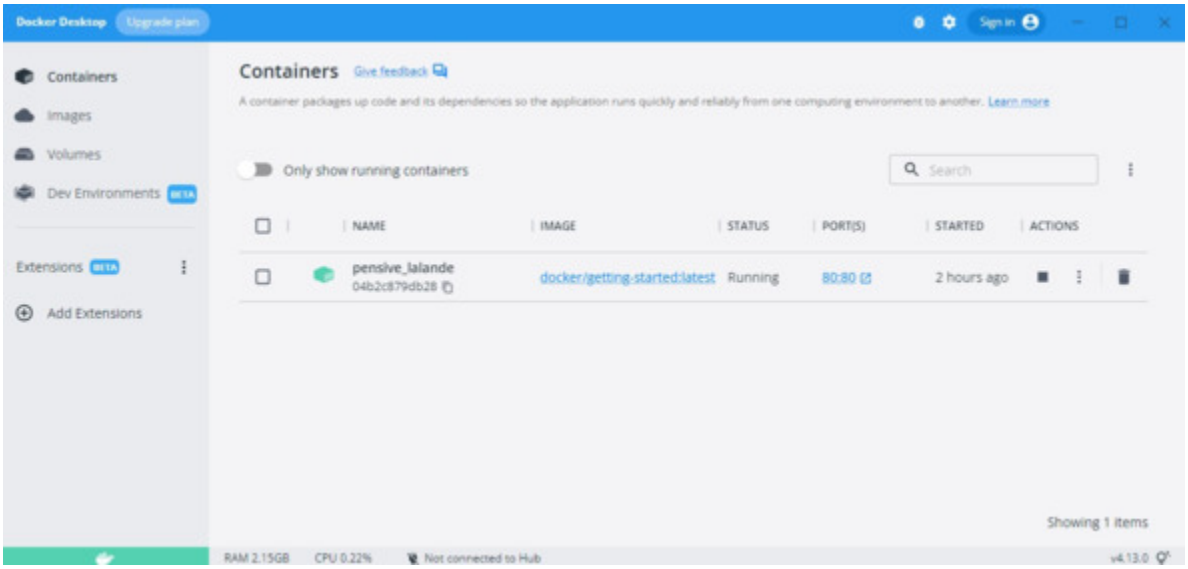


Рис. Начальный экран **Docker Desktop**

После того, как мы выполнили хотя бы одну команду — **run** — у нас появляется список образов и контейнеров, расположенных на нашем локальном компьютере.

Вспомним, что у нас происходило.

Вот первая команда, которую мы выполнили:

docker run hello-world

Буквально **run** означает «запустить на выполнение», или просто «выполнить».

Докер проверяет, есть ли у нас этот файл под названием образ. у нас уже есть. Если нет — он скачивает образ с докер-хаба.

Загруженный образ оказывается на нашем локальном компьютере.

И теперь можно посмотреть, что у нас тут есть, — с помощью графического приложения, графической оболочки Desktop Desktop.

Здесь у нас имеется несколько разделов:

— **Containers** — Контейнеры

— **Images** — Образы

— **Volumes** — Тома

В разделе контейнеры мы получаем список тех контейнеров, которые мы уже запускали на выполнение.

В колонке **IMAGE** указано название образа.

В колонке **NAME** приводится название контейнера — пара английских слов без особого смысла — просто чтобы отличать один контейнер от другого — по имени. Плюс к этому здесь имеется длинная последовательность цифр и букв — это индивидуальный номер конкретного контейнера. Для каждого контейнера указано его текущее состояние — **STATUS**. Наш контейнер работает, судя по сообщению **Running**.

В последней колонке **ACTIONS** можно выбрать действие над этим контейнером: остановить или запустить ещё раз. Такой запуск отличается от команды **run**, потому что не создаётся новый контейнер.

Само название информационно-управляющей панели — дэшборд — нужно изучить. В связи с этим есть небольшое задание.

Задание

Найдите разные значения термина Dashboard / Дашборд / Дэшборд и сформулируйте их своими словами. Затем расставьте эти технологии по времени разработки, чтобы прочувствовать историю вопроса.

https://en.wikipedia.org/wiki/Dashboard
https://en.wikipedia.org/wiki/Dashboard_(business)
https://www.merriam-webster.com/dictionary/dashboard
https://en.wiktionary.org/wiki/dashboard
https://www.macmillandictionary.com/dictionary/british/dashboard

https://www.oxfordlearnersdictionaries.com/definition/english/dashboard
https://www.dictionary.com/browse/dashboard

Нажав на значок мусорной корзинки, мы можем этот контейнер удалить. Это достаточно очевидно.

В списке контейнеров есть загадочная кнопка, которая обещает нам выдать «более подробные сведения» — **View details**.

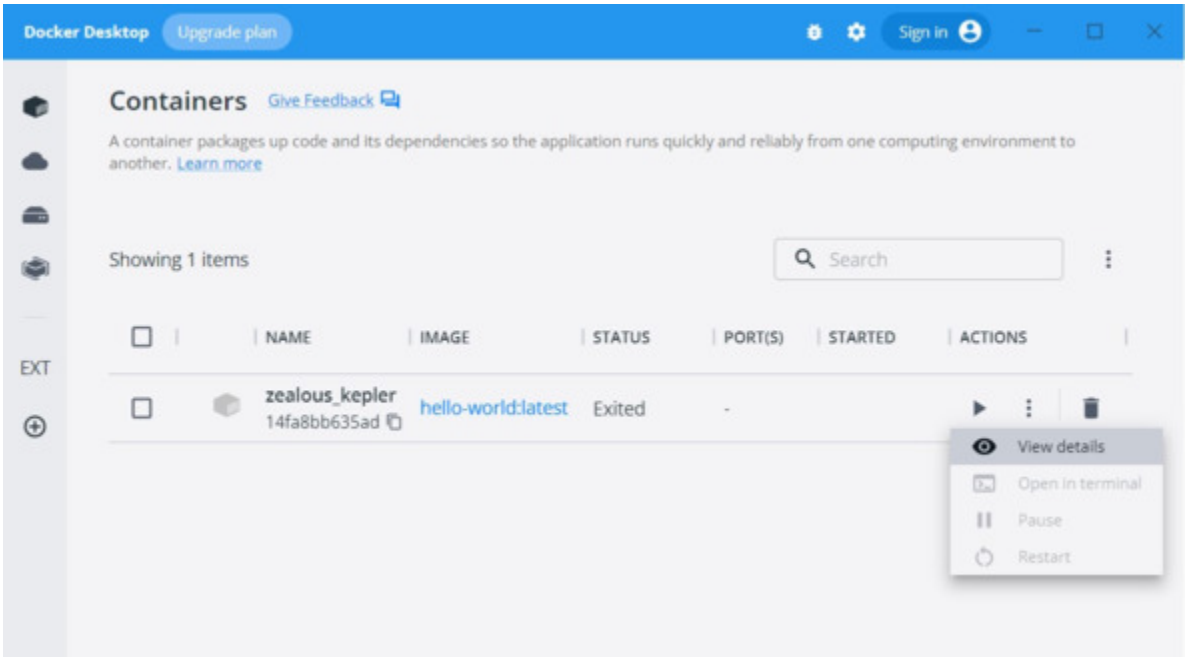


Рис. Действия над контейнером

На само деле под видом «подробностей» нам выдают результаты запуска контейнера. Фактически здесь показано то, что этот контейнер выводил на экран при запуске, — те же самые сообщения. При этом мы уже могли закрыть терминал — командное окно. А здесь мы можем увидеть историю того, что этот «контейнеризированное» приложение выводило на экран. Плюс к этому у нас имеется несколько дополнительных инструментов. Мы можем подключиться к контейнеру штуке через командную

строчку — **CLI** — **Command Line Interface**. То есть мы можем повторно подключиться к этому контейнеру и получить доступ к нему в виде командного окна.

Вот такие возможности даёт нам графический интерфейс **Docker Desktop**.

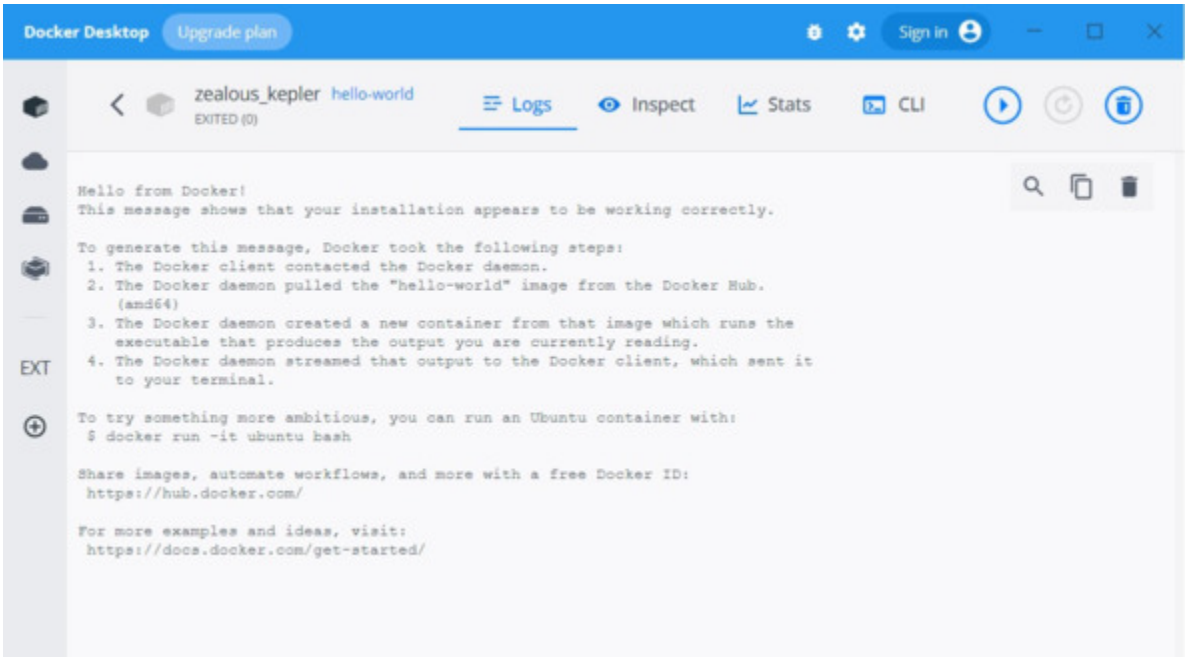


Рис. Результаты запуска контейнера

ЗАГРУЗКА ОБРАЗА

Наша следующая команда:

docker image pull alpine

Слово **pull** буквально означает «тянуть, вытащить». Это действие по «вытаскиванию» образа из докер-хаба, из облачного хранилища. Мы скачиваем этот файл на локальный компьютер, но пока не запускаем его на выполнение.

При выполнении команды **pull** нам сообщают состояние, адрес и другие многочисленные подробности.

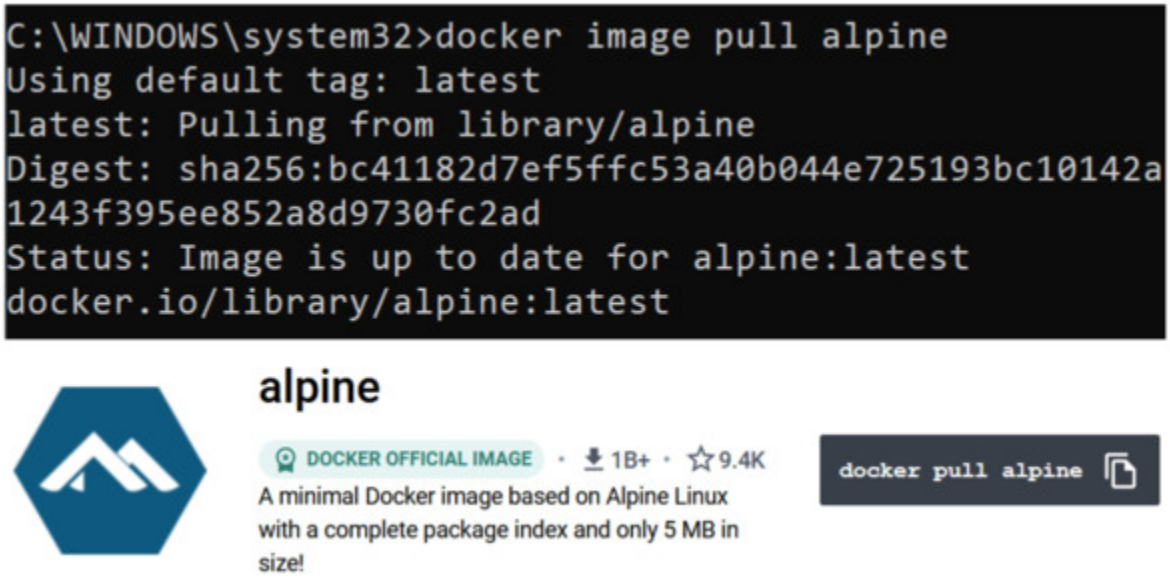


Рис. Загрузка образа **Alpine**

Мы можем поподробнее выяснить, что же такое **Alpine**. Мы заходим на докер-хаб, задаем поиск по этому слову и выясняем, что это официальный образ Докера.

Дальше: **1B+** означает «больше миллиарда скачиваний». Звёздочка и число **9,4k** означает 9400 положительных отзывов от пользователей.

Далее здесь говорится, что этот образ очень маленького размера. Здесь указан размер 5 мегабайт с восклицательным знаком.

Что такое 5 Мегабайт? Если вы уже выполняли четвертую-пятую-шестую лабораторные работы по ОС, то уже что-то про это знаете. Если мы хотим запустить Linux на виртуальной машине, то нужно скачать файл с расширением ***.ISO**, и это несколько гигабайт!

А здесь несколько мегабайт, то есть в 1000 раз меньше. Естественно, здесь урезаны дополнительные возможности и оставлено только то, что потребуется разработчикам.

Кроме того, в докер-хабе имеется подсказка: команда для запуска контейнера. Обратим внимание, что команда немножко

короче. Но, похоже, что она делает то же самое. Здесь просто написано:

docker pull alpine

Кнопочка рядом с командой позволяет сразу же скопировать в буфер эту строчку. Нам даже не нужно выделять её мышкой и нажимать **[Ctrl+C]**. Просто жмём кнопочку, а потом вставляем из буфера в терминале. Очень удобно.

ALPINE LINUX

Разберёмся, кто такой **Alpine**.

Естественно, у этой версии Linux есть свой сайт в интернете https://www.alpinelinux.org/

Здесь мы можем узнать некоторые подробности, см. рис.



Рис. Описание **Alpine Linux**

Здесь говорится, что это независимая разработка, и к тому некоммерческая, да ещё и универсальная. Затем нам перечисляют преимущества:

Простота

Эффективность

Надежность

Само слово **Alpine** означает «Альпийский» и намекает на Альпийские горы. Эмблема программного продукта — это две

горы, а сверху что-то вроде ледника. С другой стороны, можно попытаться рассмотреть стилизованную букву **A** в этом логотипе.

СПИСОК ОБРАЗОВ

Итак, с помощью команды **pull** мы вытащили этот образ из хранилища и разместили где-то на локальном компьютере.

Теперь мы можем посмотреть список образов, которые у нас уже есть.

docker image ls

Напомним, что в Линуксе команда **LS** означает «получить список», «вывести список на экран». Сокращение **LS** — от слова **LIST** — «список».

Итак, мы вводим в терминале команду **LS** и получаем список, см. рис.

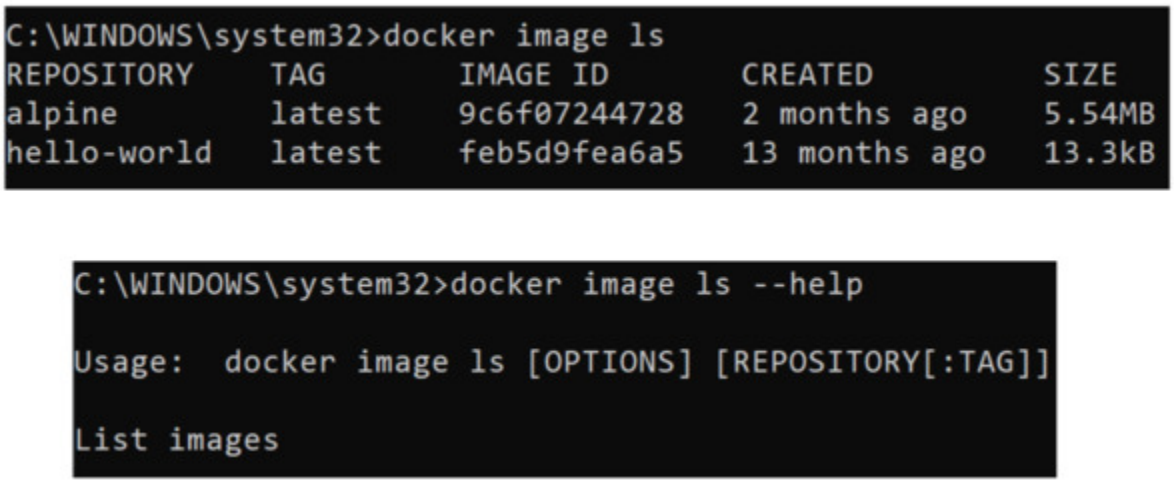


Рис. Список образов в терминале

В нашем примере показано два образа. Один называется **Alpine**, а второй **Hello world**.

Для каждого образа указан его **IMAGE ID** — идентификатор. Фактически, это просто число, индивидуальный номер — как номер

паспорта или пенсионный номер у человека. Поскольку здесь есть и цифры, и буквы, можно заключить, что это число записано в шестнадцатеричном формате.

Дальше, в колоне **SIZE** можно увидеть, какого они размера — мегабайты или килобайты.

В колонке **CREATED** говорится, когда они были созданы — столько-то месяцев или лет тому назад.

Для большего понимания мы вызываем команду **help**.

Как мы уже говорили, два минуса, две чёрточки — это такая особенность, которая характерна именно для Линукса. Это означает, что нужно взять это слово help целиком, как параметр нашей команды.

Получаем объяснение, не слишком подробное: **List images** — «вывести на экран список образов».

Но у этой команды есть много дополнительных параметров. Эти дополнительные параметры не обязательно использовать, и они указаны в квадратных скобках.

Это общепринятое обозначение для объяснения работы программ — когда мы вызываем справку по работе программы. Мы можем столкнуться с квадратными скобками, но при использовании команд эти скобки вводить не нужно! Эти скобки нужны для читателя, а компьютеру эти скобки не потребуются.

Ещё одна необязательная часть — мы можем указать, из какого репозитория мы хотим «вытащить» образ. А ещё более необязательный параметр указан внутри этого необязательного — это так называемый тег — **TAG**. Буквально это слово означает «метка», но в нашем случае это версия образа. Для одного и того же образа может быть несколько версий, и для каждой версии будет своя метка. В нашем примере мы не указали тег, поэтому скачана последняя версия — в колонке **TAG** говорится **latest** — «последняя версия».

Мы можем выбрать что-то более конкретно, если нас будет сильно волновать определённая версия образа.

Итак, мы получили список образов и теперь мы можем отдельно запустить этот образ. А когда он начинает выполняться, то новый объект будет называться словом «контейнер».

Можно использовать два варианты команды:

docker container run

docker run

И в первом приближении это даёт один и тот же результат — запуск контейнера на выполнение.

Мы можем посмотреть список образов и в графической среде, см. рис. В предыдущем примере мы выбирали слово **Containers** — Контейнеры — и получили список контейнеров. Теперь нам нужно выбрать раздел **Images** — Образы, чтобы получить список образов.

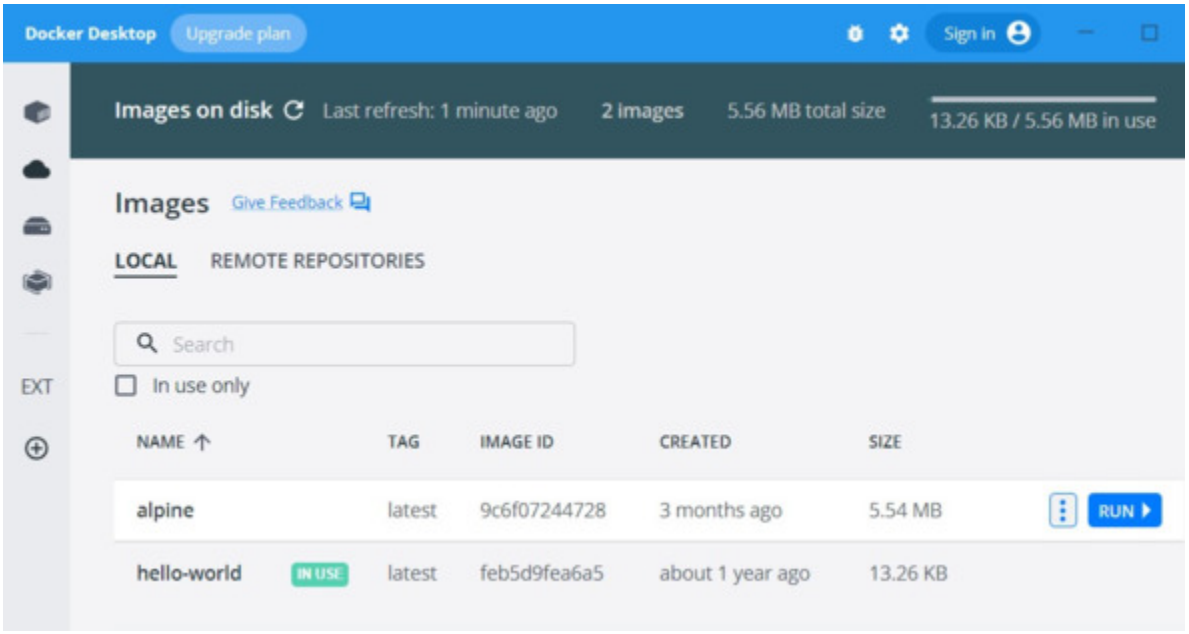


Рис. Список образов в **Docker Desktop**

В графическом интерфейсе в левой части окна могут быть названия разделов или их схематичные значки. Для контейнера — первый раздел — это некая «коробочка». Для образов это «облачко», то есть «облачное хранилище». Но при этом в списке образов мы

рассматриваем список локальных образов — **LOCAL**. На отдельной вкладе можно выбрать удалённые, облачные репозитории — **REMOTE REPOSITORIES**.

Для образа **hello-world** выводится дополнительная пометка: **IN USE** — «данный образ используется». Это значит, что данный образ уже запускали и на его основе был создан контейнер. Пока этот контейнер лежит на компьютере, соответствующий образ удалить не получится — образ как бы «занят», заблокирован.

Зато другой образ, который мы только скачали командой **pull**, не имеет такой пометки. Значит, мы можем этот образ удалить.

Остальные колонки такие же, как и терминале:

— **IMAGE ID** — Идентификатор

— **CREATED** — Дата создания

— **SIZE** — Размеры

В последней колоночке у нас есть дополнительные действия — запустить или удалить.

У нас есть образ, и мы можем его отсюда запустить на выполнение. Тогда у нас появится контейнер, с которым можно будет общаться. Вот такая у нас графическая среда.

ЗАПУСКАЕМ КОНТЕЙНЕР

Итак, мы скачали образ. Теперь мы его запускаем:

docker run alpine

docker container run alpine

Попробуем оба варианта, см. рис.

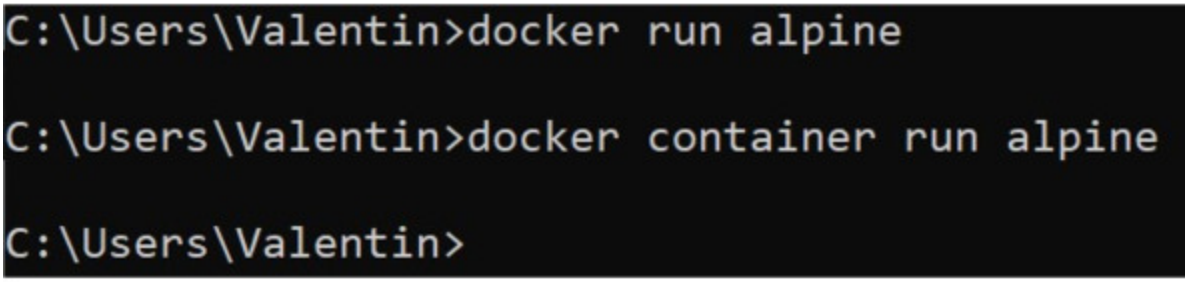


Рис. Запуск контейнера

После запуска наблюдаем пустой экран.

Ничего не происходит. Смысл этого опыта в том, что мы запустили нашу маленькую операционную систему **Alpine Linux**. Система запустилась, ничего не сделала и закончила работу. Ведь мы не сказали, что она должна сделать.

Чтобы Linux выполнил какое-нибудь действие, какую-нибудь команду — нужно эту команду ему сообщить.

Ещё раз проверяем список контейнеров в приложении Desktop и в терминале командой **LS**, см. рис.

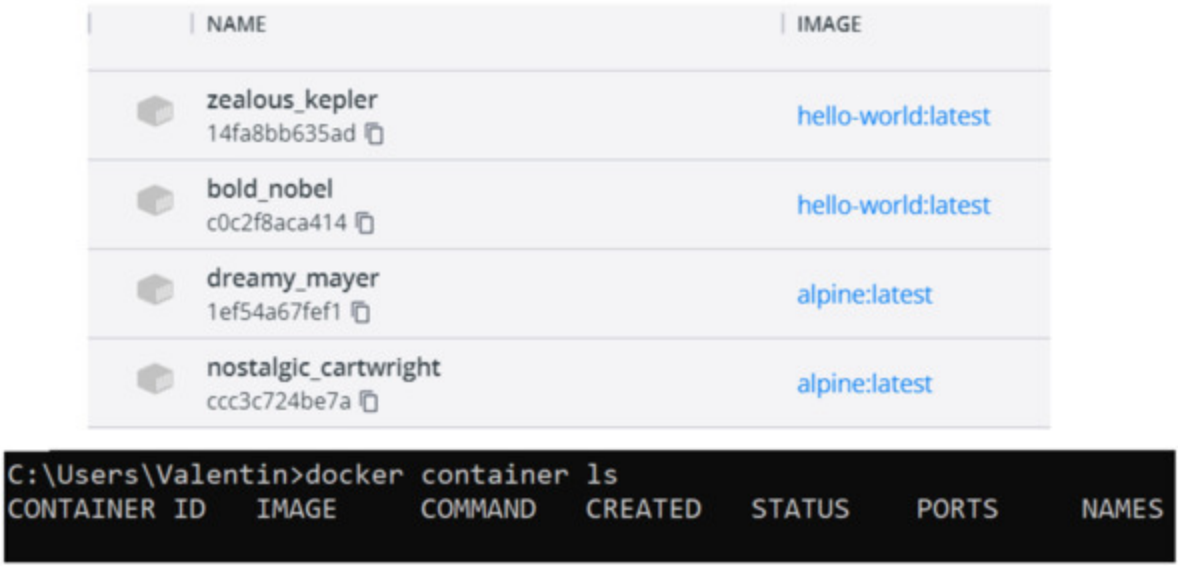


Рис. Список контейнеров

Мы уже видели список образов, которые скачаны с докер-хаба.

Теперь у нас есть список контейнеров, которые были запущены из этих образов. Если мы несколько раз запускаем один и тот же образ, у нас появляется несколько объектов под названием «контейнеры».

Два раза запустили Линукс — должны появится два контейнера. Они в виде файлов хранятся на нашем компьютере. Даже после того, как они были выполнены, они всё ещё хранятся.

Но если мы напишем команду

docker container ls

нам ничего не сообщат. Потому что все контейнеры завершили свою работу. Работающих, выполняемых прямо сейчас контейнеров нет.

При этом **Docker Desktop** выводит список всех контейнеров. В том числе и тех, которые уже завершили свою работу.

Посмотрим список образов. У нас по-прежнему два образа, см. рис.

```
C:\Users\Valentin>docker image ls
REPOSITORY    TAG       IMAGE ID       CREATED          SIZE
alpine        latest    9c6f07244728   2 months ago     5.54MB
hello-world   latest    feb5d9fea6a5   13 months ago    13.3kB
```

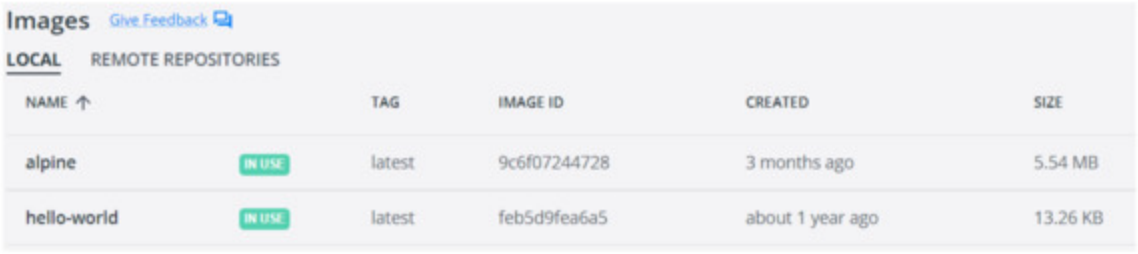


Рис. Список образов

В графическом приложении среде мы дополнительно увидим пометку **IN USE** — используются оба образа. По каждому из них были запущены контейнеры.

Разберёмся в этой ситуации более подробно:

docker container ls — help

Вызываем описание команды **CONTAINER LS**, см. рис.

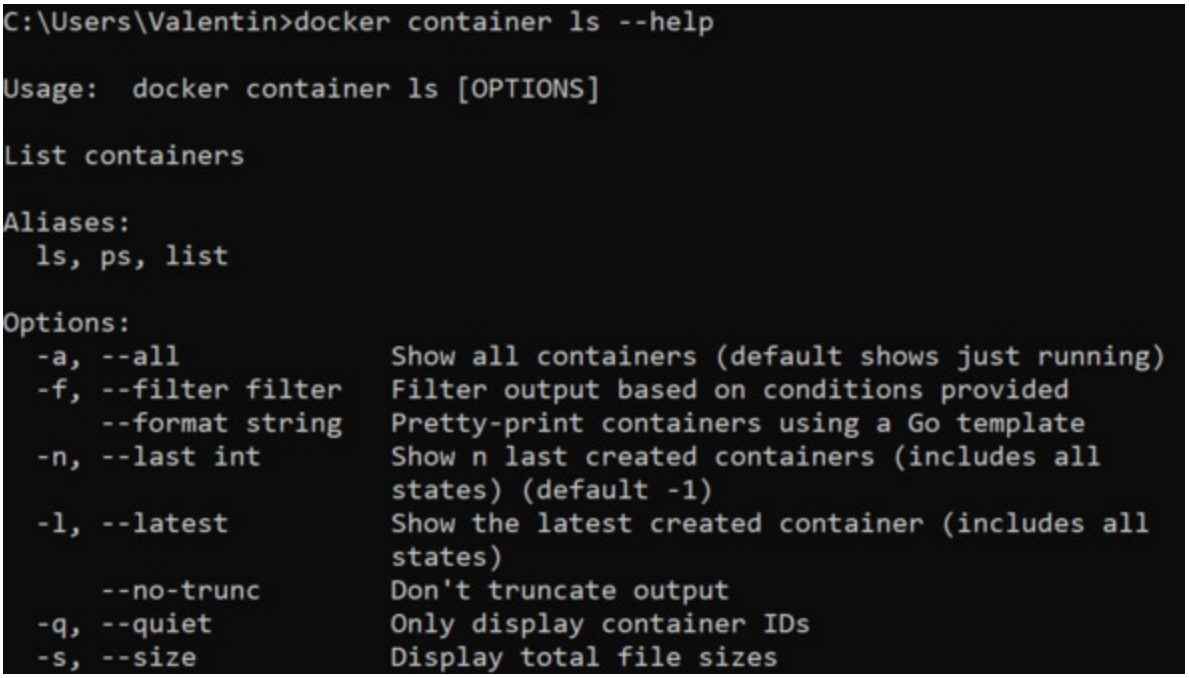


Рис. Параметры команды **LS**

Выясняется, что мы можем указать дополнительные параметры — **Options**. Нас интересует первый параметр: **-a** или — **-all**. Это означает «показать список всех контейнеров». Но дальше говорится: «по умолчанию только выполняемые». То есть если мы не указали никаких параметров, то на экран будет выведен список только тех контейнеров, которые выполняются прямо сейчас — **RUNNING**.

Получается, что если мы вызываем команду **CONTAINER LS**, мы получаем список контейнеров которые прямо сейчас работают.

Но мы можем попросить вывести все контейнеры — и те, которые сейчас работают, и те, которые мы запускали до этого, см. рис.

docker container ls -a

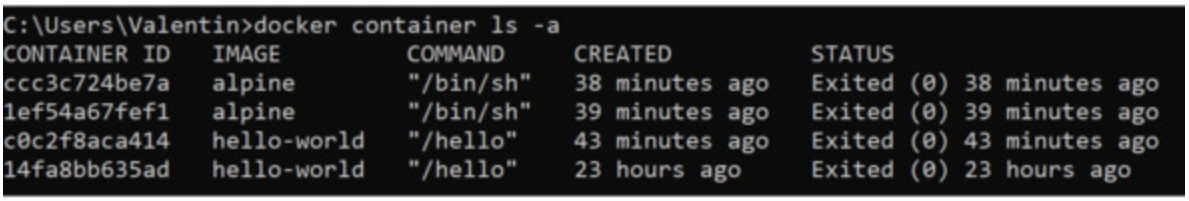


Рис. Список всех контейнеров

Итак, ключ **-a** позволяет нам вывести список всех контейнеров, включая те, что завершили работу.

Мы видим **CONTAINER ID** — идентификатор контейнера. Все идентификаторы разные, и их 4 штуки. В колонке **IMAGE** указаны образы, из которых были запущены наши контейнеры. Два контейнера — из образа **Alpine** и два контейнера — из образа **hello-world**.

Здесь также указано, какая выполнялась команда внутри каждого контейнера — это колонка **COMMAND**.

Для образа **hello-world** выполнялась команда **hello**.

Для образа Alpine вызывали команду **sh**, то есть **shell** — это командная оболочка, интерпретатор командной строки. Но поскольку этот интерпретатор не получил на вход никаких команд, он выполнил эту «пустую команду» и завершил работу.

Это всё равно что нажать в командном окне клавишу **Enter** и ждать сообщений от операционной системы. Мы увидим только приглашение к вводу команд — и больше ничего.

Далее, в колонке **STATUS** указано текущее состояние контейнера: **Exited** — «завершил работу» столько-то минут назад или столько-то часов назад.

То есть ничего на самом деле не произошло. вот это самое «ничего» и было выполнено, когда мы запускали команду

docker run alpine

Для того, чтобы наш «гостевой», «контейнерный» Linux что-нибудь сделал, нужно ему сообщить эту самую команду.

Вспоминается известное шуточное высказывание: «компьютер делает не то, что вы хотели, а то, что вы просили». Как говорится, почувствуйте разницу.

Теперь мы напишем полноценную команду:

docker run alpine ls -l

Здесь мы сообщаем, какую команду нужно выполнить внутри нашего контейнера. Мы получаем список файлов и подробные сведения про каждого из них, см. рис.

```
C:\Users\Valentin>docker run alpine ls -l
total 56
drwxr-xr-x    2 root     root          4096 Aug  9 08:47 bin
drwxr-xr-x    5 root     root           340 Oct 31 09:36 dev
drwxr-xr-x    1 root     root          4096 Oct 31 09:36 etc
drwxr-xr-x    2 root     root          4096 Aug  9 08:47 home
drwxr-xr-x    7 root     root          4096 Aug  9 08:47 lib
drwxr-xr-x    5 root     root          4096 Aug  9 08:47 media
drwxr-xr-x    2 root     root          4096 Aug  9 08:47 mnt
drwxr-xr-x    2 root     root          4096 Aug  9 08:47 opt
dr-xr-xr-x  264 root     root             0 Oct 31 09:36 proc
drwx------    2 root     root          4096 Aug  9 08:47 root
drwxr-xr-x    2 root     root          4096 Aug  9 08:47 run
drwxr-xr-x    2 root     root          4096 Aug  9 08:47 sbin
drwxr-xr-x    2 root     root          4096 Aug  9 08:47 srv
dr-xr-xr-x   11 root     root             0 Oct 31 09:36 sys
```

Рис. Список файлов

Мы видим разрешения для доступа к файлу, у нас указан владелец файла, размеры и так далее.

Мы видим, что этот образ был создан 9 августа.

Внутри этого контейнера находится целая файловая система. Это файловая система была заложена внутри образа — и, соответственно, появилась внутри нашего контейнера.

Вот таким способом мы просим «гостевой» Linux выполнить какую-то команду. Он выполняет команду и тут же завершает свою

работу. Вывел на экран результаты и отключился.

После этого мы можем снова вызвать список контейнеров, и мы обнаружим, что их создаётся все больше и больше с каждым новым запуском.

Если мы несколько раз запускаем образ с разными командами, то для каждого вызова создается новый контейнер, см. рис.



```
C:\Users\Valentin>docker container ls
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

C:\Users\Valentin>docker container ls -a
CONTAINER ID   IMAGE         COMMAND     CREATED         STATUS
ed19b135b12f   alpine        "ls -l"     4 minutes ago   Exited (0) 4 minutes ago
c88a02574266   alpine        "ls -l"     4 minutes ago   Exited (0) 4 minutes ago
320039f9c7ce   alpine        "ls -l"     3 days ago      Exited (0) 3 days ago
8cc4b208da8a   alpine        "ls"        3 days ago      Exited (0) 3 days ago
e43657439902   alpine        "/bin/sh"   3 days ago      Exited (0) 3 days ago
ccc3c724be7a   alpine        "/bin/sh"   3 days ago      Exited (0) 3 days ago
1ef54a67fef1   alpine        "/bin/sh"   3 days ago      Exited (0) 3 days ago
c0c2f8aca414   hello-world   "/hello"    3 days ago      Exited (0) 3 days ago
14fa8bb635ad   hello-world   "/hello"    4 days ago      Exited (0) 4 days ago
```

Рис. Список контейнеров после нескольких запусков

Мы можем посмотреть тот же список контейнеров и в графической среде Docker Desktop, см. рис. Все они завершили работу.

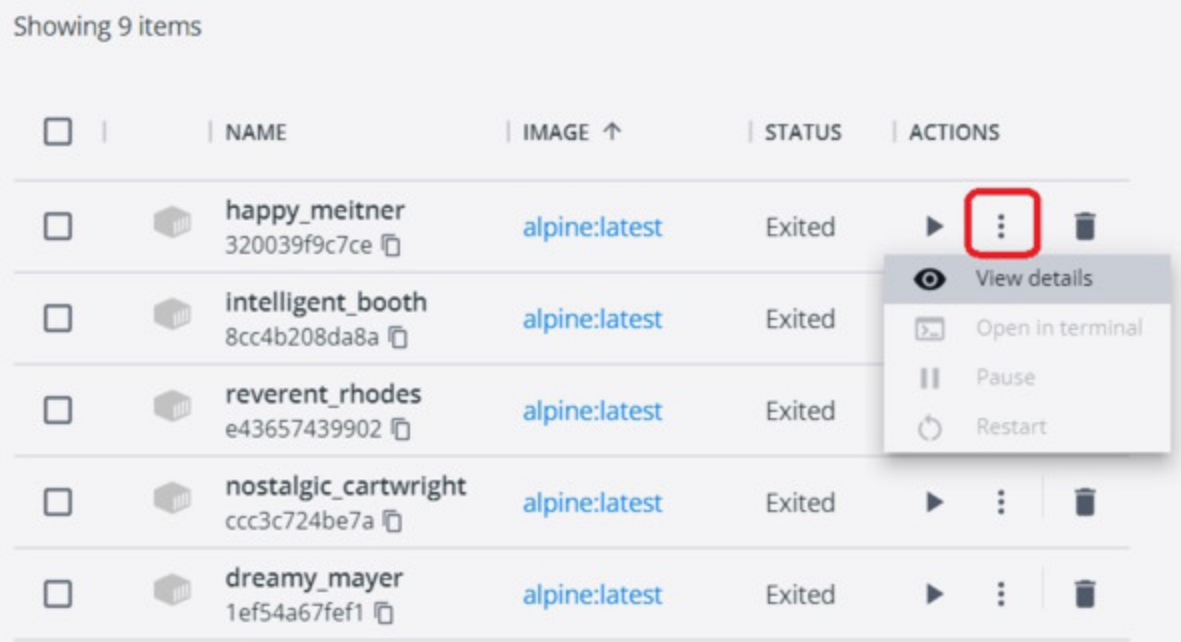


Рис. Список контейнеров после нескольких запусков

Точно также мы можем посмотреть подробности — **View details**. И точно так же мы увидим то, что этот контейнер выводил нам на экран. Увидим всю историю работы с этим объектом. Здесь мы находим тот же самый список файлов, который он выводил нам в окно терминала, см. рис.



Рис. Сообщения контейнера

При этом мы можем посмотреть ещё раз на список образов. По-прежнему мы видим, что у нас всего два образа: **hello-world** и **alpine**, см. рис. И оба образа используются — **IN USE**. Хотя контейнеры уже завершили свою работу.

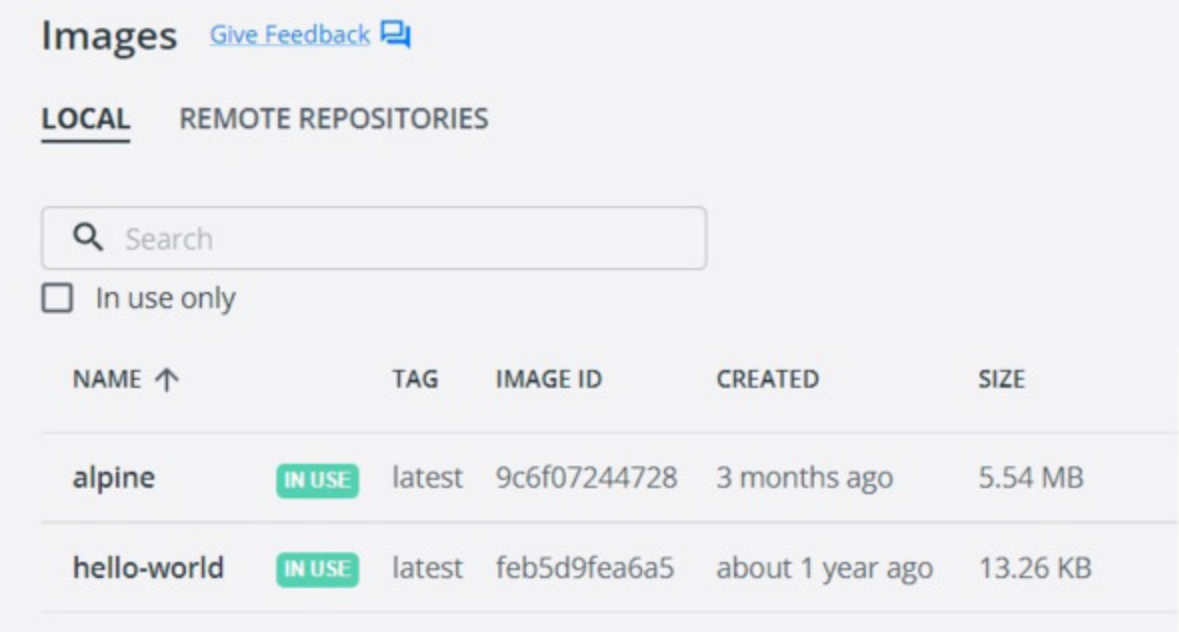


Рис. Список образов

В этом отличие контейнеров от процессов. Процесс после завершения своей работы полностью исчезает. А контейнер только останавливается, но не исчезает.

Так что контейнеры по-прежнему присутствуют на компьютере. Мы запускали их несколько раз, но образов по-прежнему всего два, а контейнеров гораздо больше.

ПОВТОРНЫЙ ЗАПУСК КОНТЕЙНЕРА

Мы можем ещё раз запустить тот же самый контейнер. Открываем список контейнеров, нажимаем стрелочку Start и можем увидеть результаты повторного запуска, см. рис.

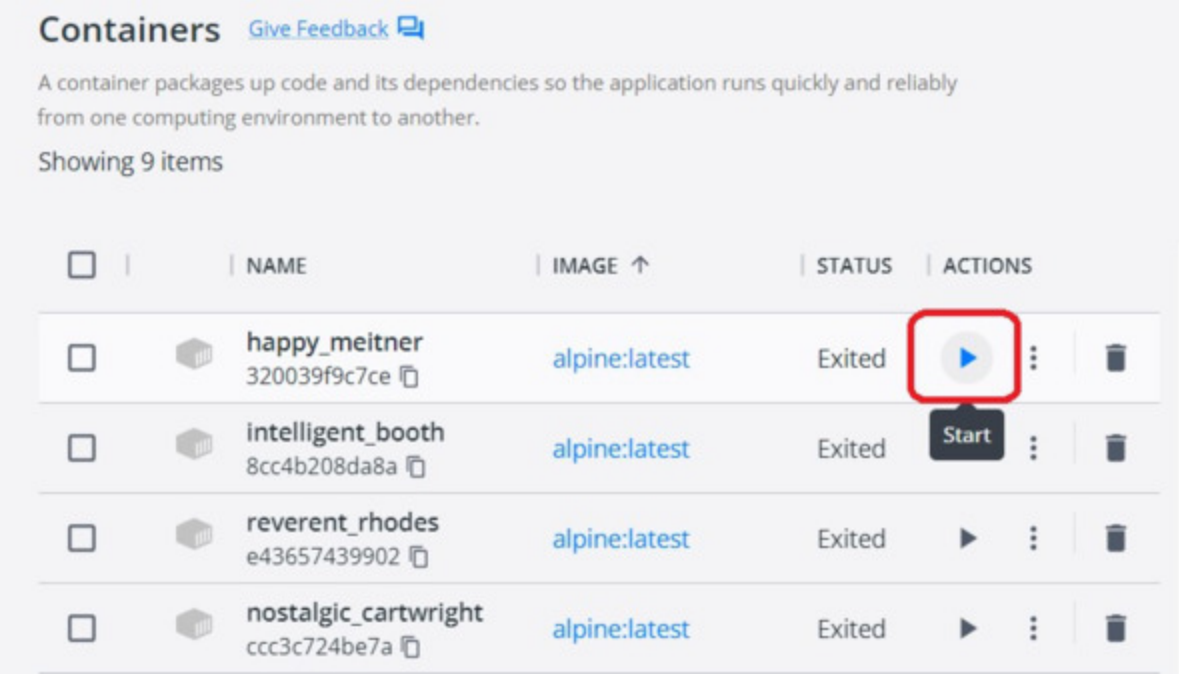


Рис. Повторный запуск контейнера

Мы можем выполнить любую команду, которую можно вызвать в командной строчке. Например, команда **echo** выводит строчку на экран. Если это Linux, то дальнейший текст должен быть в кавычках, см. рис.

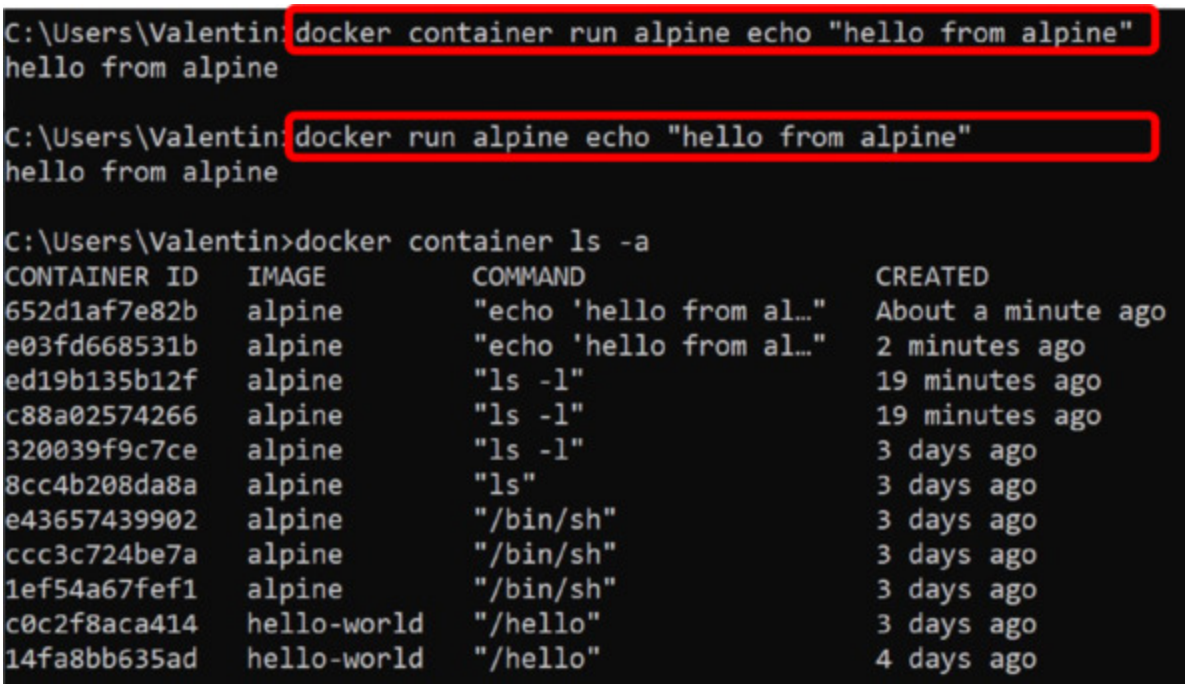


Рис. Вывод сообщения из контейнера

Теперь мы вызываем команду **CONTAINER LS** и видим все контейнеры. Их набирается всё больше и больше. Мы видим команды, которые вызывались. Последние два запуска с командой **echo** находятся в верхних строчках списка. Для каждого запуска создан новый контейнер, и у него свой идентификатор — уникальная строчка цифр и букв. И своё уникальное название.

ДИАЛОГ С КОНТЕЙНЕРОМ

Теперь мы можем напрямую вызвать **shell**, то есть командную оболочку. Но если мы вызываем её без параметров, то ничего не происходит. Это мы уже видели в предыдущих примерах. Ничего не было сделано, и контейнер просто завершил работу, см. рис.

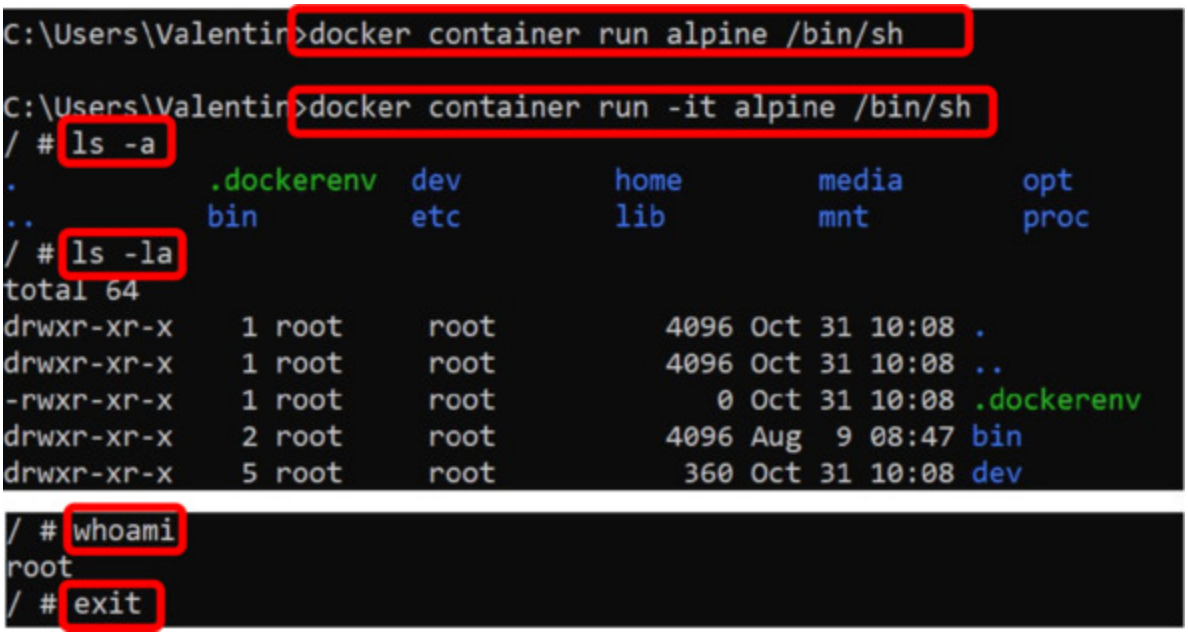


Рис. Запуск оболочки **Shell**

Если мы хотим поработать в командной строке контейнера и вручную вести несколько разных команд, нам потребуется дополнительный ключ **–it**. Это не имеет отношения к информационным технологиям, поскольку здесь одна черточка. Значит, у нас в этом параметре соединили два ключа — ключ **i** и ключ **t**.

Мы вызываем **Shell** с ключом **it**, и теперь мы можем вводить какие-то команды.

У нас появляется приглашение. Очень часто это решетка, что означает «права администратора».

Мы можем уточнить имя пользователя с помощью команды **whoami** — **Who am I** –«Кто я? Как меня зовут?»

То есть «Под каким логином я сюда зашел?»

Нам отвечают: **root**.

То есть внутри контейнера у нас права администратора.

Мы можем ещё поработать. После того, как мы наигрались, мы вводим команду **exit** — «выход из системы». В результате мы выходим из текущего сеанса работы **Alpine Linux** и возвращаемся

в основную операционную систему. Вот как выглядит диалоговый режим работы с контейнером.

Для того, чтобы с режимом **-it** чуть подробнее ознакомиться, мы используем знакомый параметр **help** с двумя чёрточками, см. рис.

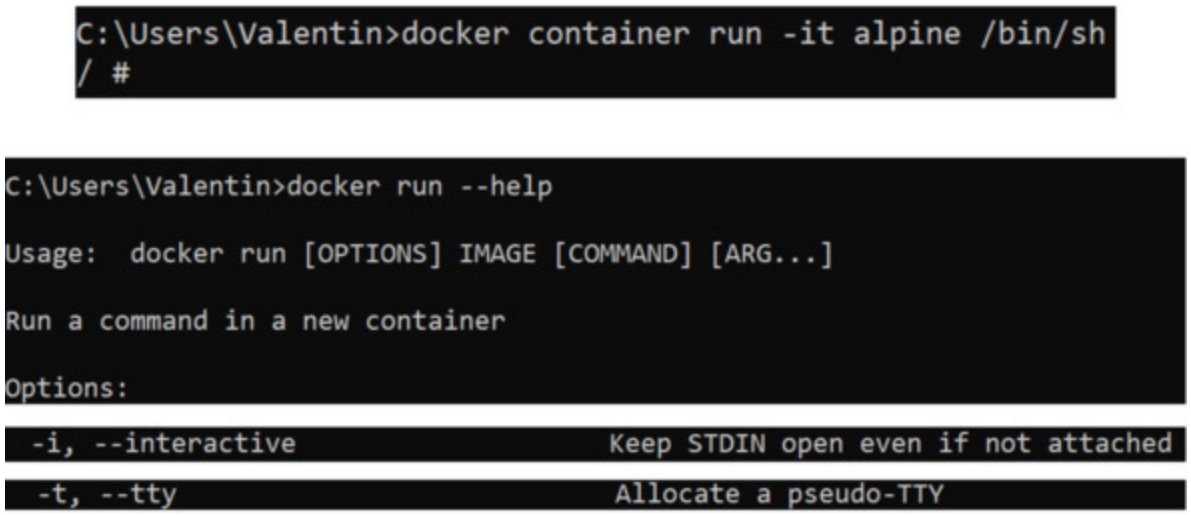


Рис. Интерактивный (диалоговый) режим

Выясняем, что ключ -**i** означает **interactive** — диалоговый режим.

Ключ **-t** означает **tty**. Это терминал Линукса, то есть командное окно.

Интерактивный режим означает, что мы вручную будем вводить команды. Всё, что нам нужно запомнить: мы запускаем контейнер с ключом **-it**.

Таким способом мы можем запустить любой образ Linux, оказаться внутри него и поработать в его терминале с правами администратора.

ПОДКЛЮЧЕНИЕ К КОНТЕЙНЕРУ

Следующий инструмент для работы с контейнером — это подключение через отдельный терминал.

Пока мы не нажали **Enter** после ввода команды **exit**, контейнер всё ещё работает — **STATUS: Running**. И мы можем запустить отдельное командное окно для работы с этим контейнером. А ещё мы можем перейти в раздел **Open in terminal**, см. рис. В обоих случаях мы подключаемся к этому же контейнеру через другое командное окно. И вот в новом командном окне мы сможем с ним поработать.

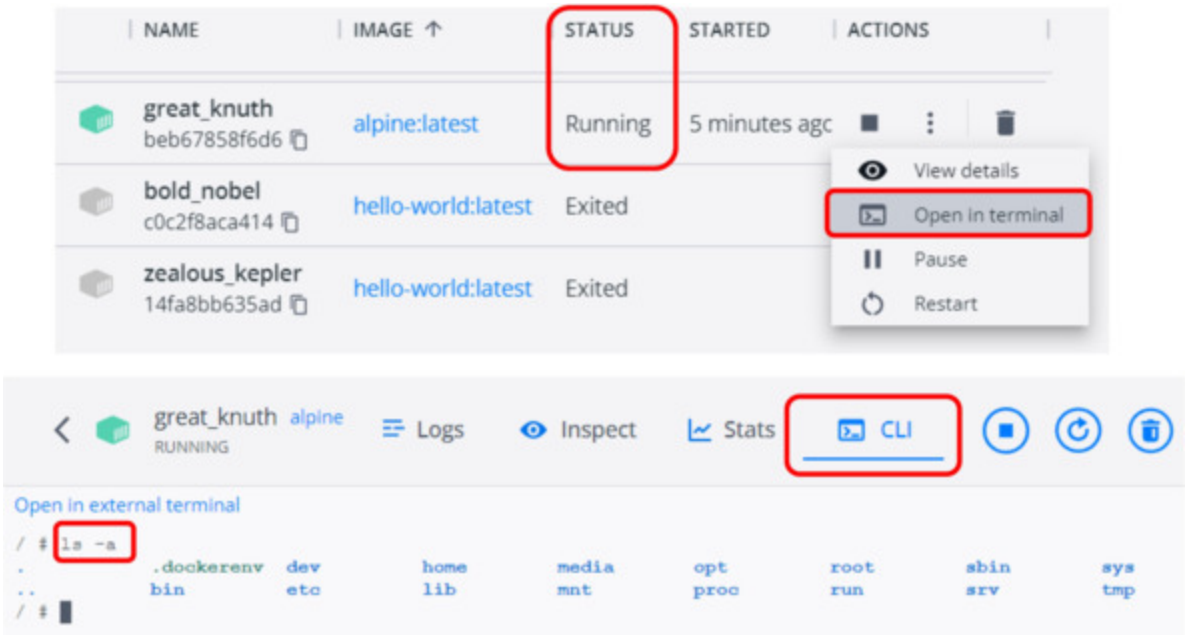
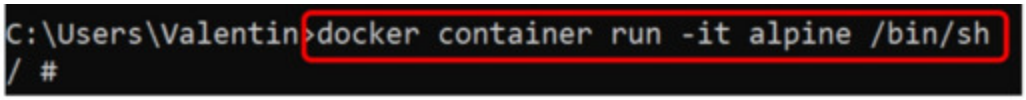


Рис. Открываем новый терминал

Пока не закончена работа с контейнером, мы можем открыть другое командное окно и в нём, к примеру, вызвать список контейнеров, см. рис.



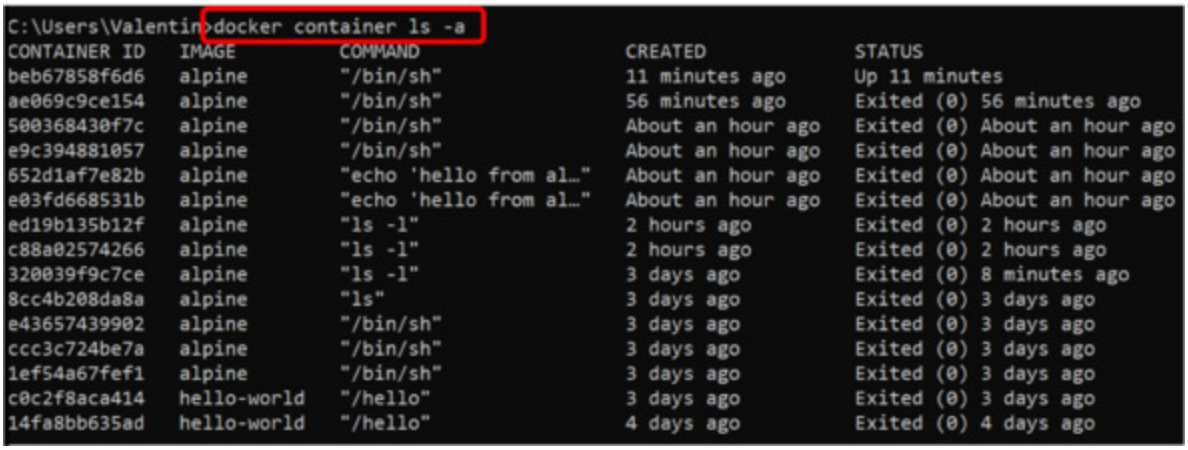


Рис. Список контейнеров — во втором **CMD**

Мы видим, что тут уже по запущено много контейнеров.

Мы видим состояние контейнера в колонке **STATUS: Up 11 minutes** — «контейнер работает уже 11 минут».

Слово **up** в данном случае означает, что объект «работает» или «бодрствует».

Так что здесь говорится, что наш контейнер всё ещё работает, а все остальные завершились.

Мы вызываем список контейнеров во втором командном окне, потому что в первом у нас до сих пор работает Линукс, который всё ещё ждет команду **exit**.

Когда мы нажимаем **exit**, мы выходим из этого режима. Соответственно, контейнер тоже завершает работу, чтоб будет отражено в списке.

Напомним, что наши контейнеры — это, в сущности, файлы. И эти файлы по-прежнему хранятся на нашем локальном компьютере. Они, естественно, занимают место. Они засоряют наш список. И если они не нужны, то желательно их удалять.

Самый простой вариант — перейти в графическую среду, нажать галочку, чтобы выделить всё в этом списке, потом нажать кнопочку

Delete — удалить. Затем мы подтверждаем: да, мы хотим удалить всех и удалить навсегда — **Delete Forever**, см. рис.

Конечно же, есть соответствующая команда для терминала.

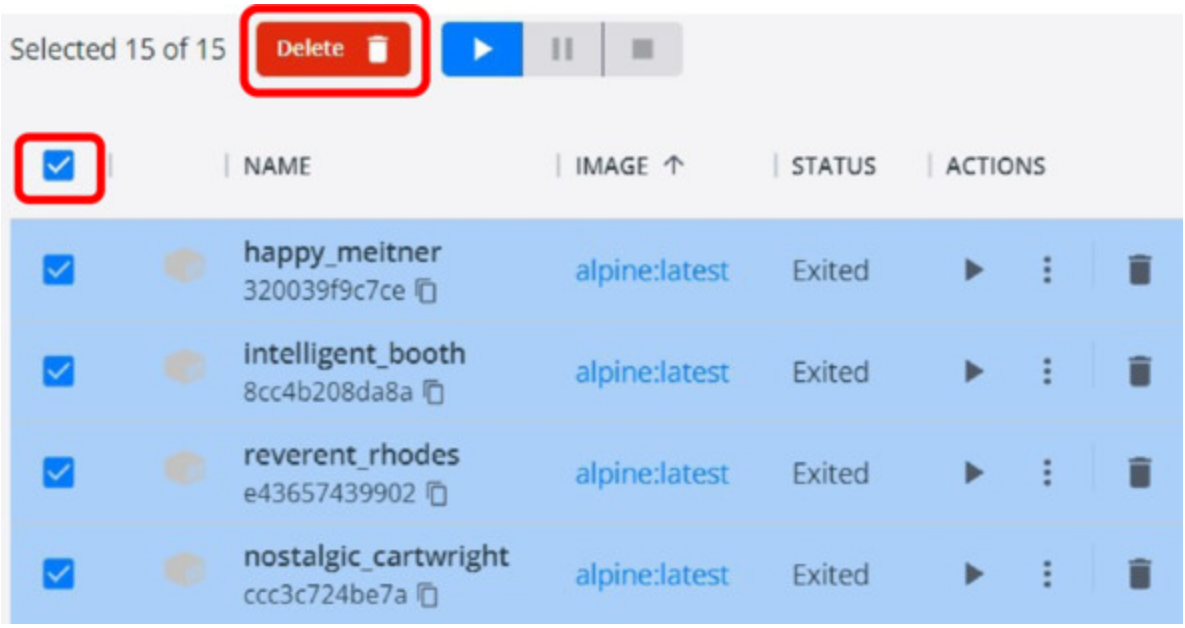


Рис. Delete Containers — Delete Forever

НАСТРОЙКА ОБРАЗОВ

Далее мы разберём вторую виртуальную лабораторную работу — в самых общих чертах. Здесь нам предстоит поработать с образами. Мы будем создавать, «собирать» свои образы — как в детском конструкторе собирают модель из деталей. Затем их можно хранить и на локальной машине, и в облачном репозитории — на Docker Hub. Оттуда мы сможем их использовать. Другие тоже смогут пользоваться этим образом.

В нашем примере мы разберём самые простые действия. Опыт показывает, что самое сложное — это освоить начальные, простые действия. Этот момент обычно пропускают, и человек сразу запутывается.

Есть даже такая поговорка: «Упрощать — сложно, усложнять — просто».

Даже некоторые книжки «для чайников» слишком сложные для первого знакомства с материалом. Поэтому мы упрощаем ещё сильнее, чтобы любой смог проделать вместе с нами самые первые шаги и освоить эту технологию.

Ссылка на страницу работы:

Virtual Lab 2. Customizing Docker Images:

https://training.play-with-docker.com/ops-s1-images/

ЗАПУСКАЕМ BASH

Вот наша первая команда:

docker container run -ti ubuntu bash

Обратим внимание, что мы можем написать ключ -**i** и ключ **-t** в любом порядке. Здесь после черточки у нас идет **-ti**. Это равносильно ключу **–it**. Так мы сможем вызвать образ **Ubuntu Linux** и поработать в его терминале, в командном окне. Точно также сможем вводить любые команды **Linux**, которые доступны в командном окне.

Затем мы познакомимся с простейшими действиями по установке программ в Linux. Как и в предыдущем примере, эту работу можно выполнять и в облаке через любой браузер, и на своей машине.

Для работы в облаке мы переходим по ссылке Play-with-docker и запускаем вторую виртуальную лабораторную работу, см. рис.

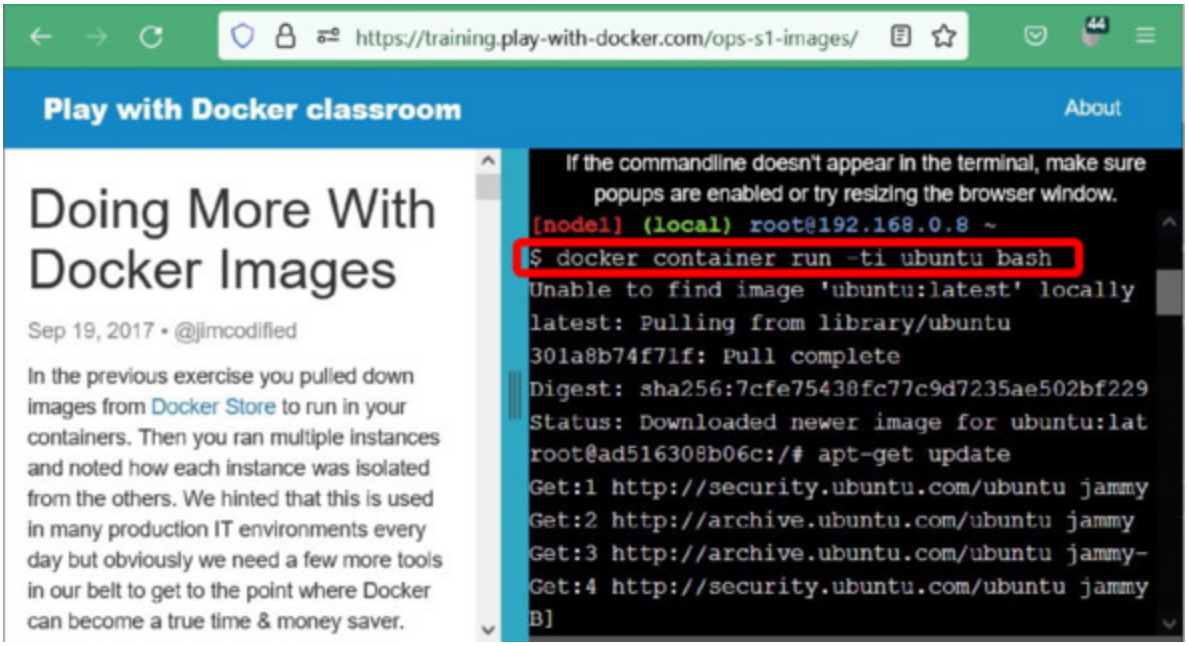


Рис. Начало работы в облаке

В левой части окна у нас появляется подробнейшее описание — с картинками и с примерами. И всё это, естественно, вам предстоит почитать и разобраться в материале. Если нужно, используйте автоматический перевод в браузере. Я только даю некоторые комментарии.

В правой части окна мы видим традиционное командное окно: черный фон и белые буковки. Некоторые буковки разноцветные — со смыслом. Над этим смыслом тоже можно поразмышлять — что какой цвет здесь может обозначать.

Итак, мы получаем доступ к полноценному командному окну. Единственное, что наши команды будут выполняться где-то там далеко, на сервере, в интернете. А результаты выводятся сюда, на наш экран, через браузер.

И вот мы пишем эту самую первую команду. Она начинает выполняться она выполняется, и мы видим первое сообщение: на локальной машине образ не найден. Значит, он будет

скачиваться — и дальше показано, откуда он скачивается и что с ним там происходит.

Точно также мы абсолютно ту же самую команду вводим на своей локальной машине, в своем командном окошечке, в своём терминале. И получаем примерно такие же сообщения, см. рис.

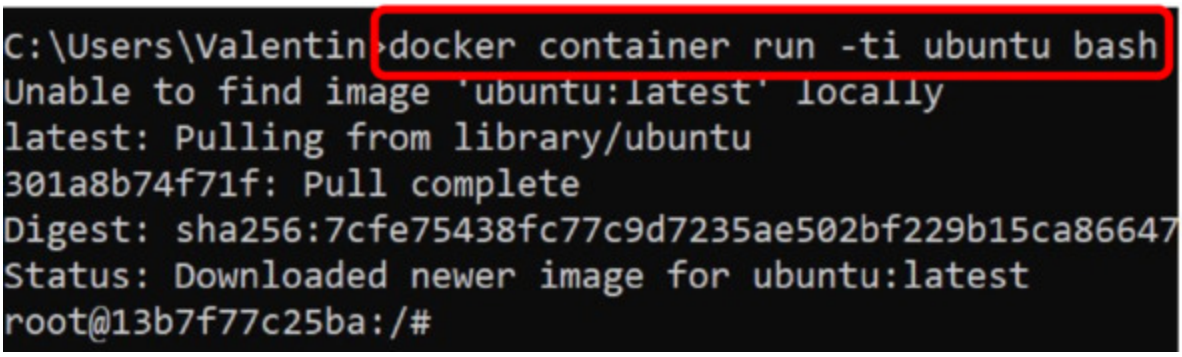


Рис. Начало работы на локальной машине

Разница будет только в обозначениях этих контейнеров. Идентификаторы для каждого контейнера назначаются индивидуально.

Здесь говорится: образ **ubuntu** — двоеточие — **latest**. То есть это самый свежий, самый последний образ. **Latest** — это тег, или метка. Если мы не указали тег, а просто попросили Ubuntu, значит не говорим, какая версия будет скачана. Нам придёт последняя версия.

Этот файл очень небольшого размера — всего несколько мегабайт. Этот Линукс разработчики «урезали» до командной строки. В конце нам говорят: «состояние: образ успешно скачали». У нас появляется приглашение к вводу команд — решёточка. Символ решёточки означает, что мы можем вводить команды с правами администратора.

GNU BASH

Внутри нашего контейнера вызывается команда **Bash**. Чтобы с этим инструментом познакомиться, вам предстоит просмотреть несколько страничек в интернете, см. рис.

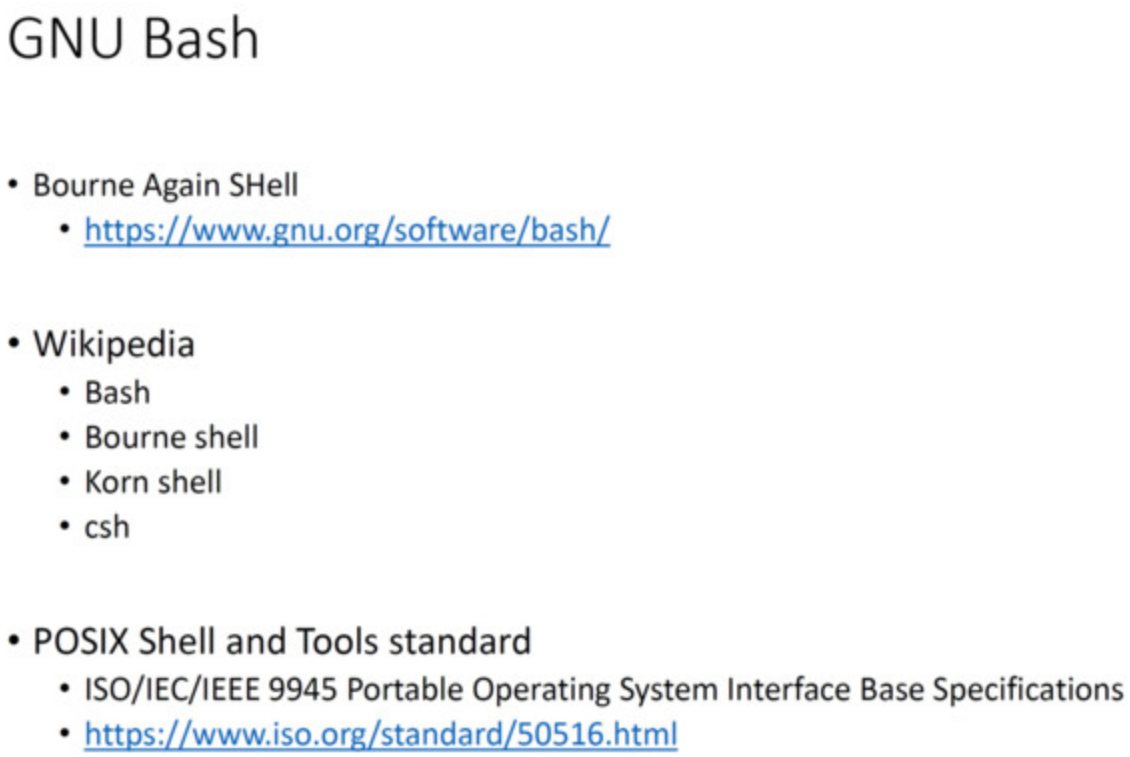


Рис. Знакомимся с **Bash**

Название **Bash** расшифровывается так:

Bourne Again SHell

Это для нас пример того, как программисты развлекаются с названиями. За этим целая история развития операционных систем Unix и Linux. Первоначально был просто **Shell** — командная оболочка ОС. Потом появились разные версии этой командной оболочки: **B Shell**, **C Shell**, **K Shell** и так далее. Например, **K Shell** — это в честь автора. Korn — это фамилия.

Наконец, **Bourne again** звучит как **Born again**. Здесь начинается игра слов. **Born** означает «рождённый», а **Bourne** — это фамилия автора. Звучит похоже, но смысл разный. Это новая версия **B Shell**.

Все эти варианты командной оболочки различаются по языку программирования скриптов. Утверждается, что **Bash** полностью соответствует международным стандартам.

Этот стандарт очень своеобразный. Во-первых, он называется **POSIX**. Этот стандарт разрабатывает организация nol названием ISO International Standardization Organization — международная организация по стандартизации.

Если присмотреться к названию стандарта — здесь на самом деле несколько участников, кроме ISO. Тут перечислены IEC и IEEE — это международные (американские) организации, которые занимаются стандартами в области электроники, компьютеров, программирования и много чем ещё.

Задание

https://www.gnu.org/software/bash/

Просмотрите в Википедии следующие статьи:

Bash

Bourne shell

Korn shell

csh

POSIX Shell and Tools standard

ISO/IEC/IEEE 9945 Portable Operating System Interface Base Specifications

https://www.iso.org/standard/50516.html

ОБНОВЛЕНИЕ И УСТАНОВКА

Итак, Bash, который вызывался в этой команде, — это один из вариантов командного окна, со своим языком программирования. Дальше мы переходим к новой процедуре. В Линуксе установка программ делается очень непривычным образом. Для тех, кто работал Windows, это будет совершенно другой процесс.

Действие первое:

apt-get update

Слово **update** означает «обновить». Обновить некий список. Мы вводим эту команду и видим много сообщений, которые говорят про какие-то сайты в Интернете. Если это был образ Ubuntu, то здесь обновляется список программ. И каждый раз она говорит: это взято на ubuntu.com. Очень длинный список. Всё, что нам нужно сделать, — это дождаться завершения этого процесса, см. рис.

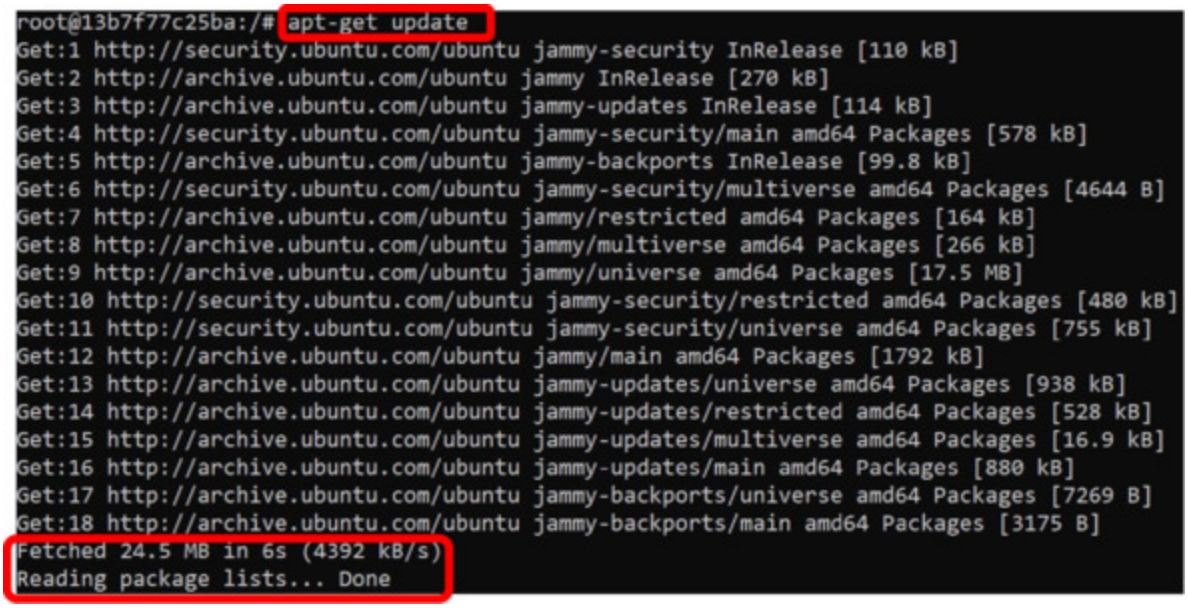


Рис. Обновление списка пакетов

В конце нам говорят: да, завершено успешно. Прочитаны списки пакетов. Слово «пакеты» — оно ключевое для Linux, и оно означает не то же самое, что в Windows.

Кроме этого, здесь говорится **fetch**, то есть получено, за 6 секунд. 24 мегабайта информации получено с какой-то скоростью.

Команда update вызывается перед тем, как что-то устанавливать. Более или менее регулярно делается обновление списка пакетов.

УПРАВЛЕНИЕ ПАКЕТАМИ ПРОГРАММ

Что здесь проиисходит? Во-первых, сама команда **apt-get** или в некоторых случаях просто **apt**. Это инструмент для управления пакетами прикладных программ — **software package manager**, см. рис.

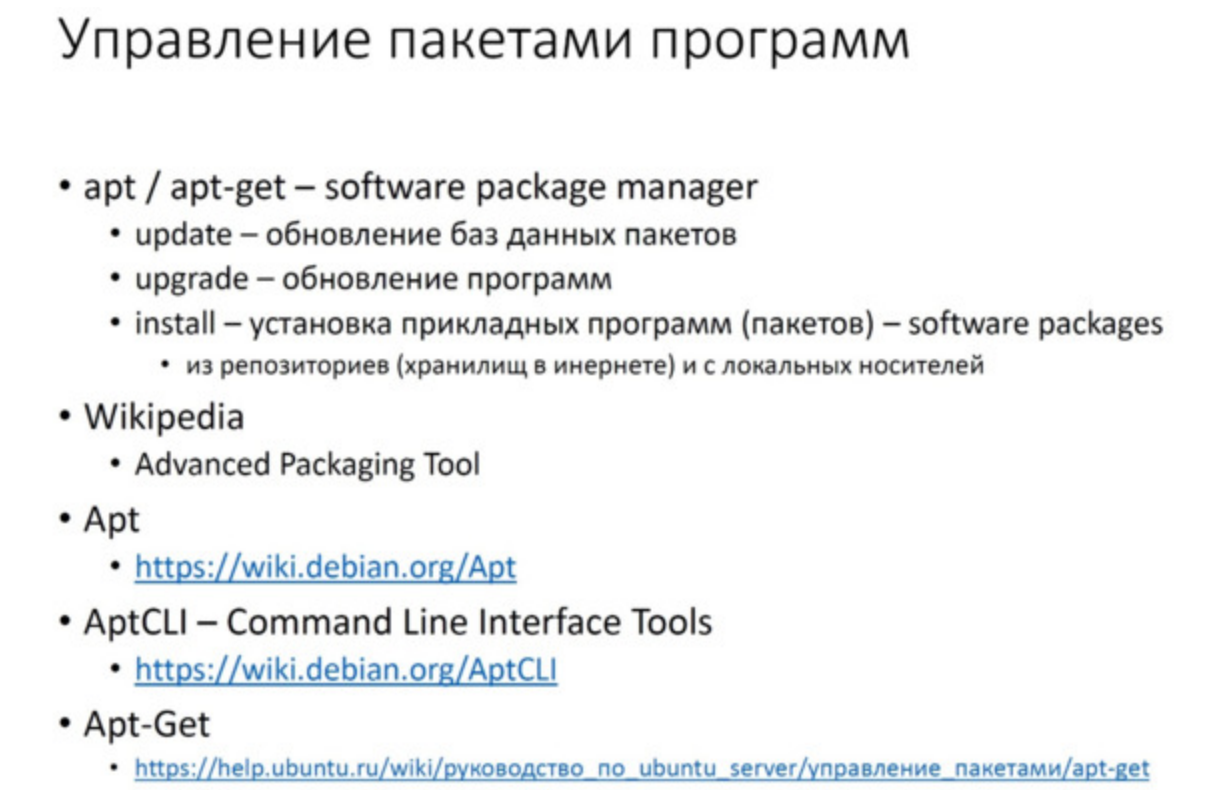


Рис. Технологии управления пакетами

Когда говорят «пакет» или «пакет программ», то для Windows это просто называется «программа», или «прикладная программа», или **Application**, или просто «приложение».

Когда мы говорим «установить программу» или «установить пакет программ», подразумевается, что в этом комплекте много разных вещей. Например, офисный пакет Microsoft Office или Libre Office, или Open Office, или My Office, или ещё какой-нибудь «офис». В этом программном продукте содержится много всего полезного. Там есть текстовый редактор, электронная таблица, презентации и что-нибудь ещё. Всё это вместе называется «пакет программ».

Но может быть одно-единственное приложение, один единственный **Application**.

В Линуксе «пакетом» называют «пакет прикладных программ» — **software package**. А команда **apt** вызывает менеджера этих самых пакетов. Подробнее можно почитать на Википедии: **Advanced Packaging Tool**.

Advanced — значит «улучшенный, с новыми возможностями». Сокращенно **apt**. Вот это и есть тот самый **apt**, иногда сюда добавляется черточка и слово **get**. Это разновидность **apt**. Это менеджер прикладных программ или пакетов программ. Он позволяет устанавливать программы и удалять программы.

В операционной системе Windows мы обычно скачиваем с интернета установочную версию — **Installer**. Мы её запускаем, и она устанавливается. В процессе установки мы много раз говорим **OK — Next — OK**. И вот наконец она установилась. А если нужно удалить программу, то мы через настройки системы переходим в список программ и говорим «удалить». Так вот здесь, в Линуксе всё это делается через командную строчку.

Вводим команду **update**. В этом случае мы только просим локальный компьютер прочитать новый список доступных пакетов программ, которые лежат в официальном хранилище — в данном случае компании Ubuntu.

Далее, у нас есть вторая команда, вторая опция **upgrade**, и это немножко другое. «Апгрейд» значит обновить те программы, которые уже установлены на компьютере. «Апдейт» — это только список, «Апгрейд» — это сами программы. «Проапгрейдить» — установить новые версии, самые последние, самые свежие.

Задание. Просмотрите в Википедии следующие статьи:

Advanced Packaging Tool

Apt

https://wiki.debian.org/Apt

AptCLI — Command Line Interface Tools

https://wiki.debian.org/AptCLI

Apt-Get

https://help.ubuntu.ru/wiki/руководство_по_ubuntu_server/управление_пакетами/apt-get

УСТАНОВКА ПРОГРАММЫ

Дальше будет интересовать команда **install**.

Первым шагом мы получили список программ, а вторым шагом мы говорим, что хотим установить программу:

apt-get install -y figlet

Установка прикладных программ в Линуксе происходит автоматически, и в этом случае нас совершенно не интересует, где что лежит и в каком каталоге. Нам не нужно десятки щёлкать мышкой и скачивать файлы. Всем этим занимается **apt**. Мы просто хотим установить какую-то программу, а всеми вопросами — где она лежит — этим занимается база данных. Она лежит в где-то в общем хранилище, или репозитории, или где-нибудь ещё, или, может быть, она даже скачана на машину. Это для нас совершенно не важно.

Итак, следующим шагом мы вызываем команду **Install**. Мы указываем название пакета — то есть программу, которая будет установлена. Плюс к этому мы сообщаем ключ **–y**. От слова **YES**. Это означает, что нам не понадобится говорить много раз: **OK — Next — OK**.

Менеджер пакетов сам всё сделает, сам всё скачает.

Мало того, он следит за тем, не нужно ли что-то ещё для нашей программы. Если мы в операционной системе Windows ставим какую-то программу, время от времени нам говорят: для работы вам нужно дополнительно установить вот это и вот это, да ещё определённую версию. В некоторых случаях вообще вручную надо скачивать дополнительные пакеты и библиотеки, самому куда-то их устанавливать.

А вот менеджер пакетов делает всё сам. И это называется **dependency** — дополнительные зависимости. То, что требуется для

работы этой программы. Менеджер **apt** сам их установит, причём в нужном, правильном порядке. Просто на экран выводится много сообщений о том, что наш менеджер для нас хорошего сделал, см. рис.

Как ни странно, в командной строчке многие вещи делаются гораздо проще и быстрее, чем с помощью мышки. Мы просто вводим одну команду — и программа установилась!

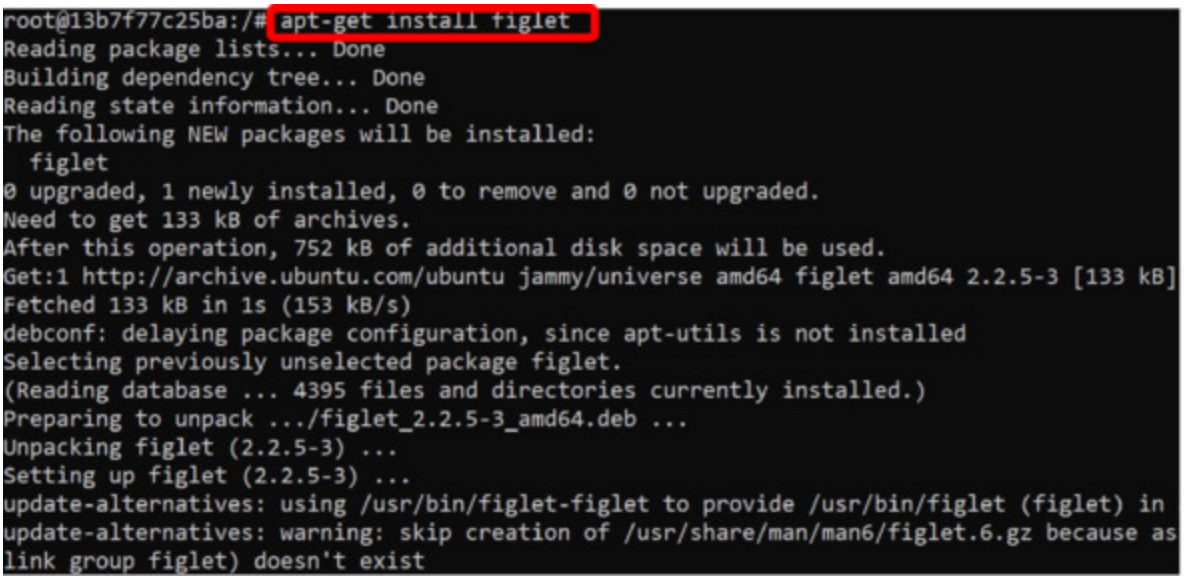


Рис. Установка программы **figlet**

FIGLET

Что же это за программа — **figlet**? Это слово состоит из двух частей.

Первая часть — **fig** означает **Figure**, а английское слово **Figure** означает «рисунок» или «картинка». Мы говорим: «Рисунок 1», «Рисунок 2». В английской статье будет сказано: Figure 1, Figure 2. Итак, **figure** — это «картинка, рисунок, иллюстрация».

Вторая часть слова: **–let**. Если слово кончается на **let**, это означает «маленький». **Figlet** — это «маленькая картина»,

«картиночка». В русском языке это слово заканчивается на «**-чка**». Вот такое «уменьшительно-ласкательное» окончание.



Рис. Расшифровка названия **figlet**

Подробнее можно про эту программу почитать на Википедии. На сайте **figlet.org** объясняется, что это такая программа, которая рисует на экране большие буквы. Но они будут составлены из обычных символов, которые имеются в обычном тексте. То есть большие буквы будут нарисованы в текстовом режиме, в командном окне, в терминале. А в командном окне мы можем выводить все символы только одного размера. Можно изменить размер шрифта, но он будет влиять на всё окно. А здесь из обычных буковок и чёрточек будут составлены большие буквы.

ЗАПУСКАЕМ FIGLET

Мы выяснили, что **figlet** — это такая шуточная программа, игрушечная, для удовольствия. Программисты иногда для удовольствия пишут программы, и это тоже любопытно. Это целые технологии, и это выглядит очень просто. Мы пишем команду **figlet** — пробел — и в кавычках какой-нибудь текст, см. рис.



Рис. Запускаем **figlet**

В нашем примере это простенькое сообщение типа «всем привет». Кавычки используем английские, двойные.

После этого наш текст выводится на экран большими буквами. Эти буквы состоят из отдельных символов — горизонтальные, вертикальные, наклонные чёрточки, скобки и так далее.

Можно через дополнительные параметры выбрать наклонные буквы или какими символами набирать текст на экране.

Мы видим здесь и запятую, и восклицательный знак — все они составлены из отдельных символов. Можете себе представить, сколько за этим развлечением работы. А вроде бы шуточная, развлекательная программа, которая просто выводит текст на экран в виде больших букв.

Вот мы с вами и посмотрели, как установить программу в Линуксе. И как её потом запустить. Ничего сложного. Нужно просто потренироваться и привыкнуть.

ФИКСИРУЕМ ИЗМЕНЕНИЯ В ОБРАЗЕ

Мы кое-что изменили в нашем контейнере — по сравнению с исходным образом. Мы установили новую программу. Теперь мы можем записать эти изменения и создать новый образ, где уже будет наша новая, установленная программа.

Для этого нам понадобится новая команда:

docker container commit CONTAINER_ID

Команда **COMMIT** означает «зафиксировать изменения».

Обычно после завершения работы контейнера все изменения пропадают. Внутри каждого контейнера свои изменения, которые никак не влияют на образ. А когда мы пишем **commit**, у нас создаётся новый образ. И его можно будет повторно запускать. Этот образ будет собран на основе предыдущего образа, и к нему будет добавлено несколько команд, которые мы вводили.

Мы вводили команды **apt update** и **apt install**. Вот эти две команды будут дополнительно записаны внутри нового образа.

Вообще-то, слово **COMMIT** пришло из технологий управления версиями программ **GIT**. Это отдельное направление, он относится к разработке программ. И разработчики программ так и говорят: «закоммитить изменения». Эта идея «фиксировать изменения» перекочевала и в контейнеры. **Git** существует лет 10—15, а docker — лет пять (цифры очень приблизительные).

Итак, мы пишем **docker commit**. При этом мы указываем идентификатор контейнера, в котором были изменения: вот мы запустили образ, внутри контейнера что-то сделали, теперь зафиксируйте, пожалуйста, все наши изменения, которые мы сделали во время работы. А идентификатор контейнера мы узнаем, кода выводим на экран список контейнеров. Дальше мы просто выделяем мышкой эту строчку, копируем и вставляем в новую команду. В результате, у нас в списке образов появится новая строчка, см. рис.

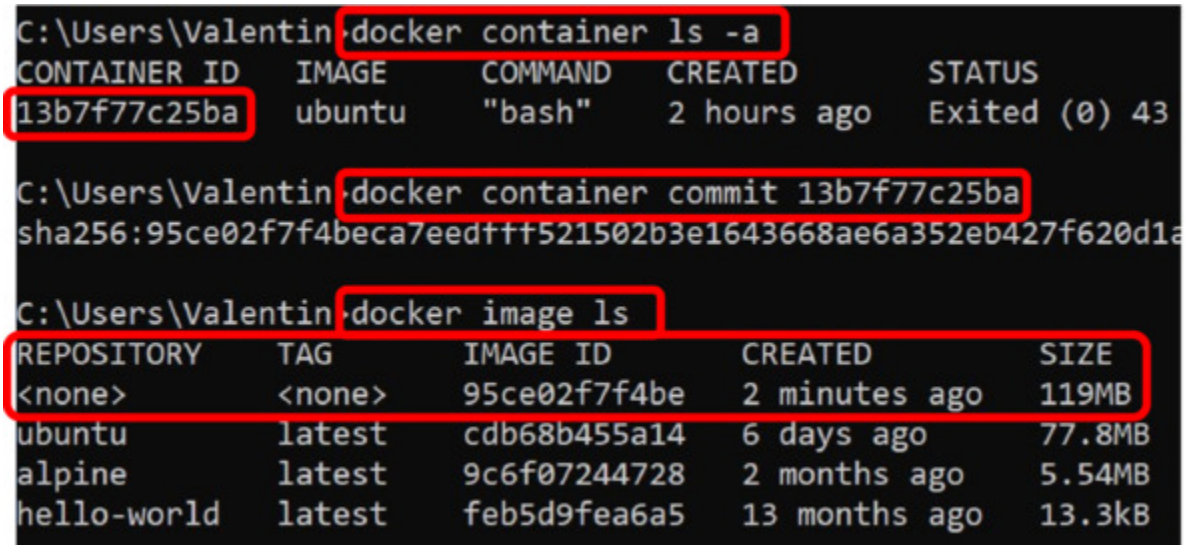


Рис. Фиксируем изменения

В нашем списке было три образа:

— Hello, World

— Alpine Linux

— Ubuntu Linux

Теперь нам достаётся четвертый образ. И он пока что без названия. У него нет ни названия, ни метки. У него есть только идентификатор **IMAGE ID**. Видно, что он был создан — **CREATED** — буквально несколько минут назад. Его размер — столько-то мегабайт. Он, естественно, больше, чем предыдущий образ **Ubuntu**. Было 70 мегабайт, стало 100 мегабайт. Всё потому, что дополнительно были скачаны все необходимые программы и библиотеки.

Итак, у нас появился новый образ.

ПЕРЕИМЕНОВАНИЕ ОБРАЗА

Следующим нашим шагом будет переименовать этот образ:

docker image tag IMAGE_ID ourfiglet

Копируем идентификатор образа **IMAGE_ID** и вставляем его в новую команду. TAG присвоит новое название, метку указанному

образу. Мы указываем новое название: **ourfiglet**, см. рис.

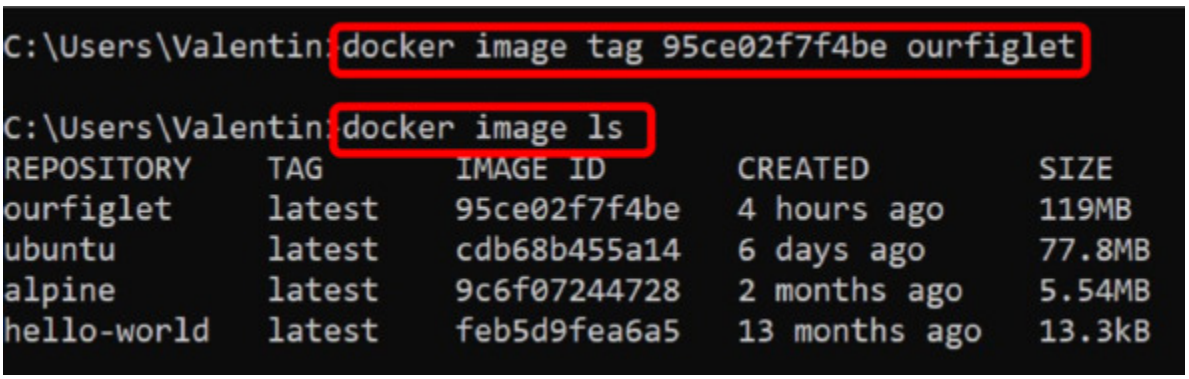


Рис. Переименуем образ

Теперь мы просмотрим список образов:

docker image ls

Мы видим, что четвертый образ получил свое новое название. Теперь с ним будет гораздо легче работать. Три предыдущих образа остались на месте.

Что же мы сделали? Мы запустили стандартный образ, потом в контейнере произвели изменения, и затем зафиксировали эти изменения и переименовали образ. И теперь у нас есть новый образ с новым названием.

ВЫПОЛНЕНИЕ КОМАНДЫ В КОНТЕЙНЕРЕ

Теперь у нас есть новый образ. Пока что он хранится на нашем локальном компьютере, и мы можем запустить его на выполнение. Тогда из него будет создан новый контейнер — на время работы.

Следующим шагом мы запускаем именно наш новый образ:

docker run ourfiglet figlet hello

Указываем имя образа, а внутри него запускаем команду **figlet**. Передаем в качестве параметра простое сообщение **hello**. И вот наш контейнер выводит слово **hello** на экран, и оно составлено вот из чёрточек и скобочек, см. рис.

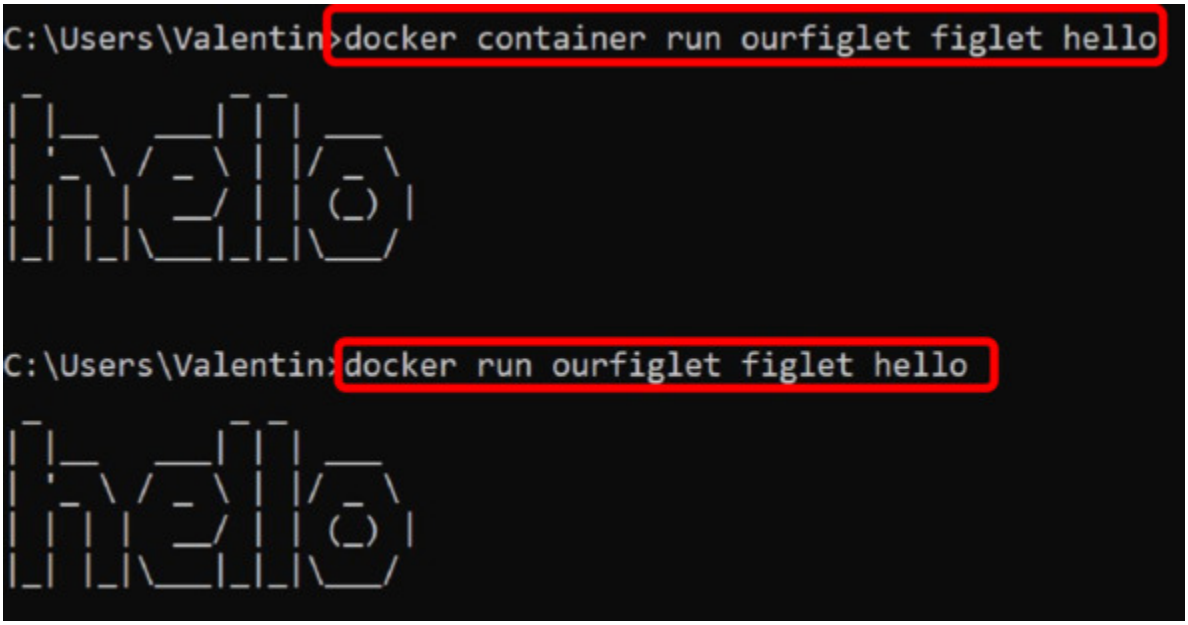


Рис. Вызов **figlet** из нового образа

Мы собрали новый, дополнительный образ из стандартного образа Ubuntu, поверх чистого Линукса были выполнены несколько команд. Эта система по-прежнему представляет собой Ubuntu, просто в ней установлена дополнительная программа под названием **figlet**.

Теперь нам не нужно каждый раз устанавливать свою программу. Мы просто выполняем команду figlet и передаем ей в качестве параметра текст.

Таким образом, мы можем создать свой собственный контейнер. И чаще всего для этого за основу берётся уже готовый, стандартный, «чистый», «голый» образ — в котором ничего нет. Мы можем установить какие-то дополнительные программы и потом использовать его.

СОЗДАЁМ ОБРАЗ С ПОМОЩЬЮ DOCKERFILE

Мы рассмотрели пример того, как создать новый образ на основе контейнера с учётом сделанных изменений. Это была демонстрация основной идеи.

Теперь рассмотрим основную технологию создания образов. Для этого вся конфигурация будущего образа описывают в текстовом файле под названием **Dockerfile**. Обратим внимание, что расширения у названия этого файла нет.

В следующем примере нам предлагают создать приложение средствами Node. js. Эта короткая программа будут выводит на экран приветствие и сообщать нам **hostname** — имя нашего компьютера.

Задание. Просмотрите в Википедии следующие статьи:

— **Hostname**

— **Node. js**

— **JavaScript**

Для начала создадим текстовый файл **index. js** — нашу маленькую программу, см. рис. В качестве упражнения будем вводить текст на клавиатуре с помощью команды:

copy con file

Эта команда копирует текст, который мы вводим на клавиатуре, и записывает его в файл с указанным названием. Специальное слово **CON** означает **console** — «консоль» (клавиатура плюс экран). После ввода текста можно нажать комбинацию клавиш **Ctrl + Z**. Описанная команда работает в ОС **Windows**.

Можно также использовать любой текстовый редактор, например **Блокнот** — **Notepad**.

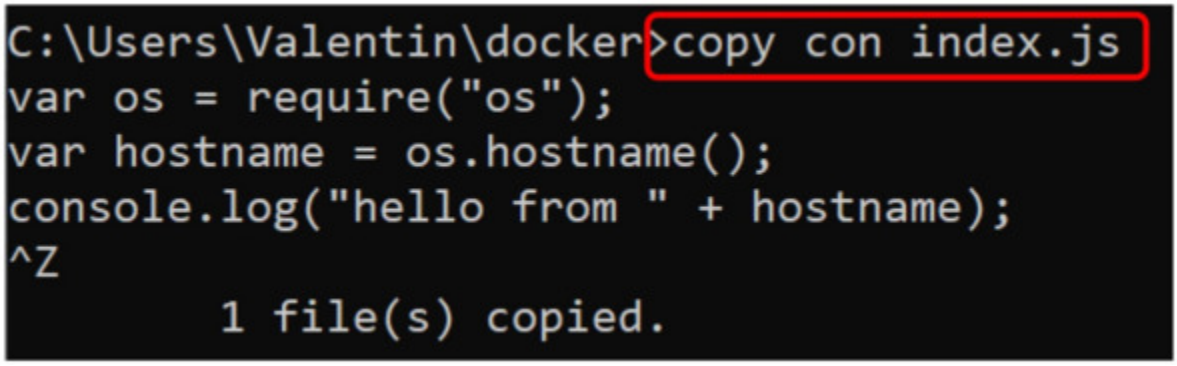


Рис. Создаём тест приложения **index. js**

Наша программа на **JavaScript** готова. Теперь опишем состав нашего будущего образа. Для этого создадим текстовый файл **Dockerfile**, см. рис.

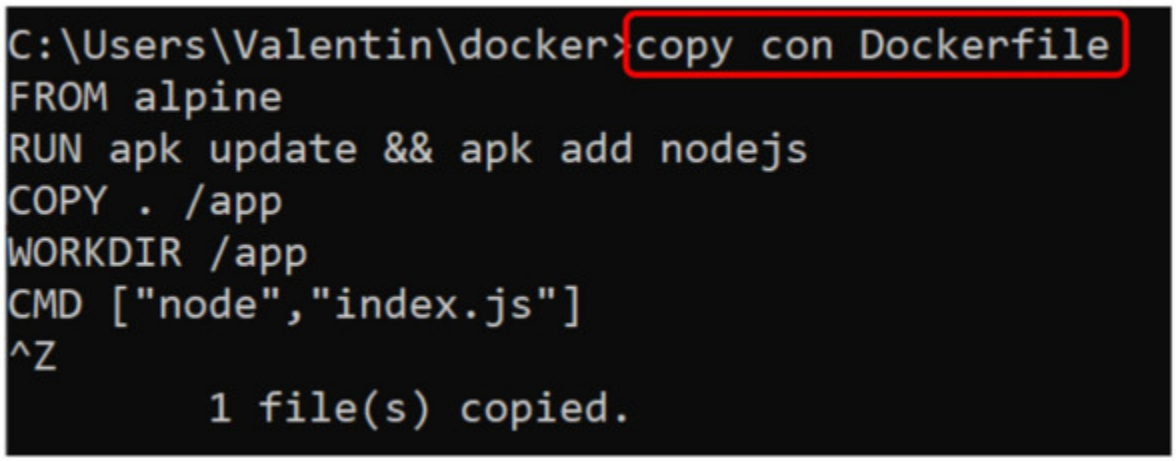


Рис. Создаём **Dockerfile**

Здесь можно увидеть основные разделы файла **Dockerfile**.

FROM: мы строим новый образ на основе стандартного образа Alpine Linux

RUN: «поверх» образа Alpine мы запускаем дополнительные команды: **apk update** для обновления базы установочных файлов и **apk add** для установки **Node. js**. Эти две команды записаны в одну строку и разделены двумя символами «амперсанд». Это означает,

что вторая команда выполняется при условии успешного запуска первой команды. В результате наш образ будет содержать новый. Дополнительный «слой» — поверх стандартного образа **Alpine**.

Задание. Просмотрите статью What does "&&" do и выясните, какие символы используют для объединения команд.

COPY: мы копируем содержимое текущего каталога (обозначен точкой) в каталог **/app** нашего будущего образа. Поэтому в текущем каталоге не должно быть никаких лишних файлов.

WORKDIR: При запуске контейнера вначале произойдёт смена текущего каталога на указанный: **/app**.

CMD: При запуске контейнера будет выполнена указанная команда — запуск нашего приложения.

При установке нового пакета программ в Alpine Linux используют **APK — Alpine Package Keeper**. С этим инструментом желательно немного познакомиться.

Задание. Просмотрите следующие статьи:
https://wiki.alpinelinux.org/wiki/Alpine_Package_Keeper
Wikipedia: List of software package management systems
Список систем управления пакетами программного обеспечения

СОЗДАЁМ ОБРАЗ

Мы проведи подготовку, теперь можем переходить к собственно сборке образа. По нашему сценарию.

Вводим команду **docker build**, см. рис. Здесь указано название образа. Затем стоит точка, что означает «использовать содержимое текущего каталога». Снова мы имеем дело с этой «точкой» — наверное, пора к ней начать привыкать.

Рис. Создание образа

Нам сообщают, что создание образа завершено успешно: **Building... FINISHED**.

Далее среди сообщений мы можем обнаружить слово **layers** — слои. Про эти слои мы ещё поговорим. Пока скажем, что каждое действие над образом фиксируется как отдельный «слой» — геологические слои почвы в археологии.

ЗАПУСКАЕМ КОНТЕЙНЕР

Образ создан, и его можно запустить.

Мы используем команду **docker run**. Здесь в качестве параметра мы указываем имя образа и через двоеточие его **tag**, то есть версию (метку, или «тэг»), см. рис.

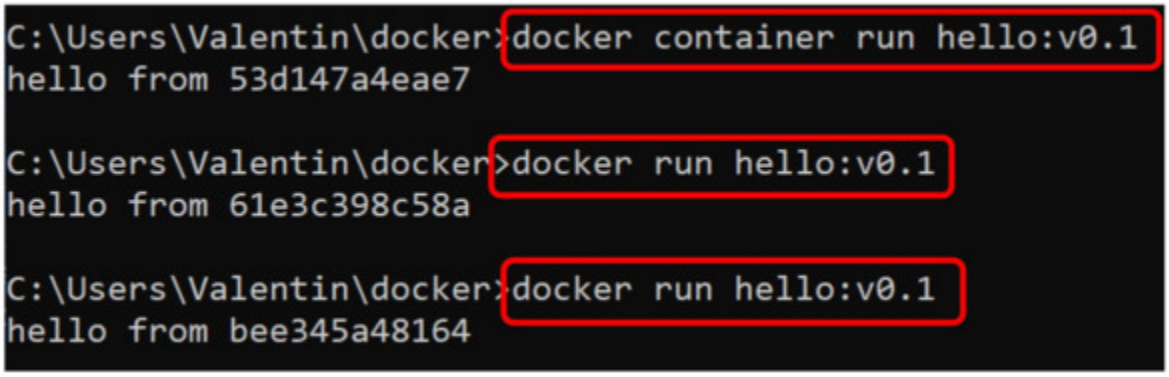


Рис. Запускаем контейнер

ЧТО ТАКОЕ «СЛОЙ» ОБРАЗА

Нам уже встретилось понятие «слой». Слои образа — image layers — это последовательная запись изменений в файловой системе. При выполнении каждого шага, описанного в **Dockerfile**, создаётся очередной слой поверх предыдущего.

Мы можем получить список слоёв и даже узнать затраты дискового пространства на каждый слой.

Вначале определим идентификатор нашего образа, см. рис. Выводим список образов командой:

docker image ls.

В полученном списке находим наш по его названию. Обратим внимание на метку **v0.1** (в нашем случае версию образа), которая теперь указана в колонке **TAG**.

Нас будет интересовать идентификатор образа **IMAGE ID**. Скопируем его в буфер обмена.

```
C:\Users\Valentin\docker>docker image ls
REPOSITORY    TAG       IMAGE ID       CREATED          SIZE
hello         v0.1      49fcf4b70835   57 seconds ago   52.7MB
ourfiglet     latest    95ce02f7f4be   4 days ago       119MB
ubuntu        latest    cdb68b455a14   11 days ago      77.8MB
alpine        latest    9c6f07244728   2 months ago     5.54MB
hello-world   latest    feb5d9fea6a5   13 months ago    13.3kB
```

Рис. Список образов

Теперь, зная идентификатор, можно вывести на экран историю образа в виде списка его слоёв, см. рис. Это именно «история» — в буквальном смысле слова. И соответствующая команда так и звучит:

docker image history.

```
C:\Users\Valentin\docker>docker image history 49fcf4b70835
IMAGE          CREATED BY                                      SIZE
49fcf4b70835   CMD ["node" "index.js"]                         0B
<missing>      WORKDIR /app                                    0B
<missing>      COPY . /app # buildkit                          195B
<missing>      RUN /bin/sh -c apk update && apk a…             47.2MB
<missing>      /bin/sh -c #(nop)  CMD ["/bin/sh"]              0B
<missing>      /bin/sh -c #(nop) ADD file:2a94968…             5.54MB
```

Рис. Список слоёв образа

В списке слоёв можно найти строчку с командами, разделёнными двумя амперсандами. Как видим, эта одна комбинированная строка позволят создать всего один слой, хотя в неё и две команды. Поэтому в **Dockerfile** можно часто встретить двойной амперсанд.

В колонке **SIZE** можно видеть, сколько места на диске занимает каждый слой.

ПРИГЛАШЕНИЕ К ВВОДУ КОМАНД

Далее нам придётся вводить длинные команды. Для удобства работы мы временно «укоротим» приглашение операционной системы к вводу команд. Здесь по умолчанию выводится полный путь к текущему каталогу. На время текущей работы нам эта информация не потребуется.

Используем встроенную команду **PROMPT** — «приглашение к вводу команд». Заметим, что иногда этим словом даже называют саму командную строку.

Вызовем описание команды с помощью ключа **/?**, см. рис.

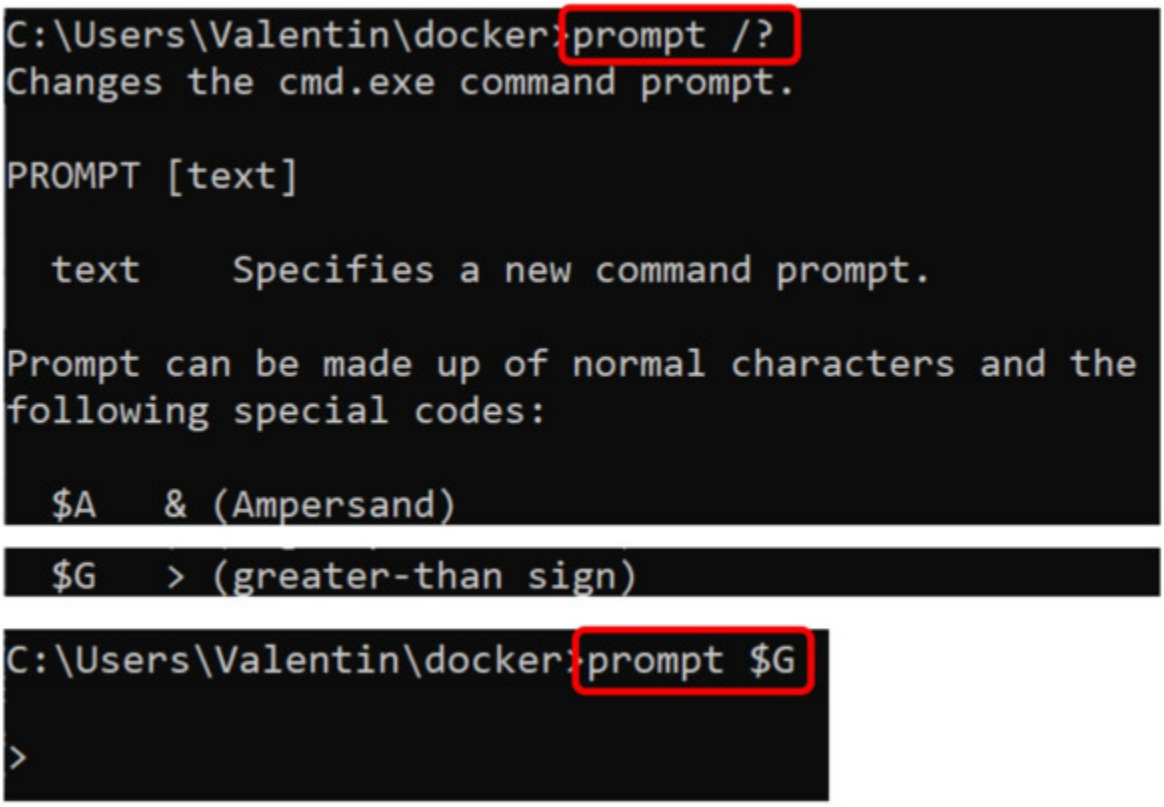


Рис. Приглашение ОС

На интересует значок «**>**». то есть символ «больше». Для него нам нужна комбинация **$G** — от выражения «**greater than**».

Теперь наше приглашение состоит из одного значка и не мешает вводить длинные команды.

ДОБАВЛЯЕМ ОДНУ КОМАНДУ

Следующий шаг: добавить новую команду в нашу программу. Мы выводим содержимое файла на экран командой **TYPE**. Затем добавляем новую строку в конец существующего файла **index. js** командой **ECHO**. Используем два символа**>>**, чтобы дописать строку в конец файла — это операция **append**. Затем снова выводим наш файл на экран, см. рис.

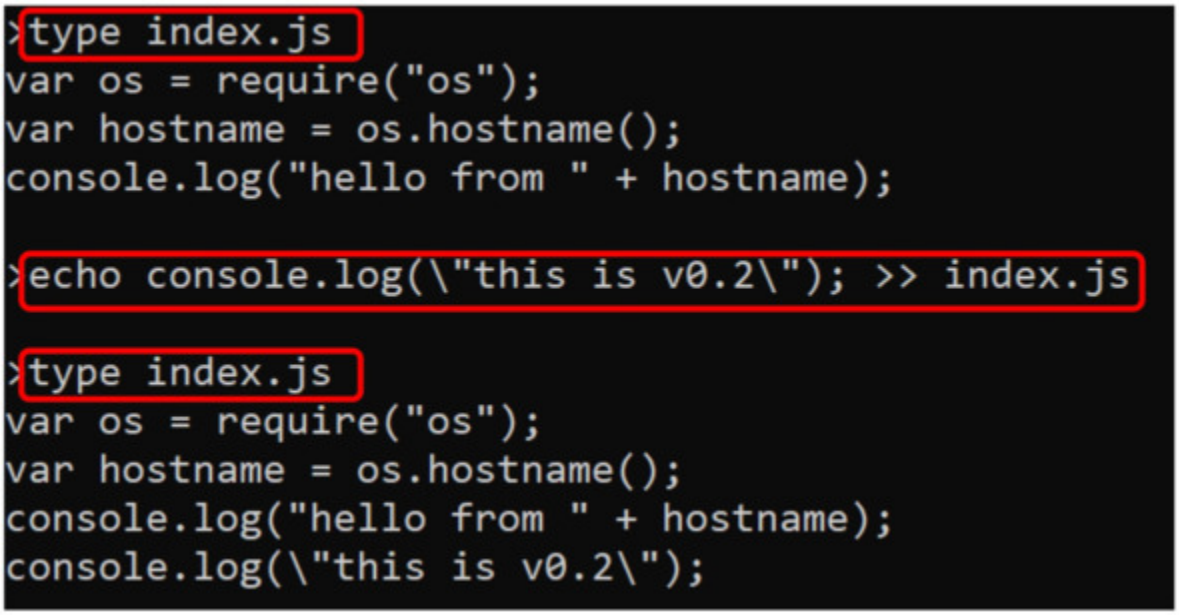


Рис. Добавляем строку в программу

СОЗДАЁМ ВТОРУЮ ВЕРСИЮ ОБРАЗА

Мы изменили нашу программу. Теперь создадим новую версию образа. Для этого запускаем команду **image build**. В качестве метки указываем новую версию: **v0.2**. здесь используется тот же самый **Dockerfile** — безо всяких изменений. Он просто возьмёт содержимое текущего каталога, где находится новый, дополненный вариант наше программы, см. рис.

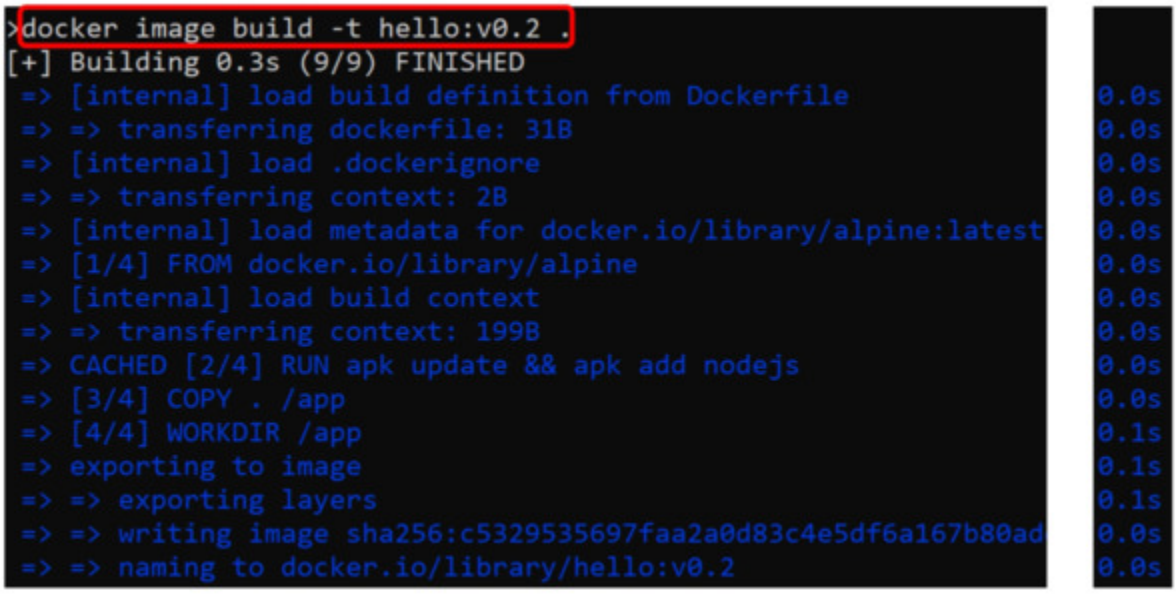


Рис. Вторая версия образа

Выводим на экран список образов, см. рис.

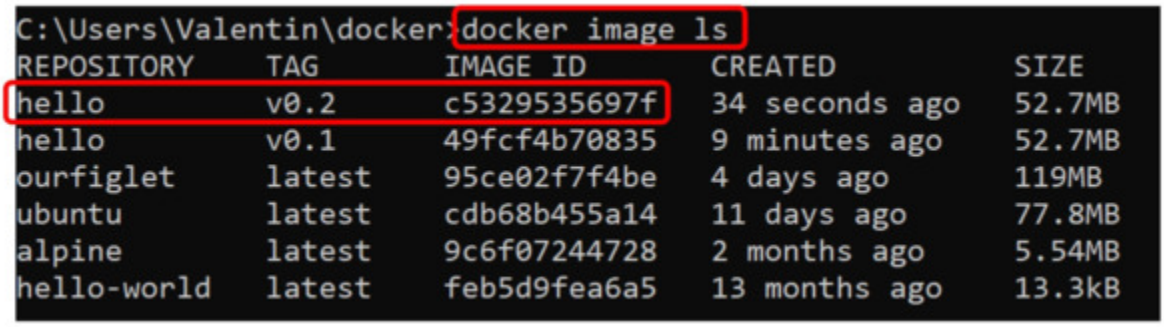


Рис. Список образов

Чтобы сравнить содержимое образов, используем команду **image inspect**, см. рис.

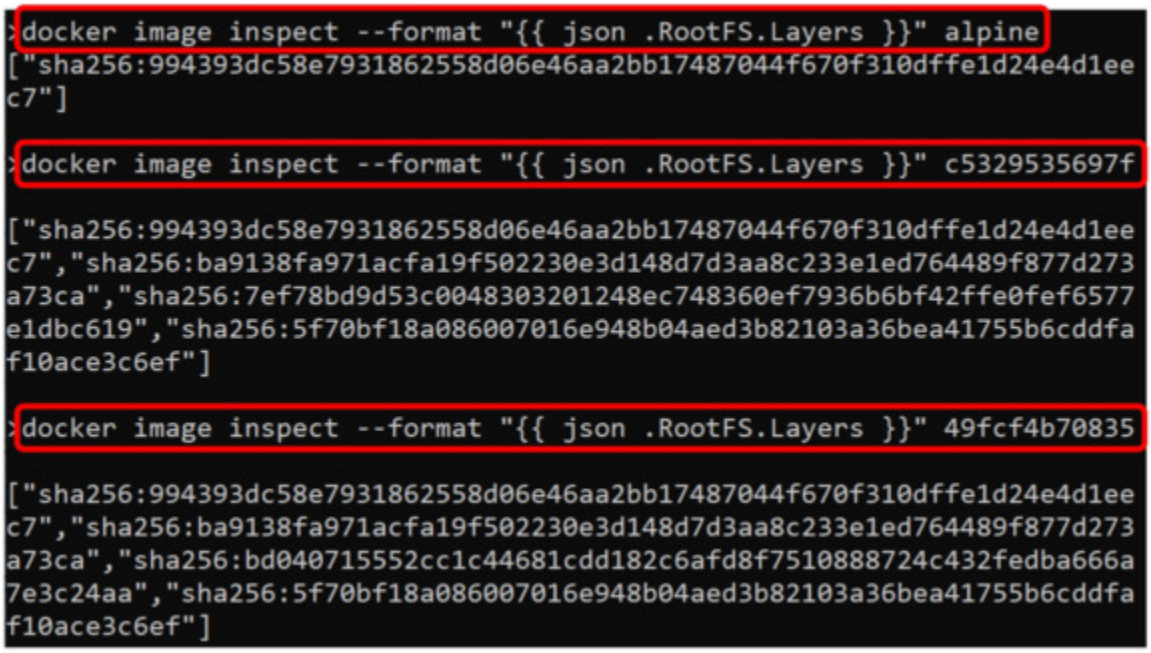


Рис. Содержимое образов

Для вывода интересующей нас информации в удобном формате используем **JSON — JavaScript Object Notation**.

Задание. Просмотрите следующие ссылки

https://www.json.org

https://ru.wikipedia.org/wiki/JSON

https://docs.docker.com/engine/reference/commandline/inspect/

Попутно можно изучить описание команды **image inspect**, см. рис.

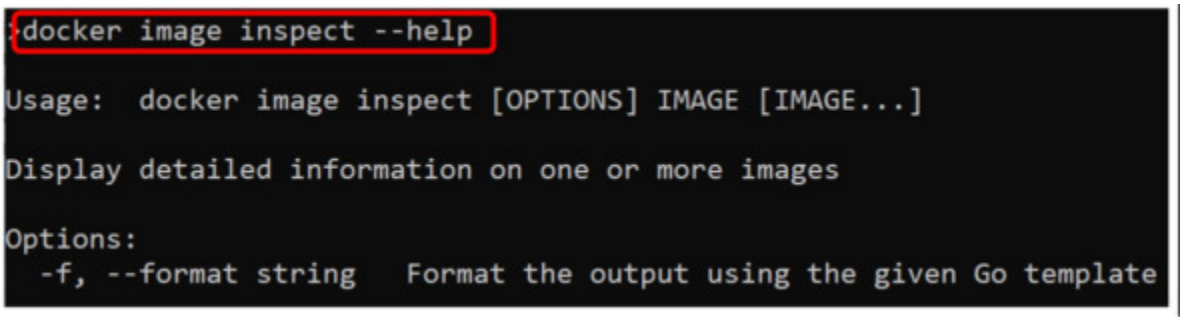


Рис. Описание команды

ОСНОВНЫЕ ТЕРМИНЫ

В данной виртуальной работе появляются следующие базовые термины, см. рис.

Мы уже рассмотрели понятия СЛОЙ и ДОКЕР-ФАЙЛ. Ещё один базовый термин — ЛОГИЧЕСКИЙ ТОМ. В ОС Windows так обычно называют диск, к которому обращаются по его букве, например **C:**. В широком смысле, это может быть каталог с общим доступом.

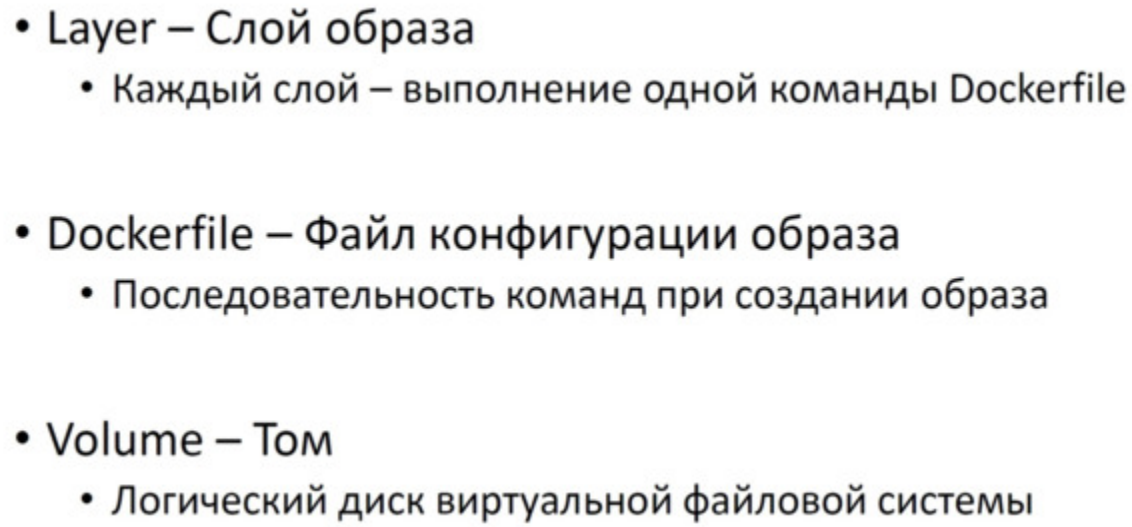


Рис. Основные термины

На этом заканчивается материал второй виртуальной лабораторной работы.

А мы пройдём ещё немного дальше.

ВЕБ-ПРИЛОЖЕНИЕ GETTING STARTED

До этого момента мы запускали контейнеры и работали с ними в терминале. Второй вариант взаимодействия с контейнерами — через веб-интерфейс в браузере.

Запустим следующую команду, см. рис.

docker run -d -p 80:80 docker/getting-started

Здесь есть новые ключи, которые мы ещё не встречали.

Ключ **–d** от выражения **detached mode** — запуск приложения в фоновом режиме. Буквально слово **detach** в данном случае означает «отсоединить, отключить от соединения (с терминалом)».

Ключ **–p** от слова **port** — «порт». Здесь задают отображение, соответствие между программными портами контейнера и основной системы. Это не «разъём на корпусе компьютера», а всего лишь «целое число, определяющее сервисы, доступные на сервере». В нашем примере 80-й порт контейнера «соединяется» с 80-м портом основной системы.

Задание. Просмотрите статью Порт (компьютерные сети) в Википедии. Обратите внимание на раздел **Использование в URL-адресах**.

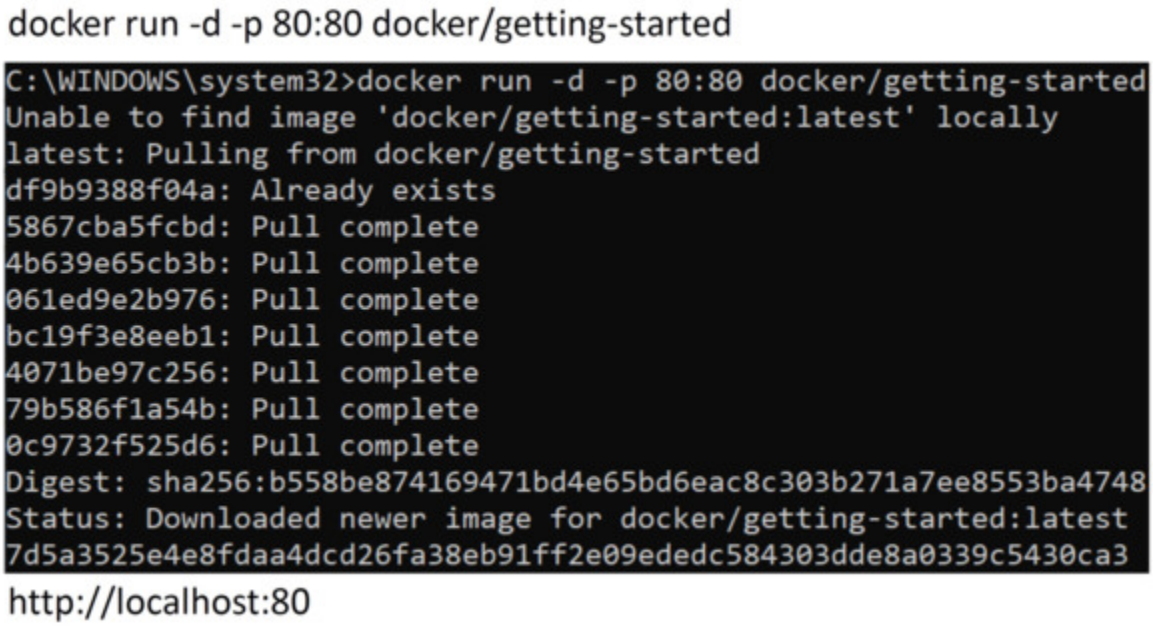


Рис. Запускаем веб-приложение

После запуска контейнера вроде бы ничего не происходит. Затем для подключения к контейнеру мы запускаем браузер и вводим в строке адреса:

http://localhost:80

localhost — это обращение к локальному компьютеру как к веб-серверу. Через двоеточие указан номер порта.

Получаем веб-страницу с описанием Докера: Начало работы — **Getting Started**, см. рис.

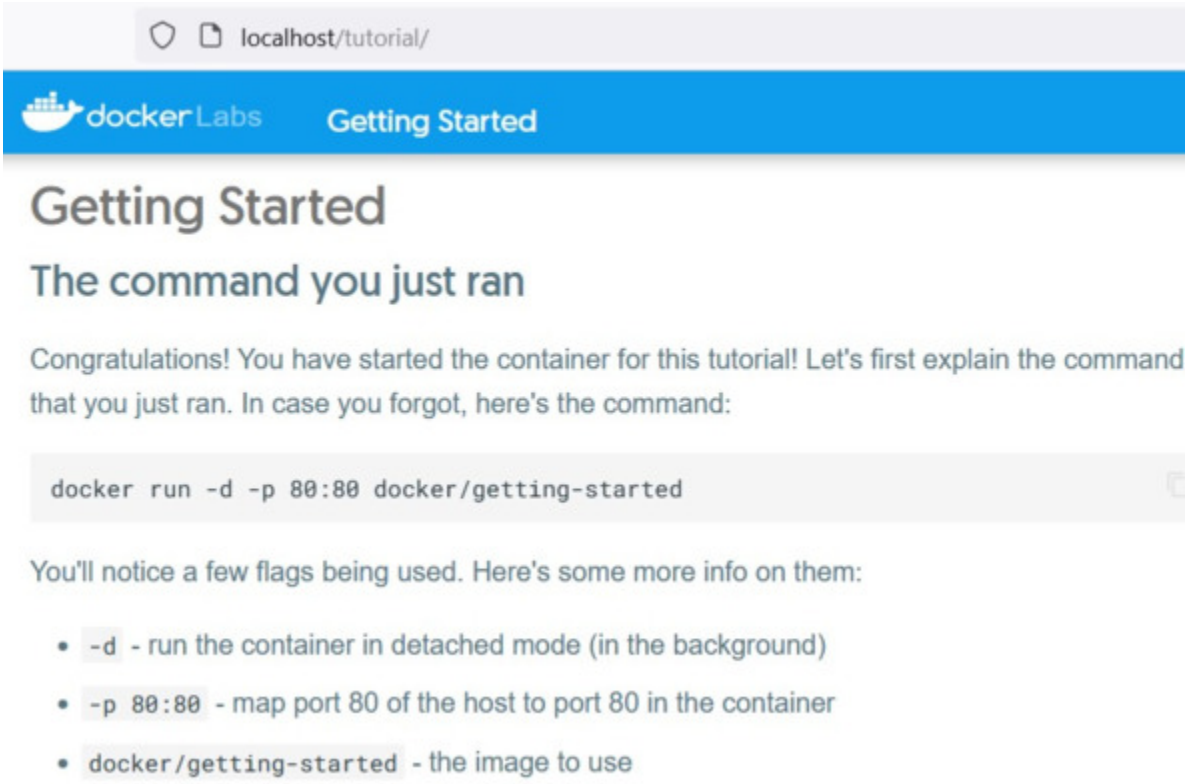


Рис. Веб-приложение

Можно просматривать страницы этого «сайта» и переходить по ссылкам.

Тот же результат можно получить, запустив **Sample Container** — Пример контейнера — через графическое приложение **Docker Desktop**, см. рис.

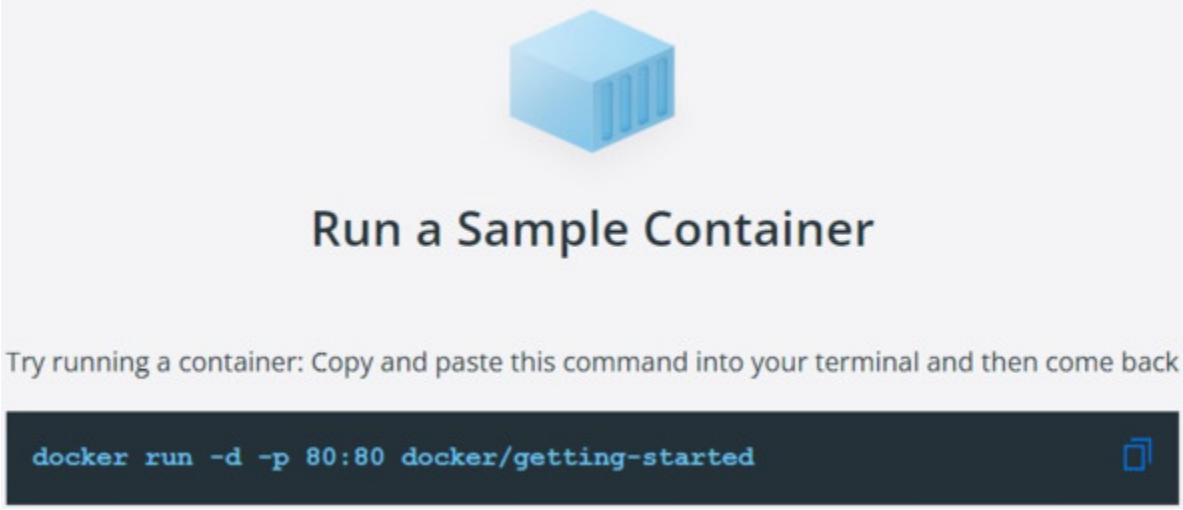


Рис. Запускаем **Sample Container**

Задание. Найдите на Докер- [a,t страницу контейнера: **docker/getting-started**

UBUNTU + MIDNIGHT COMMANDER

Теперь запустим Linux Ubuntu и немного с ним поэкспериментируем.

Для начала найдём страничку образа Ubuntu на Docker Hub, см. рис.

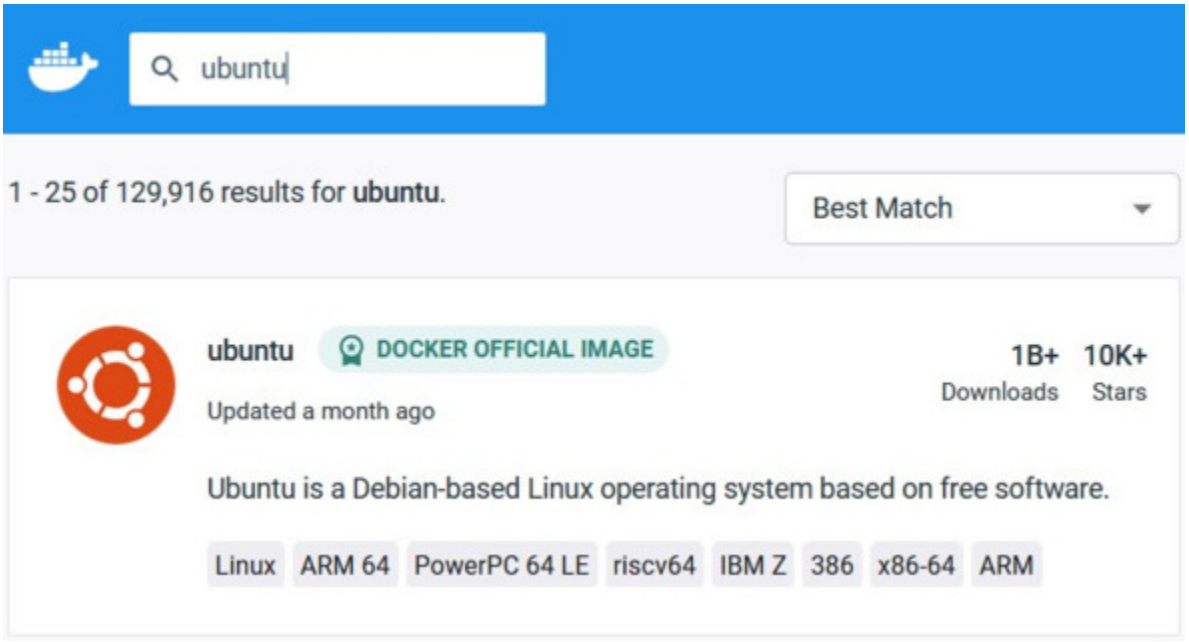


Рис. Официальный образ Ubuntu

Надеемся, вы уже освоили простейший запуск контейнеров, и вас не удивит следующая команда, см. рис.

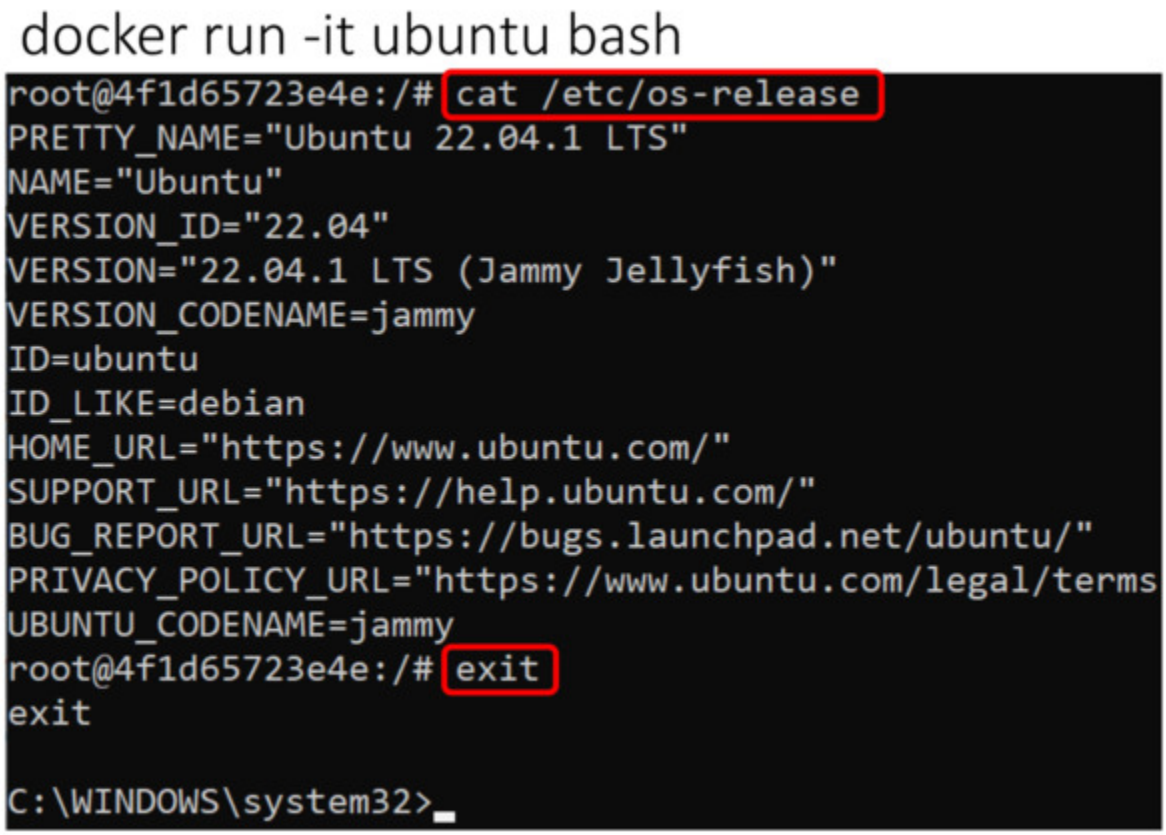


Рис. Запуск Ubuntu в диалоговом режиме

Теперь установим и запустим файловый менеджер **Midnight Commander**:

apt update

apt install mc

mc

Точно те же три команды позволяют установить **MC** в **Ubuntu** для **WSL**. Напомним, что **WSL** — **Windows subsystem for Linux** — это дополнительный инструмент Windows, который поддерживает системные вызовы Linux.

Midnight Commander — это тема нашей 5-й лабораторной работы — файловый менеджер с двумя синими панелями. Мы уже обсуждали его установку в первой части данного пособия.

В данном примере мы устанавливаем MC в контейнере, см. рис.

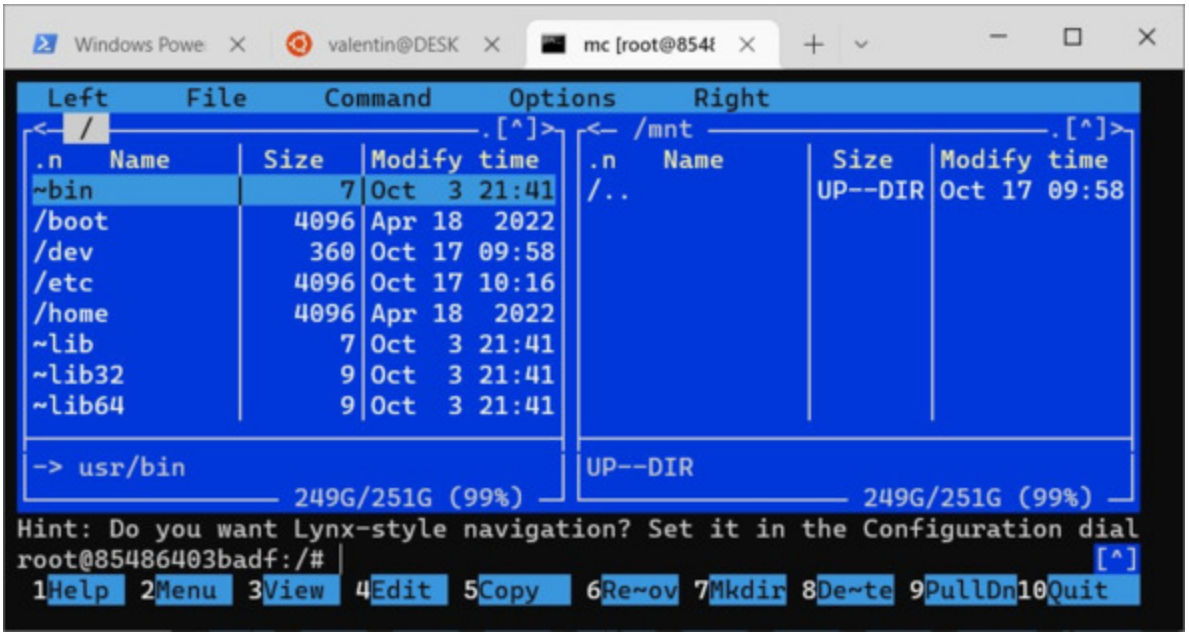


Рис. **Midnight Commander** в контейнере

Доверяем вам самостоятельно сделать самый творческий шаг: соберите новый образ **Ubuntu** с установленным **Midnight Commander** и запустите его.

В заключение просмотрите несколько статей на тему свободно распространяемого программного обеспечения.

Задание. Просмотрите в Википедии следующие статьи:
— APT (software)
— Advanced Packaging Tool
— Dependency hell
— DLL Hell
— GNU

Задание. Найдите на сайте https://www.gnu.org/ следующие сведения:
— free software respects users' freedom
— «free speech» vs. «free beer»

— four essential freedoms

Задание. Найдите на сайте **Free Software Foundation (FSF)** https://www.fsf.org описание понятия **Copyleft**.

ЛИТЕРАТУРА

Моуэт Э. Использование Docker. 2017
https://e.lanbook.com/book/93576

Сейерс Э. Х., Милл А. Docker на практике. 2020
https://e.lanbook.com/book/131719

Кочер П. С. Микросервисы и контейнеры Docker. 2019
https://e.lanbook.com/book/123710

Арьков В. Ю. Командная строка CMD: Учебное пособие. — [б. м.]: Издательские решения, 2021. — 122 с.
https://ridero.ru/books/komandnaya_stroka_cmd/

Арьков В. Ю. Операционные системы: Учебное пособие: Издательские решения, 2021. — 158 с.
https://ridero.ru/books/operacionnye_sistemy/

Арьков В. Ю., Кондратьева О. В. Интерфейс пользователя: Учебное пособие. — [б. м.]: Издательские решения, 2022. — 130 с.
https://ridero.ru/books/interfeis_polzovatelya/